Delfino 개발보드 예제 100선
PART 1

싱크웍스 지음

도서출판 싱크웍스

www.tms320.co.kr

목차

0 실습을 진행하기에 앞서... 3

0.1 DELFINO 개발보드 예제100선 핸드북 – PART 1은? 3
0.2 공통 준비사항 4
0.3 CCS V3.3에서 에뮬레이터 설정/연결/실행 방법 7
0.4 CCS V5에서 에뮬레이터 설정/연결/실행 방법 13

1 GPIO 21

1.1 GPIO 입력으로 쓰기 31
1.2 GPIO 입력, 외부 인터럽트로 사용 33
1.3 GPIO 입력, QUALIFICATION 기능 사용하기 35
1.4 GPIO 출력, LED 점멸 1 37
1.5 GPIO 출력, LED 점멸 2 39
1.6 GPIO 출력, LED 점멸 3 41

2 CPU-TIMER 43

2.1 CPU TIMER 인터럽트를 이용한 LED 주기점멸 45

3 ADC 47

3.1 ADC 1채널, 비-인터럽트(POLLING) 방식 59
3.2 ADC 2채널, 비-인터럽트(POLLING) 방식 61
3.3 ADC 1채널, 인터럽트(INTERRUPT) 방식 63
3.4 ADC 2채널, 인터럽트(INTERRUPT) 방식 65
3.5 ADC 2채널, 동시(SIMULTANEOUS) 모드 운용 67
3.6 ADC 2채널, 병렬 시퀀서(DUAL-SEQUENCER) 모드 운용 71
3.7 8채널, 순차(SEQUENTIAL) ADC 하기 75
3.8 8채널, 동시(SIMULTANEOUS), 병렬(DUAL) ADC 하기 79
3.9 ADC 8채널, MODE 2 인터럽트 결과저장 83
3.10 ADC 1채널, 오버샘플링(OVER-SAMPLING) 하기 89

4 EPWM 93

4.1 PWM 1채널 출력하기 (ASYMMETRIC, SINGLE EDGE) 101

4.2 PWM 1채널 출력하기 (SYMMETRIC, DUAL EDGE) 105
4.3 PWM 1채널 출력하기 (ASYMMETRIC, DUAL EDGE) 109
4.4 PWM 주기적인 인터럽트 생성과 이벤트 분주 113
4.5 PWM 2채널 출력하기 (INDEPENDENT DUTY CYCLE) 119
4.6 PWM 2채널 출력하기 (COMPLEMENTARY) 123
4.7 PWM 2채널에 DEAD-BAND 추가하기 127
4.8 PWM 1채널 CHOPPING 131
4.9 PWM TRIP 기능 확인하기 135
4.10 3상 SINUSOIDAL PWM 출력하기 141
4.11 3상 PHASE-INTERLEAVED PWM 출력하기 147
4.12 2채널 PWM-TO-ANALOG 출력 구현해보기 151

5 ECAP 153

5.1 ECAP 모듈의 EDGE 검출 인터럽트 159
5.2 ECAP 모듈로 펄스의 주파수 및 듀티비율 알아내기 163
5.3 ECAP 모듈을 단상 PWM 발생기로 활용하기 165

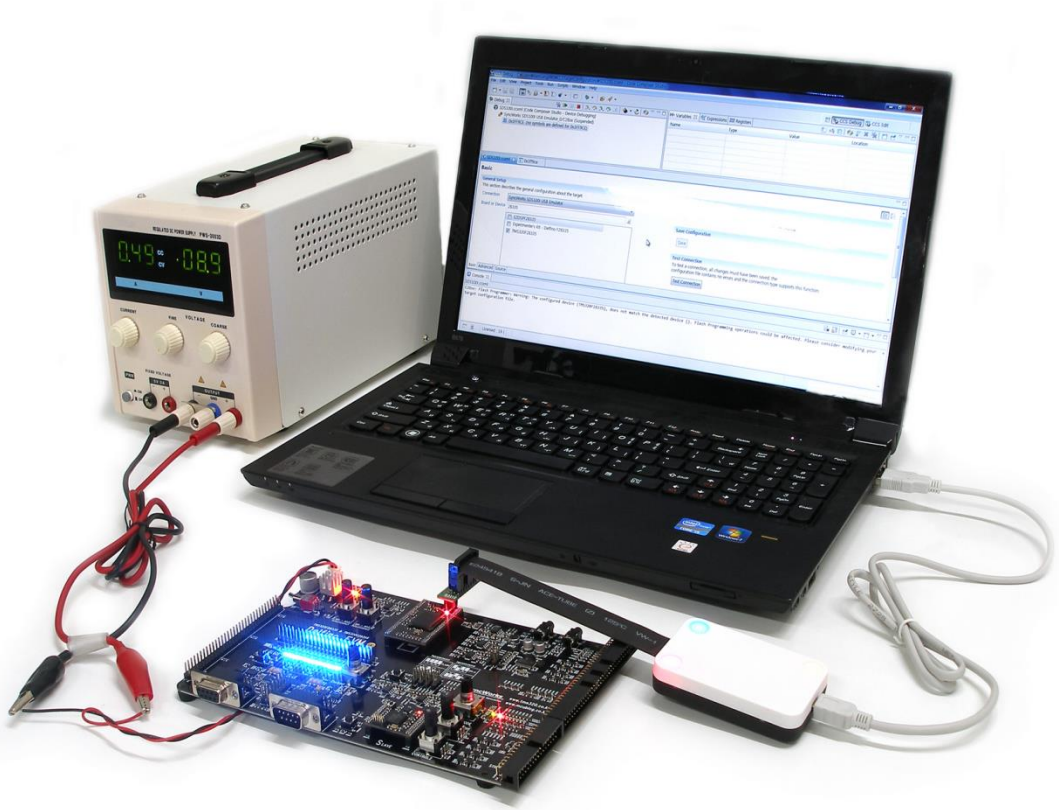
6 EQEP 167

6.1 EQEP 모듈로 INCREMENTAL 엔코더 신호 해석하기 179

0 실습을 진행하기에 앞서...

0.1 Delfino 개발보드 예제100선 핸드북 – PART 1은?

본 책자는 싱크웍스 홈페이지 (www.tms320.co.kr)의 "Delfino 예제 100선" 게시판에 현재까지 올라온 33개 예제에 대한 "Delfino 개발보드 용 예제 설명"을 개발자 분들의 편의를 위해서 책으로 제작한 것입니다. 33번 이후의 예제는 해당 게시판에서 계속 업로드 중입니다. 최신 예제를 다운로드 받으시려면, "Delfino 예제 100선" 게시판을 방문해주세요.



0.2 공통 준비사항

Delfino 개발보드 예제의 모든 실습은 다음과 같은 공통 준비사항 있습니다. 실습 전에 반드시 먼저 준비해 주세요.

A. 실습 장비

Delfino 예제 100선 실습에는 다음과 같은 장비가 필요합니다.

1) CCSv3.3, CCSv4.2 혹은 CCSv5.1가 설치된 PC

CCS는 Code Composer Studio의 약어로 TI 사의 프로세서 개발에 필요한 Editor, Complier, Debugger 등이 포함된 통합 개발 소프트웨어입니다. Window XP/Vista/7 등의 PC에 설치하여 사용합니다.

2) Delfino 개발보드 키트 (예> Delfino 개발모드 + TMS320F28335 초소형 버스 모듈)

Delfino는 TI 사의 TMS320F28x 프로세서 들 중 Floating Point Unit (부동소수연산)을 지원하는 TMS320C2834x/TMS320F2833x 프로세서 계열을 통칭하는 용어 입니다. Delfino 개발보드 키트는 싱크웍스의 TMS320F28335 초소형 버스 모듈 혹은 초소형 메모리 모듈을 탑제한 Delfino 개발보드를 말합니다. 싱크웍스에서 구매 가능합니다.

3) JTAG 에뮬레이터 (예>SDS100i, XDS100S 등등)

JTAG 에뮬레이터는 CCS와 함께 프로세서의 프로그램 다운로드와 디버깅을 지원하는 개발 하드웨어 입니다. PC와 타겟보드를 하드웨어적으로 연결해 줍니다.

B. 실습 예제

싱크웍스 홈페이지에서 "**Delfino 예제 100선 통합 예제 설치 파일**"을 다운로드 하여 설치합니다. Delfino 예제 100선과 관련된 모든 예제를 한번에 설치할 수 있습니다.

> "www.tms320.co.kr >> **고객지원 >> Delfino 예제 100선 >> Delfino EVM**" 게시판에서 "**Delfino 예제 100선 통합 예제 설치 파일**"를 다운로드 받을 수 있습니다.

만일 "**Delfino 개발보드 예제100선 (CD)**"를 가지고 있다면 CD의 다음과 같은 경로에서 통합 예제 설치파일을 찾을 수 있습니다.

> "**CD드라이브₩10 SyncWorks 자료₩00 Delfino 개발보드 예제100선 통합 설치 파일₩**" 폴더에서 "**Delfino_Example100_PART1.exe**" 파일로 예제를 설치할 수 있습니다.

만일 통합 예제 설치 파일이 아닌 개별적으로 예제를 설치하고 싶을 때는 싱크웍스 홈페이지 혹은 Delfino 개발지원 자료집(CD)를 통해서 설치할 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우

> "**CD드라이브₩10 SyncWorks 자료₩30 Delfino 개발보드 관련 예제₩10 Delfino 개발보드 예제 100선 - (진행중)**" 폴더에서 예제를 설치할 수 있습니다.

싱크웍스 홈페이지에서 다운받는 경우

> "www.tms320.co.kr >> **고객지원 >> Delfino 예제 100선 >> Delfino EVM**" 게시판에서 예제를 다운로드 받을 수 있습니다.

Delfino 예제 100선의 모든 예제들은 TI가 제공하는 헤더 및 소스 파일을 기반으로 동작합니다. 따라서, **반드시 실습 전에 "TIDCS_DSP2833x.zip" 파일의 압출을 풀어서 "TIDCS_DSP2833x.exe"를 PC에 설치해 주세요. 한번만 설치하시면 됩니다.** (통합 예제 설치 파일을 설치했다면, 설치하지 않아도 됩니다.)

0.3 CCS v3.3에서 에뮬레이터 설정/연결/실행 방법

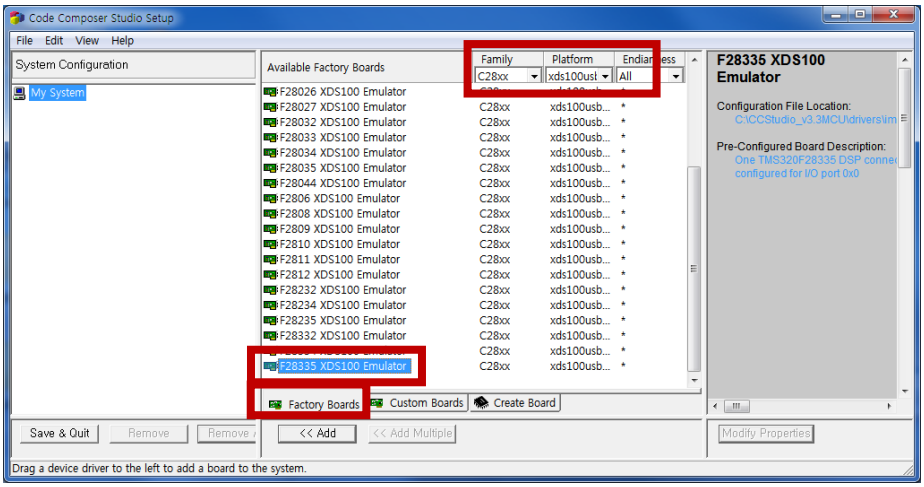
1. 개발환경을 준비합니다.

여기서는 XDS100S JTAG 에뮬레이터와 TMS320F28335 초소형 버스 모듈, Delfino 개발보드를 이용한다고 가정하겠습니다. 사용자는 이어지는 내용을 참고하여 본인의 개발환경에 맞게 설정하셔야 합니다.

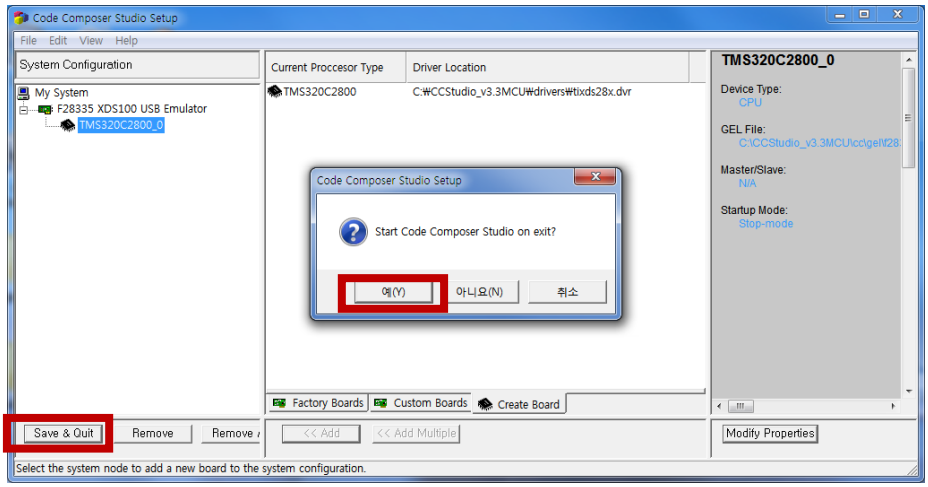
2. Setup CCStudio v3.3을 실행하고, 개발환경을 설정합니다.



바탕화면에서 위와 같은 Setup CCStudio v3.3 아이콘을 클릭하면, 아래와 같은 Code Composer Studio Setup 창이 나타납니다. 중앙 하단의 Factory Boards 탭을 클릭한 후, 컨트롤러 제품 군(**Family : C28xx**)과 JTAG 에뮬레이터(**Platform : xds100usb**)를 선택하면, **Available Factory Boards** 항목에 미리 준비된 설정들이 나타납니다. 나열된 항목들 중에서 **F28335 XDS100 Emulator** 항목을 더블클릭 합니다.

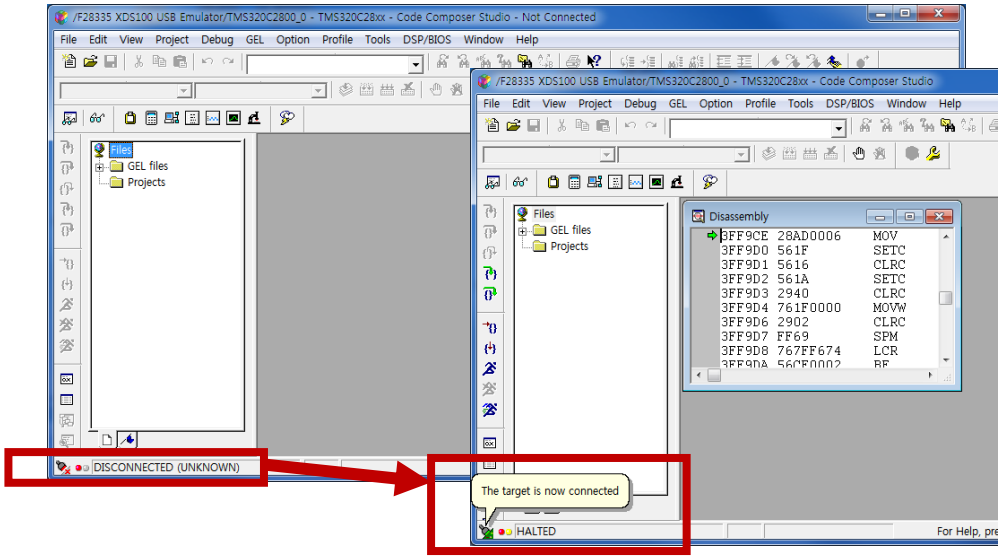


이어서 다음 그림과 같이 설정된 것이 확인되면, **Save & Quit** 버튼을 클릭하고, CCS를 시작할지 묻는 대화 창에서 "**예(Y)**" 버튼을 클릭하여 CCS를 실행합니다.



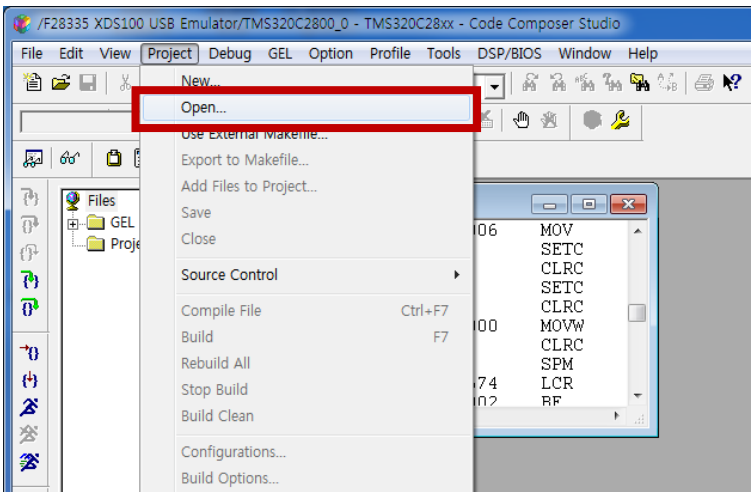
3. CCS가 실행된 것을 확인하고, 타겟과 연결합니다.

CCS가 실행되면 상단 메뉴 **Debug → Reset Emulator**를 선택한 다음, **Debug → Connect**를 선택합니다. CCS가 타겟과 연결되면 다음 그림과 같이 **Disassembly 창**이 나타나고, CCS 화면 하단에 타겟과 연결됐다는 메시지가 나타납니다.

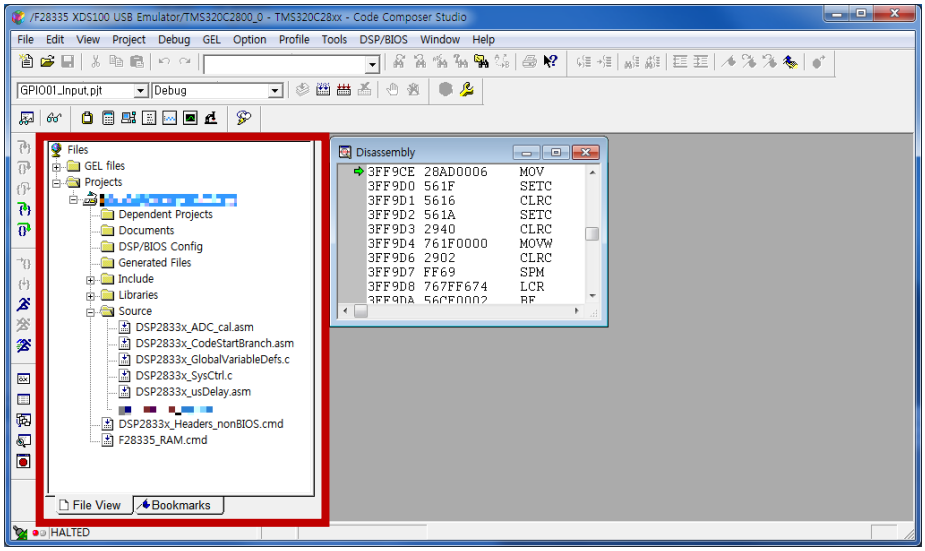


4. CCS가 타겟과 연결된 상태에서 예제 프로젝트를 불러들입니다.

CCS의 상단메뉴 Project → Open을 선택하고,
예제가 설치된 경로에서 프로젝트 파일을 찾아서 불러들입니다.

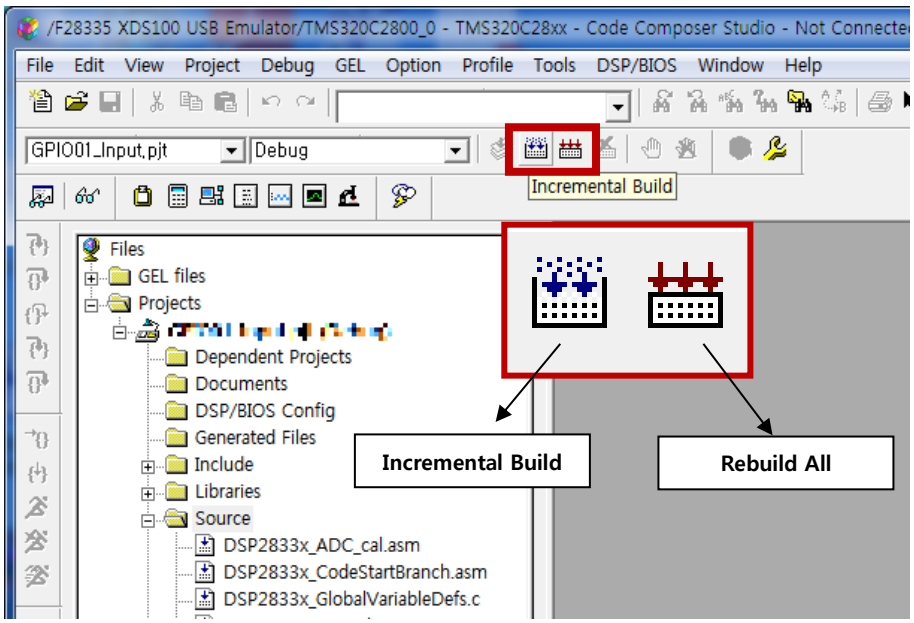


5. CCS 화면의 File view 박스에 예제 프로젝트가 불러들여졌음을 확인합니다.



6. 프로젝트를 Build 합니다.

CCS 상단메뉴 Project → build을 선택하거나, Build 아이콘을 클릭하여 프로젝트를 빌드합니다.



보충:

Rebuild All – 프로젝트 내의 모든 파일들을 재 빌드한다.

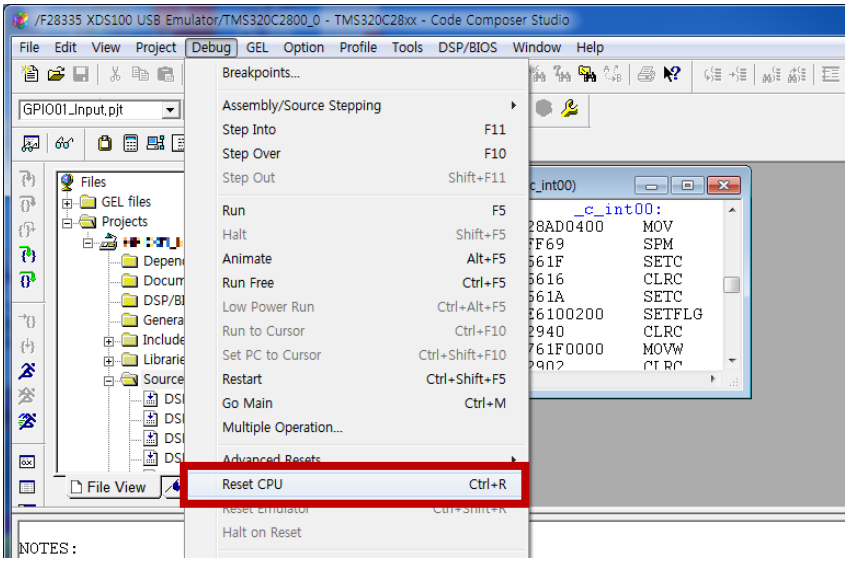
Incremental Build – 프로젝트 내에 수정되거나 새로운 파일만 추가해서 빌드한다.

7. 빌드과정으로 생성된 결과파일을 프로세서에 로드 합니다.

Customize 설정에서 중 Program/Project/CIO 탭의 **Load Program After Build** 옵션을 체크했다면, Build 과정이 완료되면 결과파일이 자동으로 프로세서에 로드 됩니다. **Load Program After Build** 옵션이 체크돼있지 않았다면, CCS 상단메뉴 File → Load Program을 선택하고 예제 프로젝트의 빌드 결과 파일(*.out)을 직접 로드 해야 합니다..

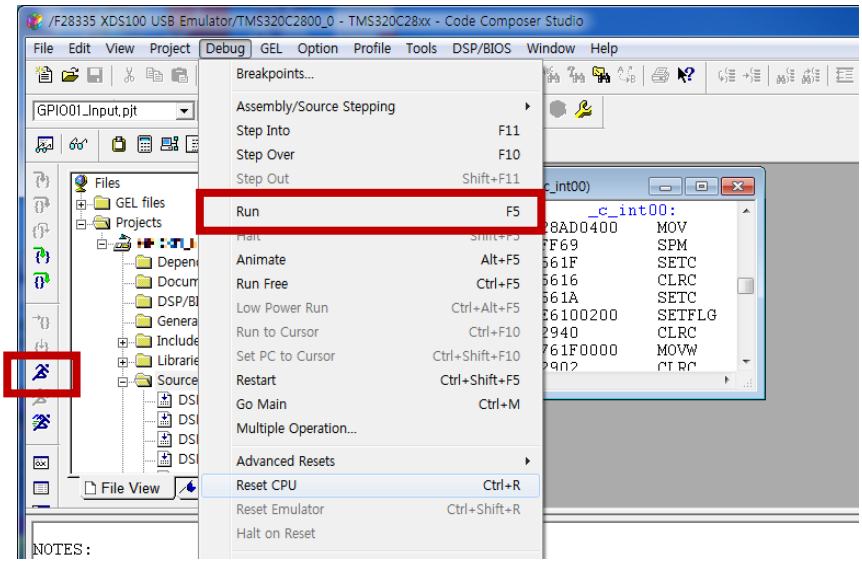
8. 프로세서를 Reset 합니다.

CCS 상단 메뉴 Debug → Reset CPU를 선택합니다.



9. 예제 프로젝트를 실행합니다.

CCS 상단메뉴 Debug → Run을 선택하거나 Run 아이콘을 클릭하여 프로세서에 로드 된 예제 프로그램을 실행하고, 실행 결과를 확인해 봅니다



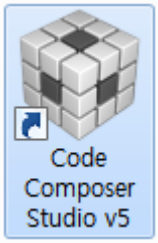
0.4 CCS v5에서 에뮬레이터 설정/연결/실행 방법

1. 개발환경을 준비합니다.

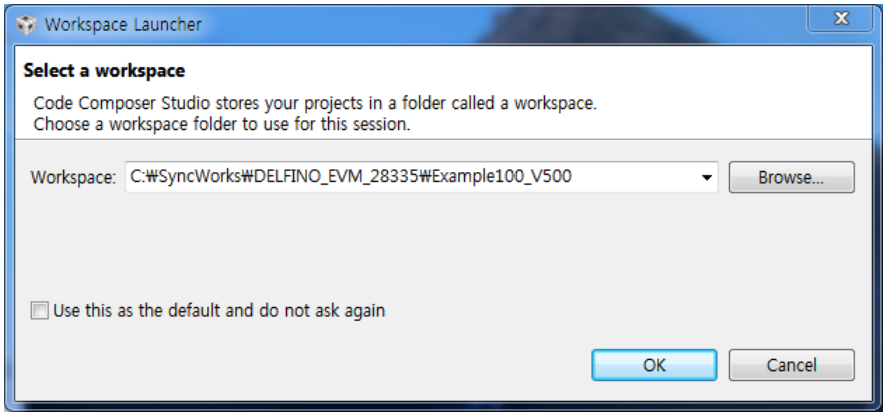
여기서는 XDS100S JTAG 에뮬레이터와 TMS320F28335 초소형 버스 모듈, Delfino EVM을 이용한다고 가정하겠습니다. 사용자는 이어지는 내용을 참고하여 본인의 개발환경에 맞게 설정하셔야 합니다.

2. CCSv5을 실행해서 개발환경을 설정합니다.

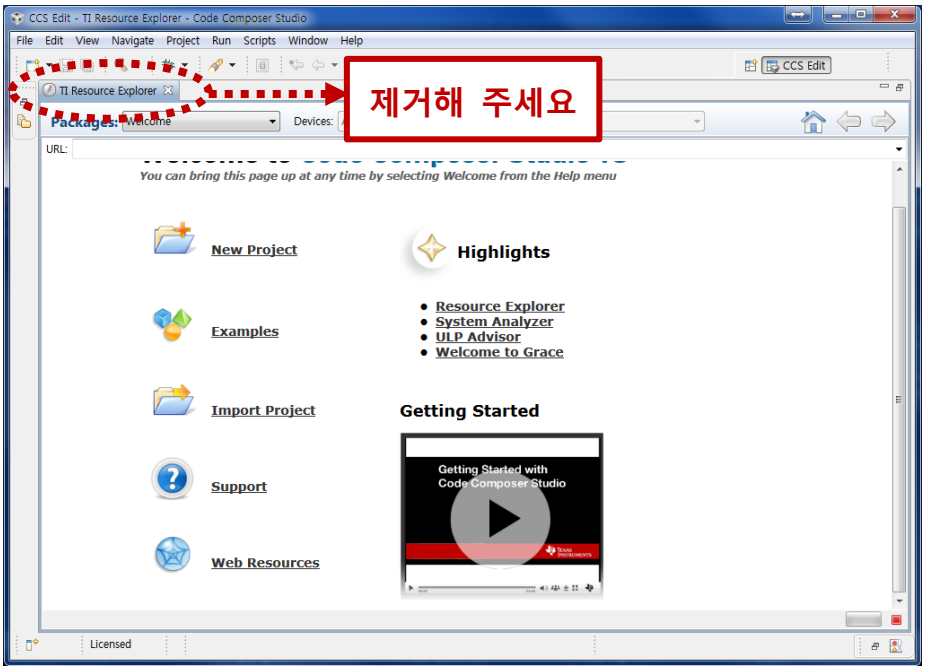
다음과 같은 Code Composer Studio v5 아이콘을 더블 클릭해서 CCSv5 실행합니다.



다음과 같은 Workspace Launcher 창이 나타나면, Browse...버튼을 클릭해서 Workspace 경로를 정당히 설정합니다.

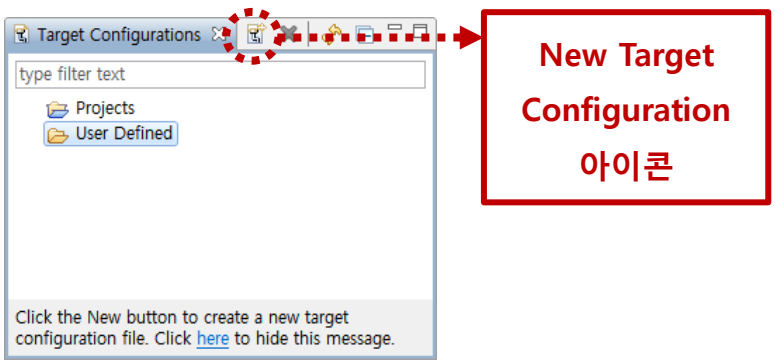


경로를 지정하고 OK 버튼을 클릭하면 다음과 같이 CCSv5가 실행됩니다. CCSv5 내부에서 실행된 "TI Resource Explorer"는 본문 예제에서 사용하지 않음으로 창을 닫아주세요.

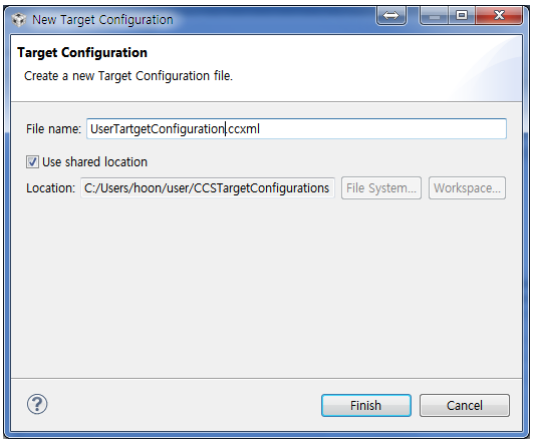


3. CCSv5에서 사용할 에뮬레이터와 타겟 프로세서를 설정합니다.

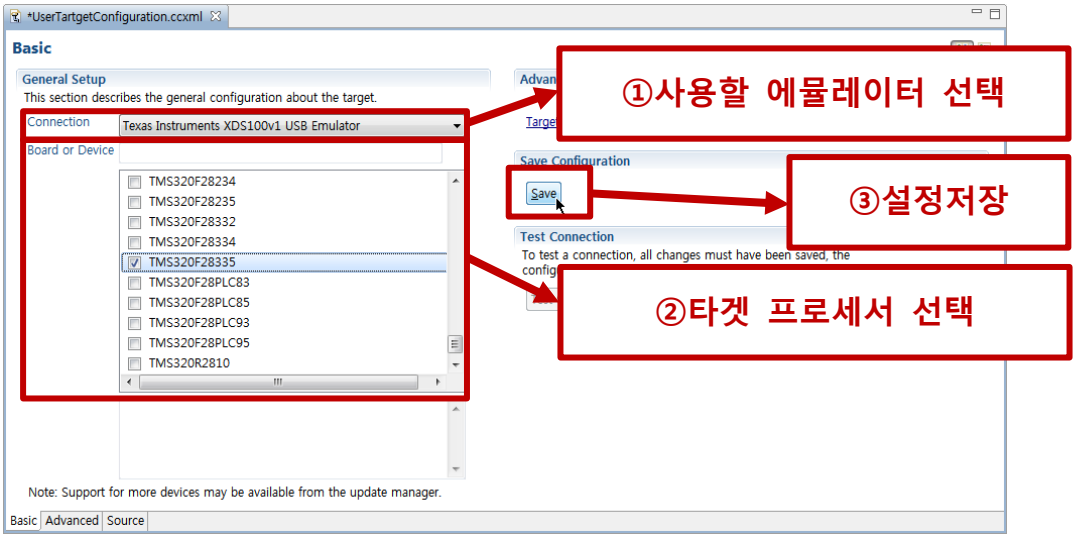
CCS의 상단메뉴 View → Target Configurations 메뉴를 선택합니다. Target Configurations 창이 나타나면 다음과 같이 New Target Configuration File 아이콘을 클릭합니다.



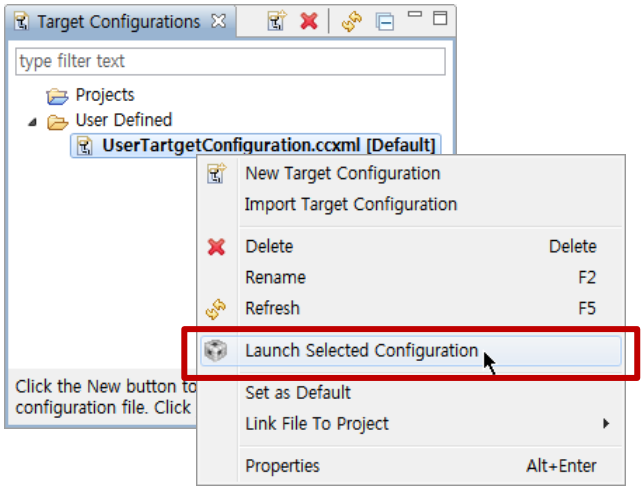
아래 그림과 같은 New Target Configuration 설정 창이 나타나면 File name 을 적당히 설정합니다. 여기서는 UserTargetConfiguration.ccxml 로 설정 해보겠습니다.



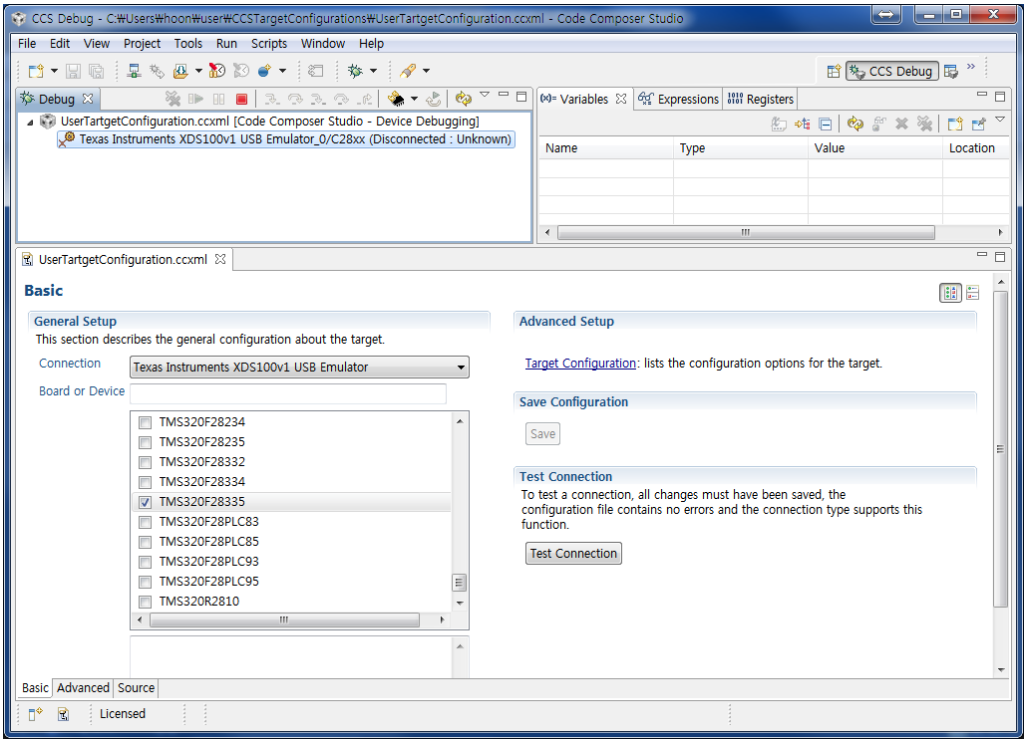
File name 설정을 완료하고 Finish 버튼을 클릭하면 아래와 같이 사용할 에뮬레이터와 타겟 프로세서를 설정하는 창이 나타납니다. 그러면, 사용할 에뮬레이터는 Texas Instruments XDS 100v1 USB Emulator를 선택하고, 타겟 프로세서는 TMS320F28335를 선택합니다. 설정을 완료했으면 Save 버튼을 클릭해서 저장합니다.



설정 저장이 완료 됐으면 다음과 같이 Target Configurations 창에 방금 설정한 UserTargetConfiguration.ccxml 파일이 추가된 것을 볼 수 있습니다.

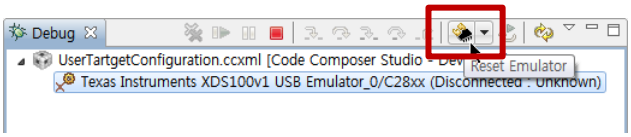


추가된 파일에 마우스 오른쪽 버튼을 클릭 후 Launch Selected Configuration 메뉴를 선택합니다. 그러면 CCS가 다음 그림과 같이 CCS Debug 모드로 전환됩니다.



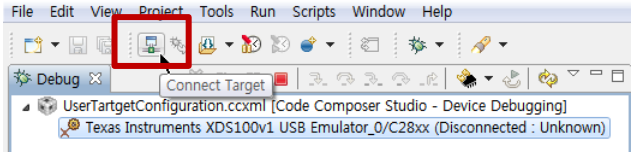
4. 에뮬레이터를 Reset합니다.

CCS 상단메뉴 Run → Reset → Emulator Reset 메뉴를 선택하거나, 다음 그림과 같은 Reset 아이콘을 클릭하여 에뮬레이터를 리셋 합니다.



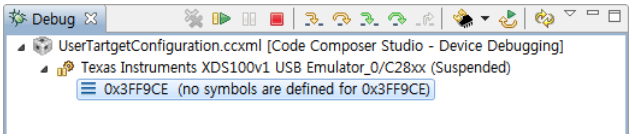
5. 타겟 프로세서와 CCS를 Connect(연결) 합니다.

CCS 상단메뉴 Run → Connect Target 메뉴를 선택하거나, 다음 그림과 같은 Connect Target 아이콘을 클릭하여 타겟 프로세서와 CCS를 연결합니다.



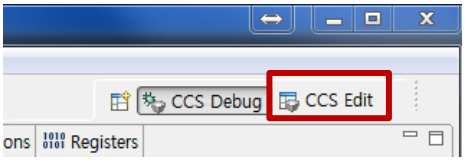
6. 타겟 프로세서와 CCS를 Connect(연결)된 것을 확인합니다.

CCS 좌측 상단의 debug 창이 다음 그림과 같이 나타났다면 타겟 프로세서와의 연결이 성공한 것입니다.



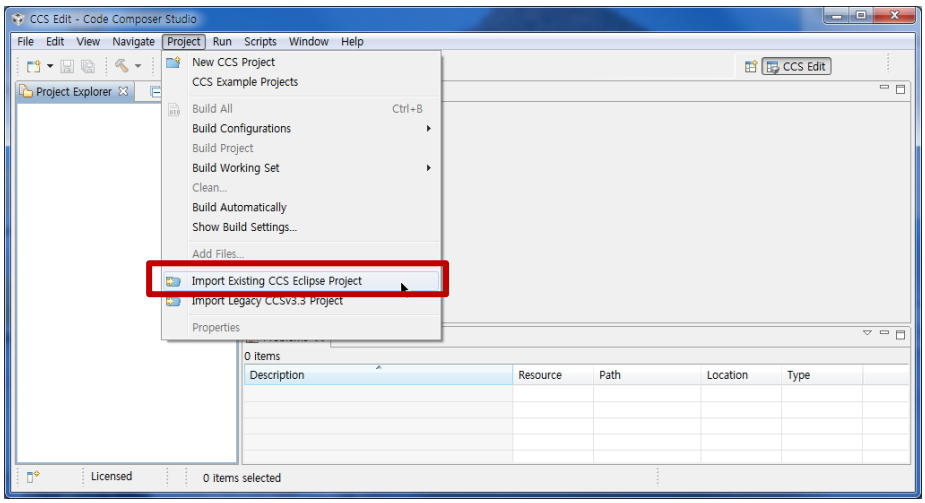
보충:

CCS 우측 상단의 CCS 모드선택에서 CCS Edit 모드를 선택해서 새로운 예제 프로젝트를 불러올 준비를 합니다.

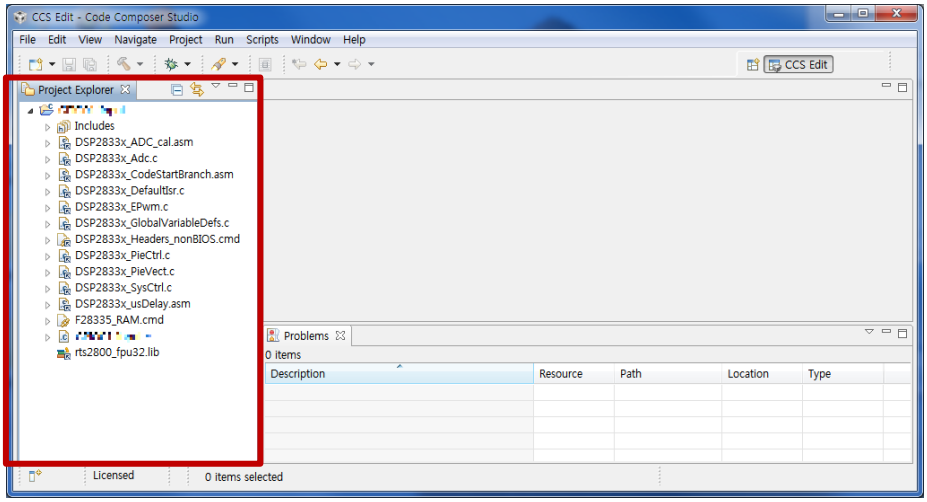


7. CCS가 타겟과 연결된 상태에서 예제 프로젝트를 불러들입니다.

CCS의 상단메뉴 Project → Import Existing CCS/CCE Eclipse Project을 선택하고,
예제가 설치된 경로에서 프로젝트 파일을 찾아서 불러들입니다.

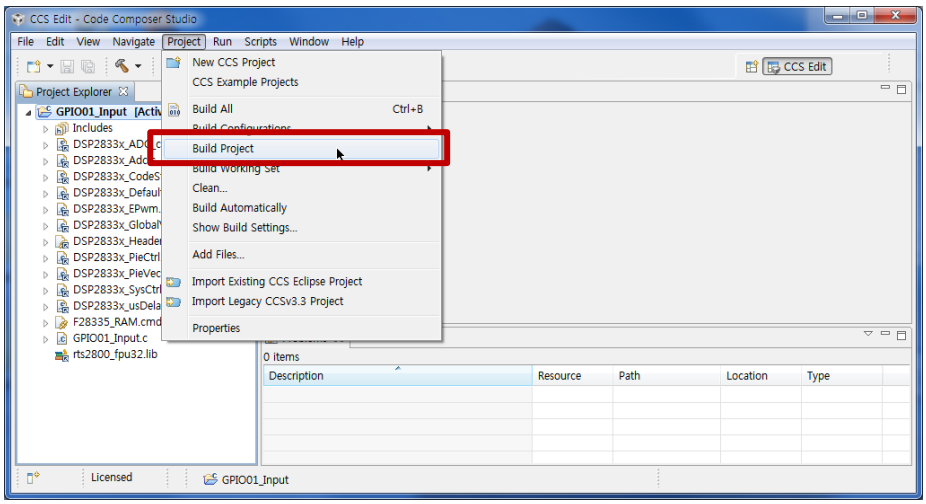


8. CCS 화면의 Project Explorer박스에 예제 프로젝트가 불러들여졌음을 확인합니다.



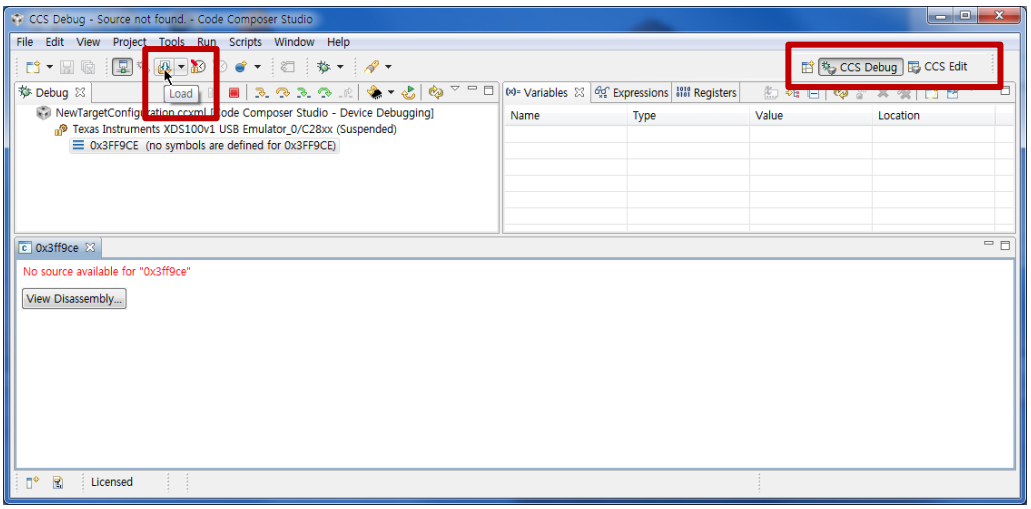
9. 프로젝트를 Build 합니다.

CCS 상단메뉴 Project → build Project을 선택하거나, Build 아이콘을 클릭하여 프로젝트를 빌드 합니다.



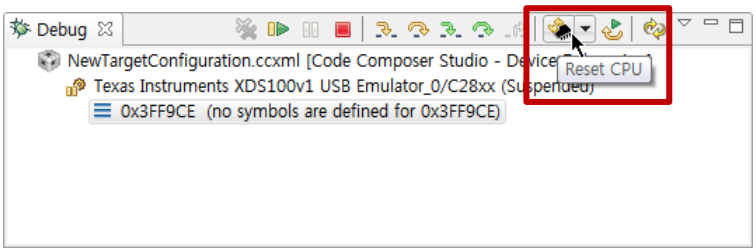
10. 빌드과정으로 생성된 결과파일을 프로세서에 로드 합니다.

타겟과 접속된 상태의 CCS Debug 모드에서 CCS 상단메뉴 Run → Load → Load Program을 선택하거나, Load Program 아이콘을 클릭해서 결과 파일을 프로세서에 로드 합니다.



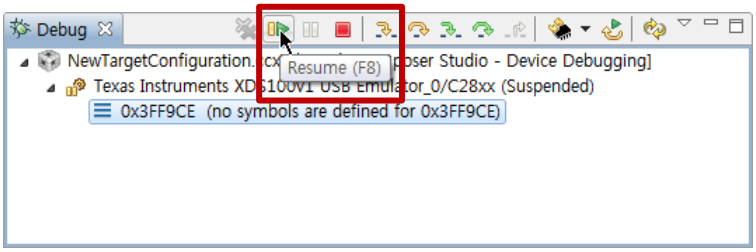
11. 프로세서를 Reset 합니다.

CCS 상단 메뉴 Run → Reset → Reset CPU를 선택하거나, Reset CPU 아이콘을 클릭해서 프로세서를 Reset 합니다.



12. 예제 프로젝트를 실행합니다.

CCS 상단메뉴 Run → Resume을 선택하거나 Resume 아이콘을 클릭하여 프로세서에 로드 된 예제 프로그램을 실행하고, 실행결과를 확인합니다.



1 GPIO

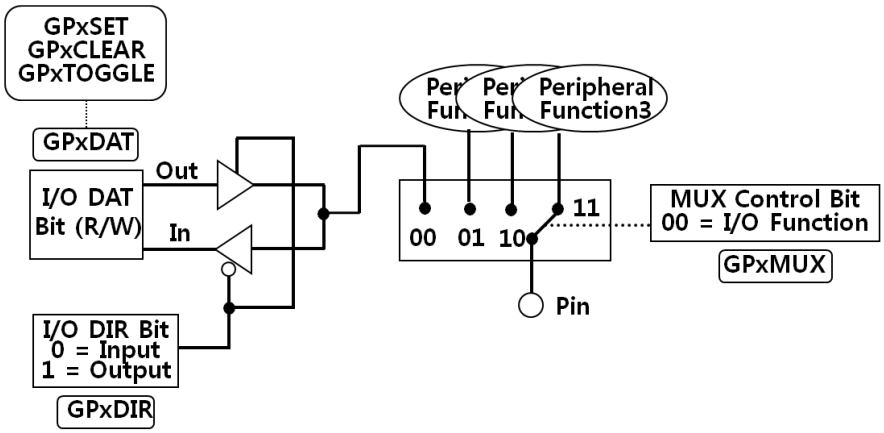
GPIO는 General Purpose Input Output의 약자로 범용적인 목적으로 사용할 수 있는 입출력 포트를 말합니다. Delfino 프로세서는 GPIO 포트를 통해서 3.3V(Logic High), 0V(Logic Low) 신호를 출력하거나, 반대로 외부에서 3.3V(Logic High), 0V(Logic Low) 신호를 입력 받을 수 있습니다.

Delfino 프로세서의 GPIO 특징

- **특수용도 핀을 제외하고는 리셋후 상태는 GPI (Input 상태)**
  - 전원 / 에뮬레이션 / ADC / 클럭
  - 메모리 버스를 GPIO로 사용 가능
- **유연성**
  - 최고 4개 기능 선택 가능
  - 모든 GPIO는 Input / Output 설정 가능
  - 모든 GPIO는 Internal Pull-Up 저항 탑재
  - 리셋 후, GPIO0 ~11핀을 제외한 나머지 모든 핀은 Internal Pull Up 상태
- **노이즈 필터 / Input Qualifier: GPIO 0 ~ GPIO 63에 탑재**
- **자유로운 외부 인터럽트 연결**
  - 외부인터럽트 1 ~ 2 / 외부 NMI : GPIO0 ~ GPIO31 중에서 선택 가능
  - 외부인터럽트 3 ~ 7 : GPIO32 ~ GPIO63 중에서 선택가능
- **모든 GPIO 통상 4mA 드라이브 가능(동시에 여러핀을 구동할 경우 4mA 불가)**

Delfino 프로세서의 GPIO 핀 구조

Delfino 프로세서의 입출력 핀은 모두 최대 4가지 기능을 가지고 있습니다. GPIO 기능은 이중 "0번"에 해당합니다. GPxMUX / GPxDIR / GPxDAT (GPxSET, GPxCLEAR, GPxTOGGLE) 레지스터를 통해서 GPIO를 사용할 수 있습니다. 각 레지스터의 상세한 설명은 Delfino 프로세서의 데이터시트 (System Control and Interrupts Reference Guide)를 참고하기 바랍니다.



Delfino 프로세서 들 중 대표적인 TMS320F28335은 88개 핀을 GPIO로 사용할 수 있습니다. 이들은 다음과 같이 각각 GPIO A MUX, GPIO B MUX, GPIO C MUX 포트로 나뉘어 관리 됩니다.

GPIO A
GPAMUX1: GPIO 0 ~ GPIO 15 (16pin)
GPAMUX2: GPIO 16 ~ GPIO 31 (16pin)

GPIA B
GPBMUX1: GPIO 32 ~ GPIO 47 (16pin)
GPBMUX2: GPIO 48 ~ GPIO 63 (16pin)

GPIA C
GPCMUX1: GPIO 64 ~ GPIO 79 (16pin)
GPCMUX2: GPIO 80 ~ GPIO 87 (8pin)

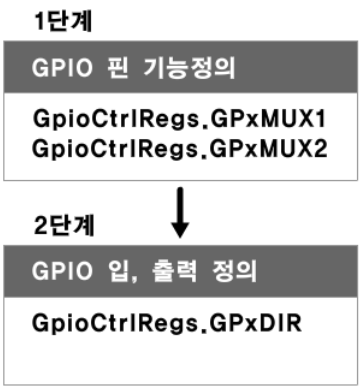
다음 그림은 TMS320F28335의 GPAMUX1, GPAMUX2 에서 설정 가능한 핀 기능 예시입니다. GPIO1를 예로 든다면, 보시는 것처럼 GPIO1 / EPWM1B / ECAP6 / MFSRB 와 같이 총 4가지 기능을 가지고 있는 것을 알 수 있습니다. 만일 사용자가 GPIO1 기능을 사용하고 싶다면, GPAMUX1 레지스터의 3,2번 비트를 각각 0,0으로 설정하면 됩니다. 만일 EPWM1B 기능을 사용하고 싶다면, GPAMUX1레지스터의 3,2번 비트를 각각 0,1으로 설정하면 됩니다.

GPIOA MUX

	Default at Reset Primary I/O Function	Peripheral Selection	Peripheral Selection 2	Peripheral Selection 3
GPAMUX1 Register Bits	**(GPAMUX1 bits = 00)**	**(GPAMUX1 bits = 01)**	**(GPAMUX1 bits = 10)**	**(GPAMUX1 bits = 11)**
1-0	GPIO0	EPWM1A (O)	Reserved[1]	Reserved[1]
3-2	GPIO1	EPWM1B (O)	ECAP6 (I/O)	MFSRB (I/O)[1]
5-4	GPIO2	EPWM2A (O)	Reserved[1]	Reserved[1]
7-6	GPIO3	EPWM2B (O)	ECAP5 (I/O)	MCLKRB (I/O)[1]
9-8	GPIO4	EPWM3A (O)	Reserved[1]	Reserved[1]
11-10	GPIO5	EPWM3B (O)	MFSRA (I/O)	ECAP1 (I/O)
13-12	GPIO6	EPWM4A (O)	EPWMSYNCI (I)	EPWMSYNCO (O)
15-14	GPIO7	EPWM4B (O)	MCLKRA (I/O)	ECAP2 (I/O)
17-16	GPIO8	EPWM5A (O)	CANTXB (O)	$\overline{\text{ADCSOCAO}}$ (O)
19-18	GPIO9	EPWM5B (O)	SCITXDB (O)	ECAP3 (I/O)
21-20	GPIO10	EPWM6A (O)	CANRXB (I)	$\overline{\text{ADCSOCBO}}$ (O)
23-22	GPIO11	EPWM6B (O)	SCIRXDB (I)	ECAP4 (I/O)
25-24	GPIO12	$\overline{\text{TZ1}}$ (I)	CANTXB (O)	MDXB (O)
27-26	GPIO13	$\overline{\text{TZ2}}$ (I)	CANRXB (I)	MDRB (I)
29-28	GPIO14	$\overline{\text{TZ3/XHOLD}}$ (I)	SCITXDB (O)	MCLKXB (I/O)
31-30	GPIO15	$\overline{\text{TZ4/XHOLDA}}$ (O)	SCIRXDB (I)	MFSXB (I/O)
GPAMUX2 Register Bits	**(GPAMUX2 bits = 00)**	**(GPAMUX2 bits = 01)**	**(GPAMUX2 bits = 10)**	**(GPAMUX2 bits = 11)**
1-0	GPIO16	SPISIMOA (I/O)	CANTXB (O)	$\overline{\text{TZ5}}$ (I)
3-2	GPIO17	SPISOMIA (I/O)	CANRXB (I)	$\overline{\text{TZ6}}$ (I)
5-4	GPIO18	SPICLKA (I/O)	SCITXDB (O)	CANRXA (I)
7-6	GPIO19	$\overline{\text{SPISTEA}}$ (I/O)	SCIRXDB (I)	CANTXA (O)
9-8	GPIO20	EQEP1A (I)	MDXA (O)	CANTXB (O)
11-10	GPIO21	EQEP1B (I)	MDRA (I)	CANRXB (I)
13-12	GPIO22	EQEP1S (I/O)	MCLKXA (I/O)	SCITXDB (O)
15-14	GPIO23	EQEP1I (I/O)	MFSXA (I/O)	SCIRXDB (I)
17-16	GPIO24	ECAP1 (I/O)	EQEP2A (I)	MDXB (O)
19-18	GPIO25	ECAP2 (I/O)	EQEP2B (I)	MDRB (I)
21-20	GPIO26	ECAP3 (I/O)	EQEP2I (I/O)	MCLKXB (I/O)
23-22	GPIO27	ECAP4 (I/O)	EQEP2S (I/O)	MFSXB (I/O)
25-24	GPIO28	SCIRXDA (I)	$\overline{\text{XZCS6}}$ (O)	$\overline{\text{XZCS6}}$ (O)
27-26	GPIO29	SCITXDA (O)	XA19 (O)	XA19 (O)
29-28	GPIO30	CANRXA (I)	XA18 (O)	XA18 (O)
31-30	GPIO31	CANTXA (O)	XA17 (O)	XA17 (O)

(1) The word "Reserved" means that there is no peripheral assigned to this GPxMUX1/2 register setting. Should it be selected, the state of the pin will be undefined and the pin may be driven. This selection is a reserved configuration for future expansion.

GPIO 사용방법



왼쪽 그림은 GPIO를 사용하기 위한 방법을 2단계로 간단히 보여 주고 있습니다 1단계에서 GPIO 기능을 사용하기 위해서 GPxMUX1 혹은 GPxMUX2 레지스터를 설정합니다. (여기서, x는 A or B or C 입니다.) 핀 기능이 GPIO로 설정됐다면, 다음 2단계에서는 GPIO를 입력으로 사용할지 출력으로 사용할지 GPxDIR 레지스터를 선택합니다. 이외에도 GPIO 핀의 내부 Pull-up 상태, 혹은 자격심사(Qualification) 등을 각각 GPxPUD(Pull-up 상태 결정) 레지스터나 GPxQSEL(Qualification 옵션 결정) 레지스터를 통해 설정해줄 수 있지만, 중요한 것은 위의 1,2단계 설정입니다.

◈ **GPxMUX1/GPxMUX2 레지스터** (여기서, x는 A or B or C 임.)
'GPIO MUX Control Register'의 약자입니다. 사용법은 해당 비트에 0을 써 넣으면 핀을 GPIO로 사용하는 것이고, 1을 써 넣으면 핀을 GPIO가 아닌 다른 기능(ex. EPWM1B)으로 사용하겠다는 뜻입니다. 실제 코드상에서는 다음과 같이 설정하면 됩니다.

```
GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 0;
GpioCtrlRegs.GPAMUX2.bit.GPIO16 = 2;
```

이것은 GPIO A 포트에 있는 GPIO 1번 핀을 GPIO로 사용하고, GPIO 16번 핀을 CANTXB 기능으로 사용 하겠다고 설정한 예 입니다.

◈ **GPxDIR 레지스터** (여기서, x는 A or B or C 임.)
'GPIO Direction Control Register'의 약자입니다. GPIO의 입출력을 정의하는 레지스터로 해당 비트에 0을 써 넣으면 "입력", 1을 써 넣으면 "출력"으로 설정됩니다. 실제 코드상에서는 다음과 같이 설정하면 됩니다.

```
GpioCtrlRegs.GPADIR.bit.GPIO0 = 0;
GpioCtrlRegs.GPADIR.bit.GPIO1 = 1;
```

이것은 GPIO 0번 핀은 GPIO 입력으로, GPIO 1번 핀은 GPIO 출력으로 사용하겠다고 설정한 예 입니다.

◈ **Protected**

레지스터 중에는 Protected. 즉 보호처리 된 영역이 있습니다. 이는 조작에 주의를 요하는 레지스터들을 보호하기 위한 일종의 안전 장치입니다. Protected 된 영역의 레지스터를 조작할 때는 항상 EALLOW란 코드로 보호를 해제하고, 레지스터를 설정한 다음, 끝나면 다시 보호를 사용하기 위해서 EDIS란 코드를 사용해야 합니다. 만약 Protected 된 레지스터에 EALLOW나 EDIS를 사용하지 않으면 레지스터 설정이 제대로 먹히지 않습니다. 참고로 GpioCtrlRegs 그룹의 레지스터들은 모두 Protected 되어있습니다. (설정해야 하는 레지스터의 Protected 여부는 데이터 시트의 레지스터 설명에 표시되어 있습니다.)

Protected를 고려해서 "A포트의 2핀은 GPIO 입력으로, 2핀은 GPIO4핀은 출력으로 설정"한다면 다음과 같이 코딩 할 수 있습니다.

```
EALLOW;                                 // Protected 된 영역의 보호 해제
GpioCtrlRegs.GPAMUX1.bit..GPIO0 = 0;    // GPIO0~3 핀 정의- GPIO로 사용
GpioCtrlRegs.GPAMUX1.bit..GPIO1 = 0;
GpioCtrlRegs.GPAMUX1.bit..GPIO2 = 0;
GpioCtrlRegs.GPAMUX1.bit..GPIO3 = 0;
GpioCtrlRegs.GPADIR.bit.GPIO0 = 0;      // GPIOA0~1 입력, GPIOA2~3 출력
GpioCtrlRegs.GPADIR.bit.GPIO1 = 0;
GpioCtrlRegs.GPADIR.bit.GPIO2 = 1;
GpioCtrlRegs.GPADIR.bit.GPIO3 = 1;
EDIS;                                   // Protected 된 영역의 보호 설정.
```

혹은 다음과 같이 코딩 할 수도 있습니다. 위의 코딩은 레지스터의 비트 별로 접근해서 설정한 것이고, 아래 코드는 레지스터 전체에 접근해서 설정한 것입니다.

```
EALLOW;                                 // Protected 된 영역의 보호 해제
GpioCtrlRegs.GPAMUX1.all = 0x00000000;  // GPIO0~3 핀 정의- GPIO로 사용
GpioCtrlRegs.GPADIR.all = 0x00000000C;  // GPIOA0~1 입력, GPIOA2~3 출력
EDIS;                                   // Protected 된 영역의 보호 설정.
```

여기까지는 GPIO를 설정하는 방법입니다. 다음은 GPIO를 제어하는 방법을 알아야 합니다. 만일 GPIO로 입력으로 설정했다면, 입력된 신호의 Low, High 기록하는 레지스터가 필요하고, 반대로 출력으로 설정했다면, 출력 신호의 Low, High를 설정하는 레지스터가 필요할 것입니다.

◈ **GPxDAT 레지스터** (여기서, x는 A or B or C 임.)
'GPIO Data Register'의 약자입니다. GpioCtrlRegs 그룹이 아닌 GpioDataRegs 그룹에 속해 있는 레지스터입니다. GPIO가 입력으로 설정된 경우 해당 레지스터 비트는 입력신호(0V or 3.3V)에 따라서 0 혹은 1로 변화합니다. 예를 들어 입력으로 설정된 GPIO1에 3.3V가 입력된다면, 다음과 같은 코드에서 "SW1" 변수 값은 "1"이 됩니다.

```
SW1 = GpioDataRegs.GPADAT.bit.GPIO1;
```

만일, GPIO가 출력으로 설정된 경우엔 해당 레지스터 비트에 설정한 값(0 or 1)에 따라서 출력신호가 0V 혹은 3.3V로 조절됩니다. 예를 들어 출력으로 설정된 GPIO1에서 다음과 같은 코드는 GPIO1 핀에 3.3V를 출력하게 됩니다. (단, 출력으로 사용할 때에는 GPxDAT 레지스터 보다는 되도록 GPxSET/GPxCLEAR/GPxTOGGLE 레지스터를 사용하길 권장합니다.)

```
GpioDataRegs.GPADAT.bit.GPIO1 = 1;
```

이외에 GPIO를 출력으로 사용하는 경우 GPxSET, GPxCLEAR, GPxTOGGLE 레지스터에서도 GPIO 제어가 가능합니다. GPxSET, GPxCLEAR, GPxTOGGLE 레지스터를 조작하는 C 코드는 컴파일러에 의해 atomic instruction으로 변환되기에 코드크기가 줄어드는 이점이 있습니다. 따라서 처리속도도 빠르며, 인터럽트에 의해서 중단되지도 않기에 수행속도가 늘 일정합니다. 그러나, atomic instruction으로 인해 인터럽트가 지연될 수도 있기에, GPIO 조작 보다 인터럽트 처리가 우선일 경우에는 GPxDAT 레지스터를 조작해야 합니다.

◈ **GPxSET 레지스터** (여기서, x는 A or B or C 임.)
이 레지스터는 핀이 출력 상태로 정의되어 있을 때, 출력 값을 Logic HIGH로 만들어 주는 기능을 합니다. GPxDAT 레지스터 해당 비트에 '1'을 써 넣는 것과 동일한 기능을 한다고 이해하시면 됩니다. 그러나, 이 레지스터의 조작 방법은 GPxDAT 레지스터와는 조금 다릅니다. 이 레지스터는 해당 비트 영역에 '1'을 써 넣으면 핀의 출력 상태가 "Logic HIGH"로 바뀌지만, '0'을 써 넣는 것은 아무런 의미를 가지지 못합니다.

모든 GPIO 포트는 GPxSET 레지스터를 가지며, 이 레지스터는 쓰기(write)전용 입니다. 즉 이 레지스터를 Watch Window로 읽으면 무조건 '0'으로 읽힙니다.

◈ **GPxCLEAR 레지스터** (여기서, x는 A or B or C 임.)

이 레지스터는 핀이 출력 상태로 정의되어 있을 때, 출력 값을 Logic LOW로 만들어 주는 기능을 합니다. 즉 GPxSET와 반대의 기능을 한다고도 볼 수 있는데, 레지스터 해당 비트 영역에 '1'을 써 넣으면 핀의 출력 상태가 Logic LOW로 됩니다. 이 레지스터 역시 해당 비트 영역에 '0'을 써 넣는 것은 아무런 의미가 없습니다.

모든 GPIO 포트에서 GPxCLEAR 레지스터를 가지며, 이 레지스터 역시 쓰기(write) 전용입니다. 이 레지스터를 Watch Window로 읽으면 무조건 '0'으로 읽힙니다.

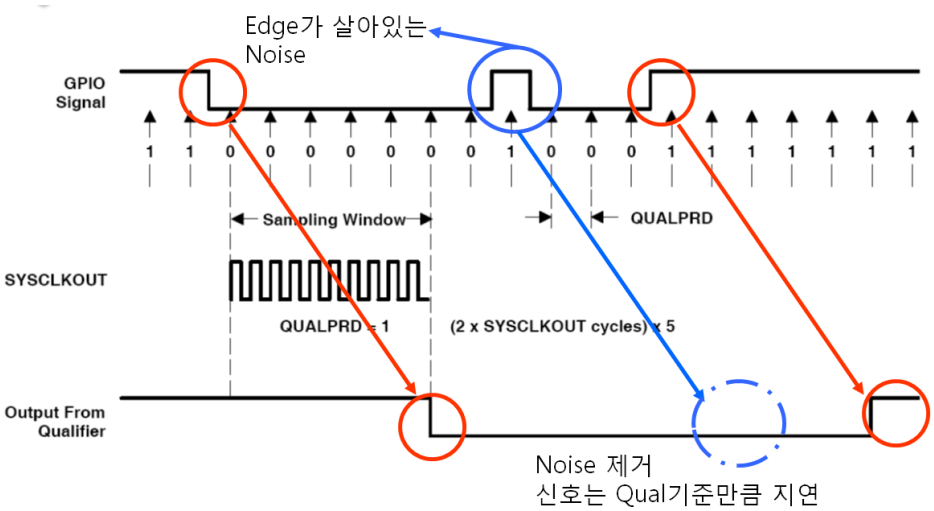
◈ **GPxTOGGLE 레지스터** (여기서, x는 A or B or C 임.)

이 레지스터의 역할은 그 이름에서 눈치 챌 수 있듯이, GPIO 출력 상태를 반전하는 것입니다. 해당 비트 영역에 '1'을 써 넣으면 GPIO의 출력이 반전됩니다. 예를 들어, GPIOA0번 핀의 현재 상태가 Logic HIGH일 때 "GpioDataRegs.GPATOGGLE.bit.GPIOA0 = 1"이라고 써 넣으면 GPIOA0번 핀의 상태는 Logic LOW로 바뀌게 됩니다. 이런 상태에서 다시 위의 코드를 만나면 GPIOA0번 핀은 다시 Logic HIGH 상태로 바뀌게 됩니다. 이 레지스터에 '0'을 써 넣는 것은 아무런 의미가 없습니다.

모든 GPIO 포트에서 GPxTOGGLE 레지스터를 가지며, 이 레지스터 역시 쓰기(write) 전용입니다. 이 레지스터를 Watch Window로 읽으면 무조건 '0'으로 읽힙니다.

Delfino 프로세서의 GPIO 입력 시 특수기능 (Input Qualifier)

신호에는 노이즈가 섞여있습니다. 아날로그 신호라면 여러 가지 필터로 노이즈를 걸러낼 수도 있겠습니다. 디지털 신호에서도 아날로그 필터를 사용하기도 하고, 비교기를 사용해서 디지털 신호를 재생하기도 합니다. 디지털 신호에서는 노이즈가 아무리 섞여있어도 0인지 1인지만 정확히 파악하면 그걸로 끝입니다. 아날로그 신호에 비해서, 노이즈 대응이 매우 쉽습니다.



하지만, 위 그림과 같이 "Edge가 살아있는 Noise"가 인가되면 디지털 시스템에서도 속수무책입니다. 그림에서 보이는 노이즈는 아날로그 필터로는 걸러내기가 어렵습니다. 엣지 성분이 고스란히 살아있어서, 디지털 시스템은 반응을 하게되는데, 엣지 성분을 없애기 위해서 Low Pass Filter를 사용한다면, 신호 고유의 엣지 성분도 차단되어 신호를 잃어버리게 됩니다. 에러 컨트롤 코드를 이용하여, 소프트웨어적으로 원 신호를 복원할 수도 있지만, 시간 지연과 연산 자원 소비로 실시간 시스템에는 부적합한 경우가 많습니다.

이런 경우에 Input Qualifier 라는 회로가 유용합니다. 'Qualifier'란, 자격 심사를 의미합니다. 입력되는 신호의 자격이 유효한가를 판정하는 것이 곧 Input Qualifier 입니다.. 따라서 디지털 회로로 구현하는 것도 매우 간단합니다.

입력신호의 규격, 즉 펄스의 길이를 감지하여, 펄스 길이가 기준에 미치지 못한다면, 무시해버리는 회로를 구현하는 것입니다. 이것이 바로 Input Qualifier 입니다.

Input Qualifier 사용방법

Input Qualifier를 사용하기 위해서는 2가지 레지스터 설정이 필요합니다. 먼저, GPxCTRL 레지스터를 설정해서 Qualification을 위한 샘플링 주기를 설정합니다. 다음은 GPxSEL1 혹은 GPxSEL2 레지스터를 설정해서 Qualification에 몇 번의 샘플을 사용할 것 인지 설정합니다. 이러한 2개의 레지스터 설정을 통해서 사용자는 Input Qualifier 기능을 사용할 수 있습니다.

◈ **GPxCTRL 레지스터** (여기서, x는 A or B 임.)
'GPIO Qualification Control Register'의 약자입니다. 8비트 씩 QUALPRD1~3 이란 이름으로 총 4개의 영역을 가지고 있습니다. 각 영역은 다음과 같이 각각의 GPIO 포트들의 Qualification 샘플링 주기를 별도로 설정할 수 있습니다.

GPACTRL 레지스터
QUALPRD0 -> GPIO 0 ~ GPIO 7
QUALPRD1 -> GPIO 8 ~ GPIO 15
QUALPRD2 -> GPIO 16 ~ GPIO 23
QUALPRD3 -> GPIO 24 ~ GPIO 31

GPBCTRL 레지스터
QUALPRD0 -> GPIO 32 ~ GPIO 39
QUALPRD1 -> GPIO 40 ~ GPIO 47
QUALPRD2 -> GPIO 48 ~ GPIO 55
QUALPRD3 -> GPIO 56 ~ GPIO 63

해당 비트영역에 0~255 값을 (8비트) 설정할 수 있고, 다음과 같은 공식으로 샘플링 주기가 결정됩니다. **(샘플링 주기 = 설정값 x 2 x $T_{SYSCLKOUT}$)** 여기서 $T_{SYSCLKOUT}$ 는 시스템 클럭의 주기 입니다. 예를 들어 시스템 클럭이 150MHz이고, 다음과 같이 GPxCTRL 레지스터를 설정한다면

```
GpioCtrlRegs.GPBCTRL.bit.QUALPRD1 = 100;
```

이것은 GPIO40~47 포트의 Qualification 샘플링 주기를 약 1.33us로 설정한 것입니다.

◈ **GPxQSEL1/GPxQSEL2 레지스터** (여기서, x는 A or B 임.)

'GPIO Qualification Select'의 약자 입니다. GPIO0부터 GPIO63까지 개별적으로 Qualification 샘플링 횟수를 3회 혹은 6회로 설정 할 수 있습니다. 만일 Qualification 샘플링 주기가 1.33us 설정됐다면, 3회 샘플링 시에는 Qualification 샘플링 윈도우가 2.66us가 되고, 6회 샘플링 시에는 6.65us 가 됩니다.

```
GpioCtrlRegs.GPBQSEL1.bit.GPIO44 = 2;
```

예를 들어, 위와 같은 코드는 GPIO44의 Qualification 샘플링 횟수를 6회로 설정한 것입니다.

1.1 GPIO 입력으로 쓰기

실습 목적

TMS320F2833x의 GPIO 포트를 입력으로 사용해서 Delfino EVM의 4개의 스위치(SW1, SW2, SW3, SW4)에서 들오는 신호를 확인해 봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩GPIO01_Input
CCSv4.2용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩GPIO01_Input
CCSv5.1용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩GPIO01_Input

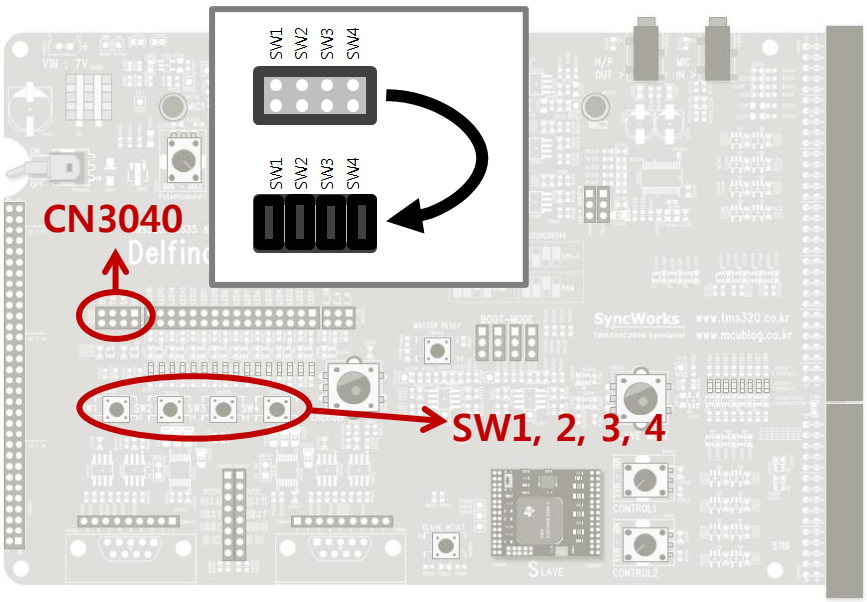
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[GPIO-01] GPIO 입력으로 쓰기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - CCS 프로젝트 만들기(TMS320F28335)" 게시물

실습해 보기

Delfino EVM의 스위치(SW1, SW2, SW3, SW4)를 사용하기 위해서 CN3040 점퍼를 모두 연결합니다.



CCS에서 예제를 리얼타임 모드로 실행하고, CCS의 Watch Window를 통해서 스위치 조작에 따라서 달라지는 SW1, SW2, SW3, SW4 변수를 확인해 봅니다

1.2 GPIO 입력, 외부 인터럽트로 사용

실습 목적

TMS320F2833x 의 GPIO 포트를 입력으로 사용해서 Delfino EVM 의 4 개의 스위치(SW1, SW2, SW3, SW4)에서 들오는 신호를 외부 인터럽트로 사용해 봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩GPIO02_Input_Xint
CCSv4.1용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩GPIO02_Input_Xint
CCSv5.2용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩GPIO02_Input_Xint

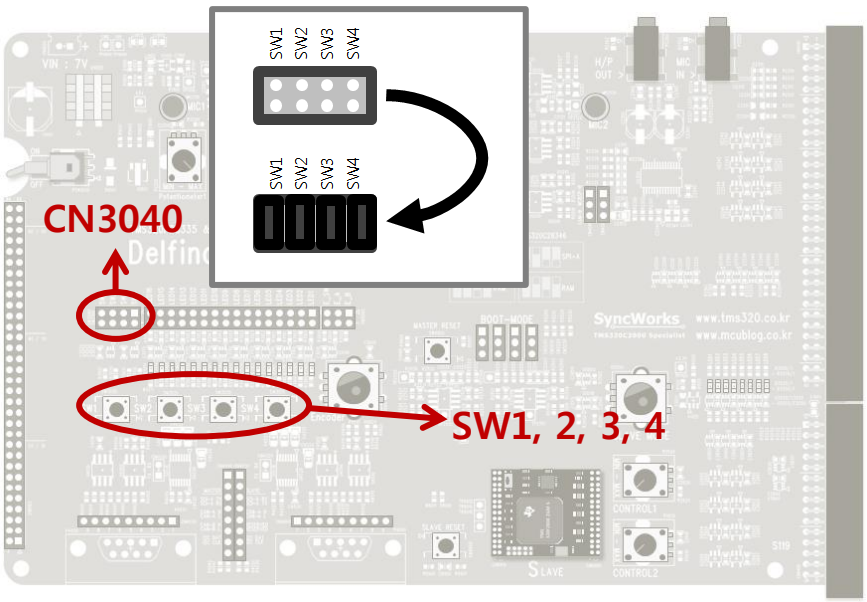
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[GPIO-02] GPIO 입력, 외부 인터럽트로 사용" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - GPIO 입력, 외부 인터럽트로 사용" 게시물

실습해 보기

Delfino EVM의 스위치(SW1, SW2, SW3, SW4)를 사용하기 위해서 CN3040 점퍼를 모두 연결합니다.



CCS에서 예제를 리얼타임 모드로 실행하고, CCS의 Watch Window를 통해서 SW1_cnt, SW2_cnt, SW3_cnt, SW4_cnt 변수들을 관찰해 봅니다. 위 변수는 각각의 스위치 입력으로 발생하는 외부 인터럽트 횟수를 카운팅하는 변수 입니다.

퀴즈!!

스위치를 한번 입력했는데도 인터럽트 카운트가 여러번 올라가는 경우가 있습니다. 이유는 뭘까요?

1.3 GPIO 입력, Qualification 기능 사용하기

실습 목적

TMS320F2833x 의 GPIO 포트를 입력으로 사용해서 Delfino EVM 의 4 개의 스위치(SW1, SW2, SW3, SW4)에서 들오는 신호를 외부 인터럽트로 사용하고, Qualification 기능을 사용해봅니다. 스위치 입력에서 발생하는 하는 노이즈를 감소 시킬 수 있습니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩GPIO03_Input_Qual
CCSv4.2용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩GPIO03_Input_Qual
CCSv5.1용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩GPIO03_Input_Qual

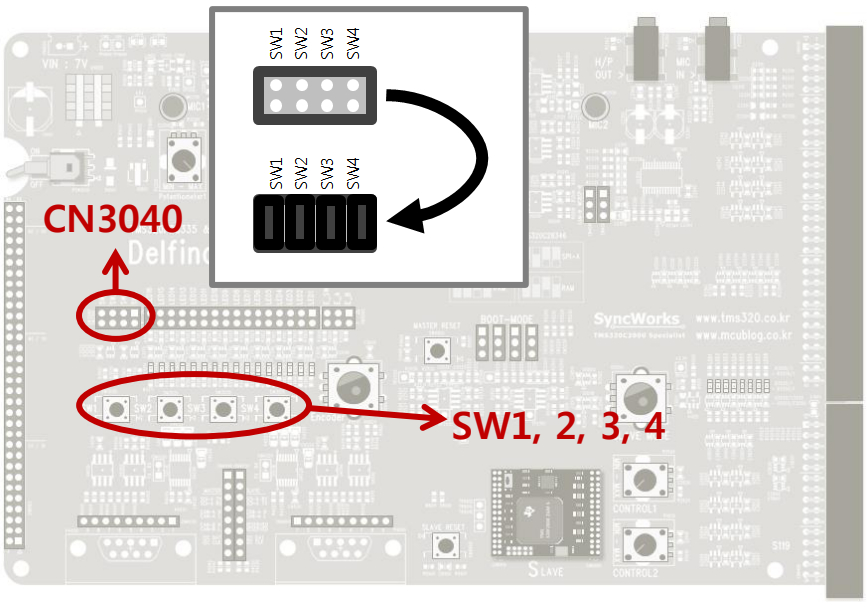
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[GPIO-03] GPIO 입력, Qualification 기능 사용하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - GPIO 입력, Qualification 기능 사용하기" 게시물

실습해 보기

Delfino EVM의 스위치(SW1, SW2, SW3, SW4)를 사용하기 위해서 CN3040 점퍼를 모두 연결합니다.



CCS에서 예제를 리얼타임 모드로 실행하고, CCS의 Watch Window를 통해서 SW1_cnt, SW2_cnt, SW3_cnt, SW4_cnt 변수들을 관찰해 봅니다. 위 변수는 각각의 스위치 입력으로 발생하는 외부 인터럽트 횟수를 카운팅하는 변수 입니다.

퀴즈!!

앞서 "GPIO 입력, 외부 인터럽트" 사용예제에서 스위치를 한번 입력했는데도 인터럽트 카운트가 여러번 올라가는 현상이 이번 " GPIO 입력, Qualification 기능 사용" 예제에서는 상당히 줄어듭니다. 왜 이와 같은 현상이 생기는지 생각해보세요.

1.4 GPIO 출력, LED 점멸 1

실습 목적

TMS320F2833x 의 GPIO 포트를 출력으로 사용해서 Delfino EVM 의 16 개의 LED 를 제어해 봅니다

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩GPIO04_Output_1
CCSv4.2용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩GPIO04_Output_1
CCSv5.1용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩GPIO04_Output_1

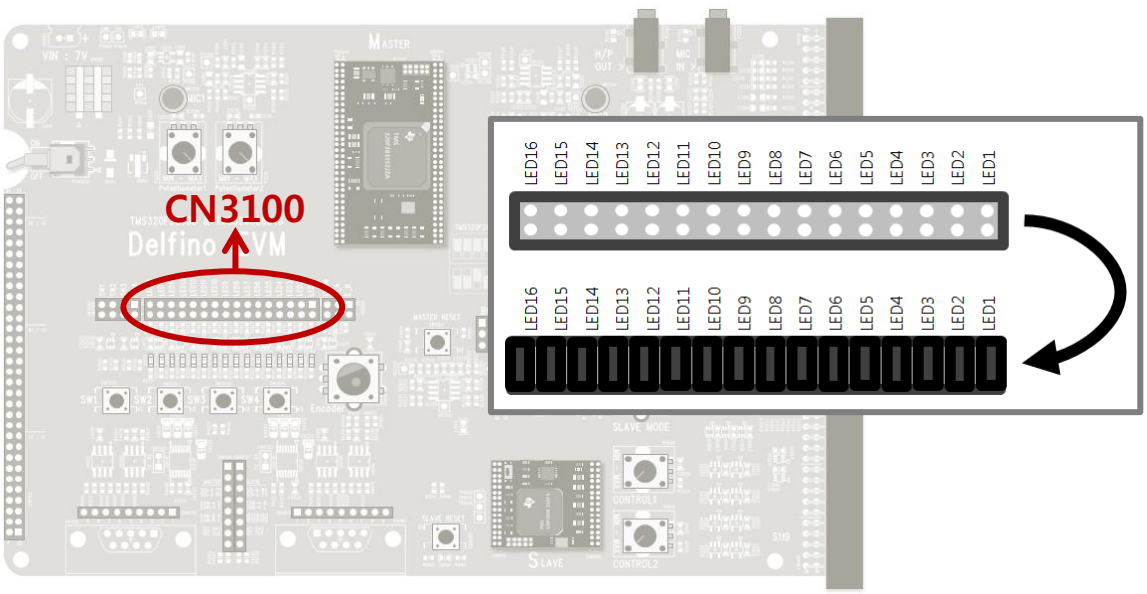
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[GPIO-04] GPIO 출력, LED 점멸 1" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfion EVM - GPIO 출력, LED 점멸 1" 게시물

실습해 보기

Delfino EVM의 LED 를 사용하기 위해서 CN3100 점퍼를 모두 연결합니다.



CCS에서 예제를 실행하고, CCS의 Watch Window를 통해서 usec_delay 변수 값을 변경하면서 LED 점멸 패턴을 관찰해 봅니다.

퀴즈!!

예제코드를 수정해서 LED 점멸 패턴을 여러가지로 바꿔보세요.

1.5 GPIO 출력, LED 점멸 2

실습 목적

TMS320F2833x 의 GPIO 포트를 출력으로 사용해서 Delfino EVM 의 16 개의 LED 를 제어해 봅니다

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩GPIO05_Output_2
CCSv4.2용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩GPIO05_Output_2
CCSv5.1용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩GPIO05_Output_2

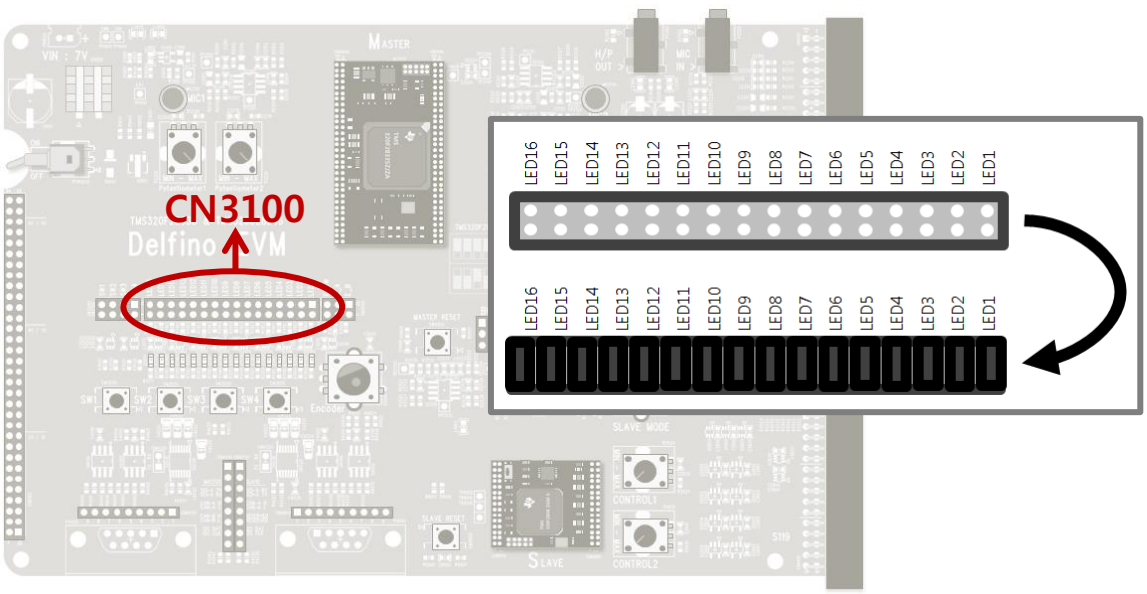
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[GPIO-05] GPIO 출력, LED 점멸 2" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfion EVM - GPIO 출력, LED 점멸 2" 게시물

실습해 보기

Delfino EVM의 LED 를 사용하기 위해서 CN3100 점퍼를 모두 연결합니다.



CCS에서 예제를 실행하고, CCS의 Watch Window를 통해서 usec_delay 변수 값을 변경하면서 LED 점멸 패턴을 관찰해 봅니다.

퀴즈!!

GPIO04 예제코드와 비교해서 어떤 점이 달라졌는지 확인해 보세요.

1.6 GPIO 출력, LED 점멸 3

실습 목적

TMS320F2833x 의 GPIO 포트를 출력으로 사용해서 Delfino EVM 의 16 개의 LED 를 제어해 봅니다

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩GPIO06_Output_3
CCSv4.2용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩GPIO06_Output_3
CCSv5.1용 : C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩GPIO06_Output_3

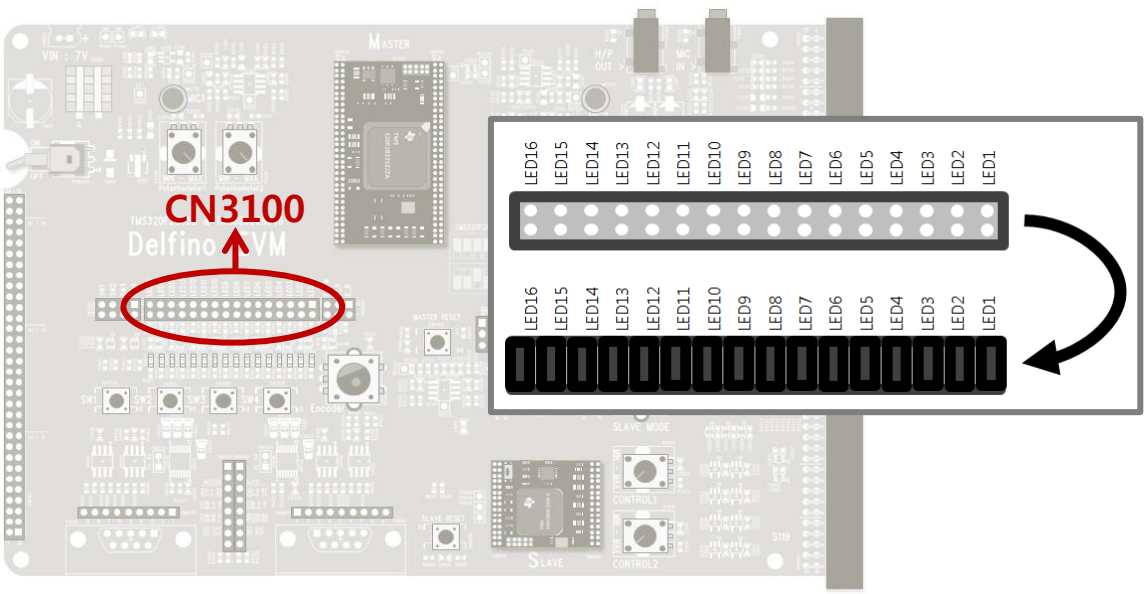
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[GPIO-06] GPIO 출력, LED 점멸 3" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfion EVM - GPIO 출력, LED 점멸 3" 게시물

실습해 보기

Delfino EVM의 LED 를 사용하기 위해서 CN3100 점퍼를 모두 연결합니다.



CCS에서 예제를 실행하고, CCS의 Watch Window를 통해서 usec_delay 변수 값을 변경하면서 LED 점멸 패턴을 관찰해 봅니다.

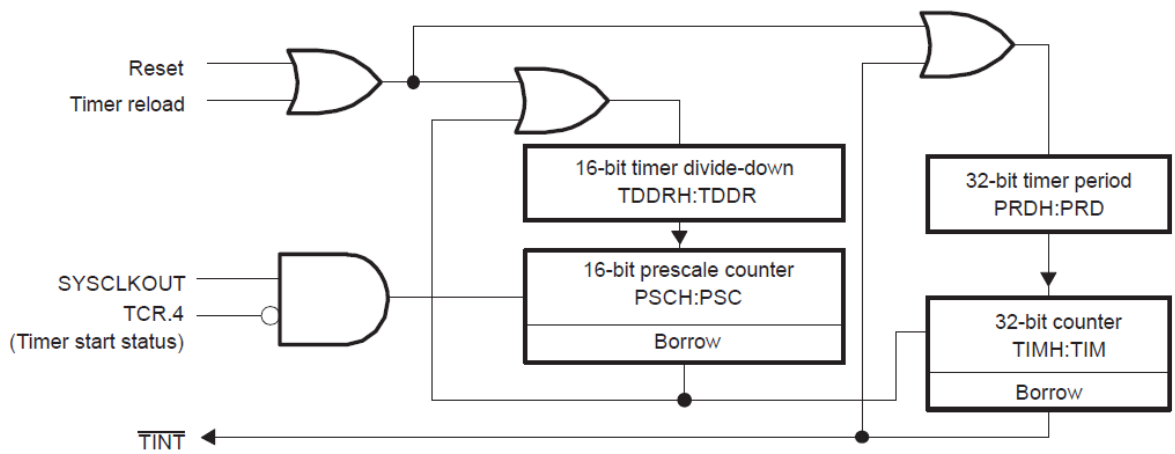
퀴즈!!

GPIO04, GPIO05 예제코드와 비교해서 어떤 점이 달라졌는지 확인해 보세요.

2 CPU-Timer

Delfino 프로세서의 28x CPU Core는 여러가지 용도로 사용할 수 있는 3개의 CPU-Timer를 가지고 있습니다. Timer0, Timer1, Timer2가 그것인데, 이들은 모두 32bit-Timer로서 down-Counting 방식으로 동작합니다. 사용자는 이 3개의 CPU-Timer를 통해서 타임 스케줄링이 필요한 작업을 할 수 있습니다. 일반적으로는 주기적인 인터럽트 생성에 사용되지만, TI의 Real-Time OS인 DSP/BIOS에서도 사용됩니다.

CPU Timer의 구조



CPU Timer는 32bit Counter Register(TIMH:TIM)와, Period Register(PRDH:PRD)를 가지고 있고 SYSCLKOUT를 자체 분주하기 위한 16bit Prescale Register(PSCH:PSC)와 Divide-down Register(TDDRH:TDDR)를 가지고 있습니다. 그 외 그림에는 언급되어 있진 않지만 전체 Timer의 동작을 제어하기 위한 16bit Control Register를 가지고 있습니다.

먼저 PLL회로에 의해 분주된 SYSCLKOUT은 CpuTimer의 Prescale Register에 공급되게 됩니다. 그럼 Prescale Counter Register는 Timer Divide-down Register의 값을 읽어와서 Down Counting을 시작하게 되고, 카운팅을 계속하여 0 이하로 떨어지면 Borrow가 발생하여 그 때 32bit Counter Register에 클럭을 공급하게 됩니다.

Counter Register는 클럭을 공급받으면 32bit Period Register의 값을 읽어와서 Down Counting을 시작하게 됩니다. 이때 Counter Register는 Prescale Register로부터 클럭을 공급받을 때 마다 읽어온 값을 하나씩 떨어뜨린다. 역시 카운팅을 계속하여 Borrow가 되면 Timer Interrupt 신호를 발생하게 됩니다.

CPU Timer의 사용방법

앞서 봤던 CPU Timer 구조에서 사용자가 설정한 가능한 레지스터는 TDDRH:TDDR 레지스터와 PRDH:PRD 레지스터 입니다. 두 레지스터 값을 설정하고, TCR 레지스터 4번 TSS 비트를 1로 set 하면 타이머가 시작됩니다. 다만, 이러한 설정들은 TI가 기본 제공하는 DSP2833x_CpuTimers.c 파일에 함수 형태로 정의 되어 있습니다. 사용자는 TI가 제공하는 함수를 사용해서 간단하게 CPU Timer 설정을 할 수 있습니다.

◈ **CPU Timer 설정 예**

```
InitCpuTimers();                    // CPU Timer 초기화

Cpu_Clk = 150;                      // 현재 시스템 클럭을 설정 (MHz 단위)
Timer_Prd = 1000000;                // 타이머 주기 설정 (usec 단위)

// CPU Timer0 을 1초 주기로 설정
ConfigCpuTimer(&CpuTimer0, Cpu_Clk, Timer_Prd);

// CPU Timer0 시작
StartCpuTimer0();
```

2.1 CPU Timer 인터럽트를 이용한 LED 주기점멸

실습 목적

TMS320F2833x 의 CPU Timer 인터럽트를 사용해서 Delfino EVM 의 16 개의 LED 를 1 초 주기로 점멸합니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩CPUtimer01_Period_Interrupt
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩CPUtimer01_Period_Interrupt
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩CPUtimer01_Period_Interrupt

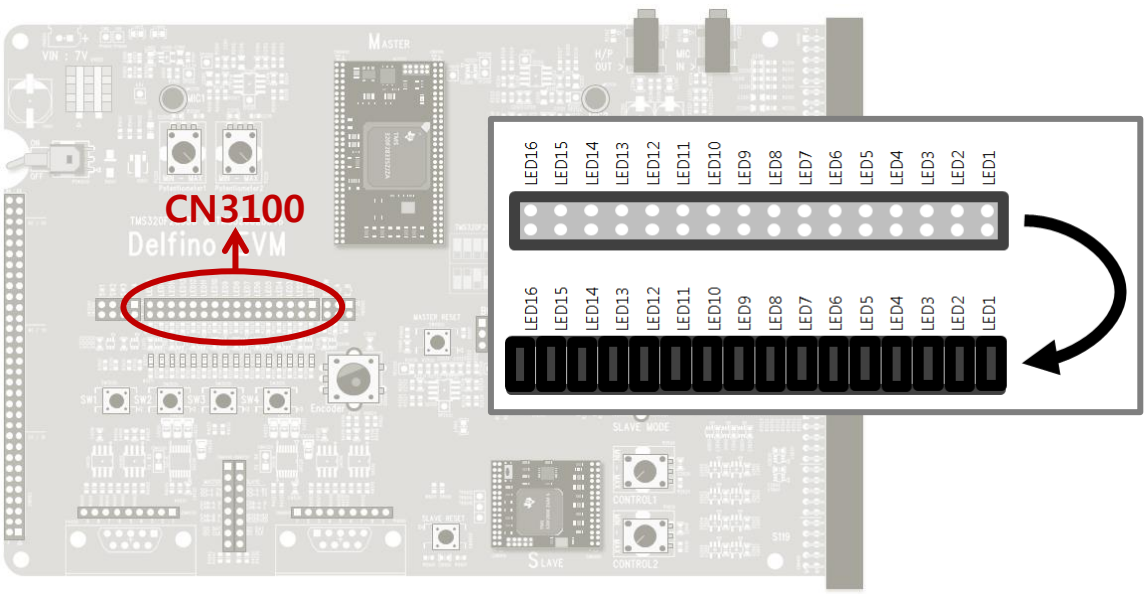
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[CPU Timer-01] CPU Timer 인터럽트를 이용한 LED 주기점멸" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfion EVM - CPU Timer 인터럽트를 이용한 LED 주기점멸" 게시물

실습해 보기

Delfino EVM의 LED 를 사용하기 위해서 CN3100 점퍼를 모두 연결합니다.



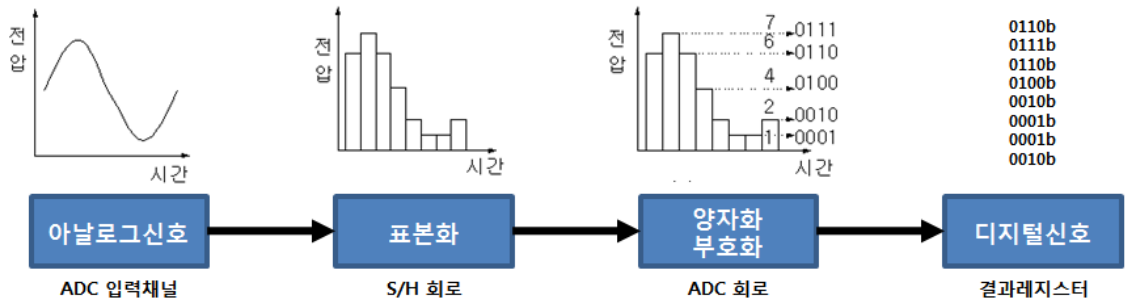
CCS에서 예제를 실행하고, LED 점멸 속도가 1초 인지 확인해 봅니다.

퀴즈!!

CPU Timer 설정을 수정해서 LED 점멸 주기를 바꿔보세요.

3 ADC

ADC는 Analog to Digital Converter의 약자로 아날로그 신호를 디지털 신호로 변환해주는 장치를 말합니다. 시간과 크기에서 연속적인 아날로그 신호를 S/H(Sample-and-Hold) 회로를 통해서 표본화(Sampling)하고, 이를 ADC 회로에서 양자화(Quantization)와 부호화(Coding)를 거쳐 디지털 신호로 변환됩니다. Delfino 프로세서는 이러한 ADC를 칩 자체에 내장하고 있습니다.

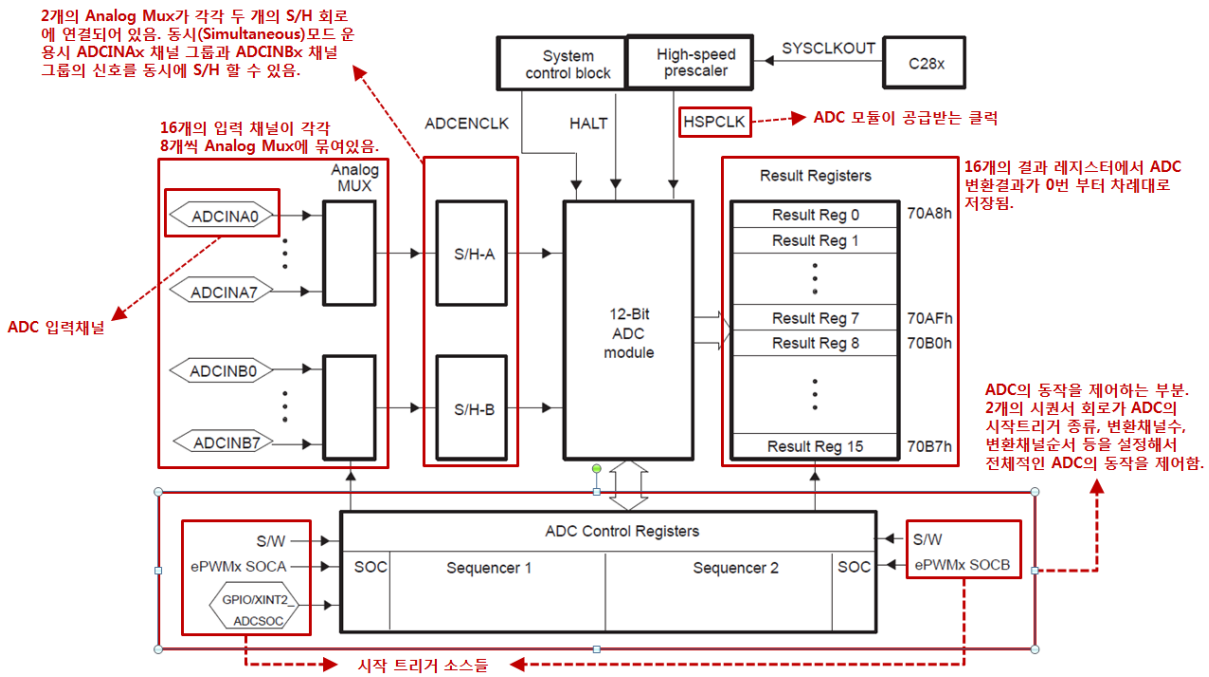


Delfino 프로세서의 ADC 특징

- **16채널, 12bit 분해능, 최대 12.5MSPS의 속도로 샘플링 가능한 ADC 회로 탑재**
- **2개의 Sample/Hold 회로 탑재**
  - 별도의 분주기로 데이터 수집 시간을 조절할 수 있음.
- **동시(Simultaneous) 샘플링/순차(Sequential) 샘플링 모드 선택 가능**
- **0V ~ 3V의 아날로그 입력 범위**
- **직렬(Cascaded) 혹은 병렬(Dual) 운용 가능한 시퀀서 회로 탑재**
  - 직렬 시퀀서 운용 시 16ch 까지 변환횟수와 변환순서를 지정할 수 있음
  - 병렬 시퀀서 운용 시 8ch 까지 변횐횟수와 변화 순서를 지정할 수 있는 독립적인 시퀀서 2개 사용할 수 있음
- **ADC 결과를 저장하는 16개의 결과 (Result) 레지스터 탑재**
  - 입력 ≤ 0V 일 때, 디지털 값 = 0
  - 0V < 입력 < 3V 일 때, 디지털 값 = 4096 * (입력 아날로그 전압 – ADCLO) / 3
  - 입력 ≥ 3V 일 때, 디지털 값 = 4095
- **ADC 시동(Start-Of-Conversion sequence)을 위한 여러가지 트리거 선택가능**
  - Software / ePWM1~6 / GPIO XINT2
- **시퀀스 종료(End-Of-Sequence(EOS))시 인터럽트 사용가능**
- **병렬(dual) 시퀀서 모드에서 ePWM 트리거는 두 시퀀서를 별도 독립적으로 운용 가능**

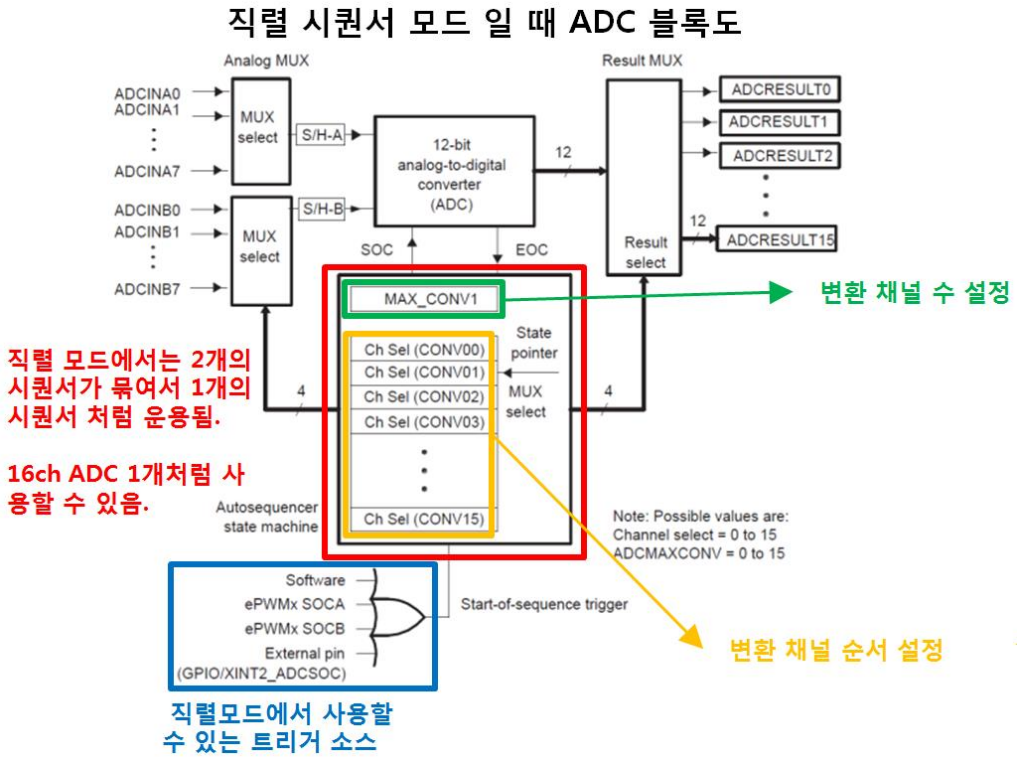
Delfino 프로세서의 ADC 구조

Delfino 프로세서의 ADC는 총 16개의 입력 채널을 가지고 있습니다. 각 입력채널을 8채널씩 묶여서 A0~7, B0~8 그룹으로 나눠지고, 각 그룹은 별도의 Analog MUX와 S/H 회로를 통해서 12bit ADC module에 연결됩니다. ADC Module에서 아날로그 신호는 디지털 데이터로 변환되고, 변환결과는 Result 레지스터에 0번부터 순차적으로 저장됩니다. 이때, 이러한 모든 ADC 동작 과정은 2개의 시퀀서 회로를 통해서 제어 됩니다. 사용자는 시퀀서1,2에서 시작트리거 종류 / 변환채널수 / 변환채널순서 등을 설정해서 ADC를 운용하게 됩니다.



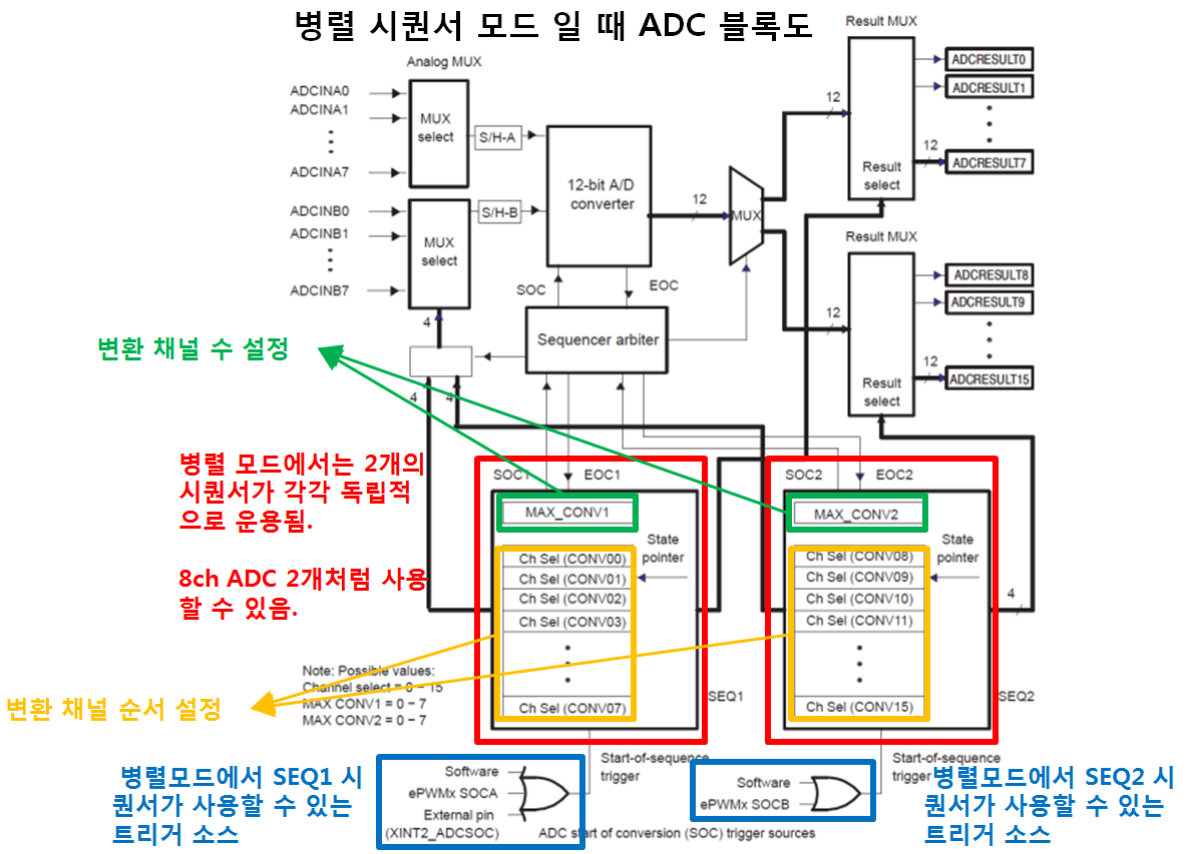
ADC 시퀀서(Sequencer) 회로

ADC 모듈은 시작 트리거 종류, 변환채널 수, 변환채널 순서, 인터럽트 설정에 따라 일련의 과정을 거쳐 ADC 결과를 출력하게 됩니다. 여기서 ADC가 시작(트리거)된 후 변환 결과가 나오기까지의 일련의 과정을 시퀀스(Sequence)라고 합니다. 그리고 이러한 시퀀스를 설정, 제어하는 회로를 시퀀서(Sequencer) 라고 합니다. ADC 모듈은 이러한 시퀀서 회로를 2개 가지고 있습니다. 각각의 시퀀서는 8채널씩 ADC 입력을 처리할 수 있습니다. 따라서 시퀀서 운용 방법에 따라서 2개의 시퀀서를 직렬(Cascaded) 혹은 병렬(Dual)로 선택 운용 하는 것이 가능합니다.



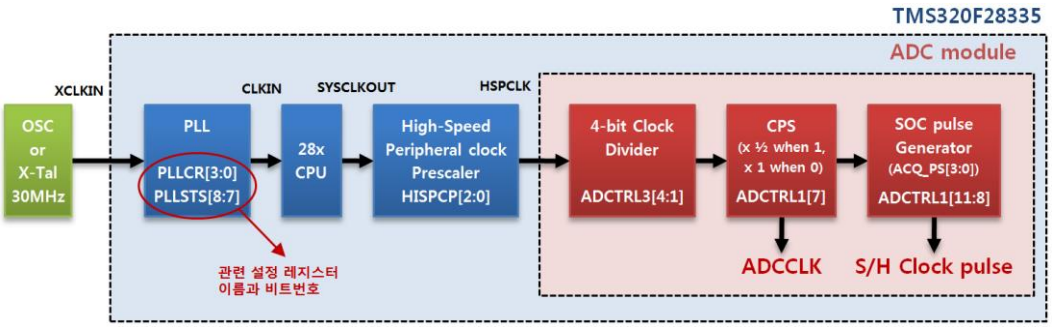
직렬 시퀀서 모드에서는 2개의 시퀀서가 묶여서 1개의 시퀀서처럼 운용됩니다. 이때는 16ch ADC 1개를 사용하는 것과 같습니다. 4가지의 트리거 소스를 사용할 수 있고, ADC결과는 순차적으로 ADCRESULT0 레지스터부터 저장됩니다. 예를 들어, 1번→5번→3번 순서로 3개의 채널을 ADC 한다면 각각의 결과는 채널번호에 대응해서 ADCRESULT1,5,3 에 저장되는 것이 아니고, 순차적으로 ADCRESULT0,1,2 에 저장되는 것입니다.

병렬 시퀀서 모드에서는 2개의 시퀀서가 각각 독립적으로 운용됩니다. 이때는 8ch ADC 2개를 사용하는 것과 같습니다. 각각의 시퀀서 별로 트리거 소스가 다른 것에 주의해야 합니다. ADC 결과는 시퀀서1에서 ADCRESULT0~7에 순차적으로 저장되고, 시퀀서2에서는 ADCRESULT8~15에 순차적으로 저장됩니다.



ADC 구동클럭(ADCCLK)과 S/H 클럭 펄스

아래 그림은 외부 클럭 소스(OSC or X-Tal)에서 TMS320F28335 프로세서의 ADC 모듈 구동클럭(ADCCLK)과 S/H 클럭 펄스가 어떻게 만들어지는지 블록도로 보여주고 있습니다. 각 분주 비율에 대한 자세한 설명은 프로세서의 System Control 가이드(spru078f)와 ADC 가이드(spru812a)의 레지스터 설명 부분을 참고하시기 바랍니다.



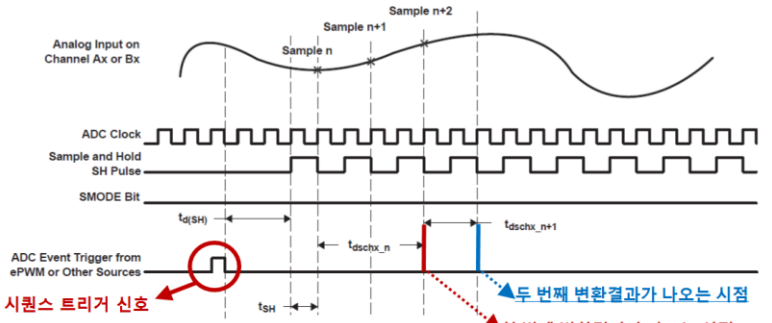
ADC 변환 시간 계산 (순차 샘플링 모드 일 때)

ADC 구동클럭(ADCCLK)과 S/H Width가 결정되면, 아래 Timing 도표를 참고하여 ADC가 Analog 신호를 Digital 신호로 변환하는데 걸리는 시간을 계산할 수 있습니다. 단, 아래 Timing 도표는 ADC를 순차 샘플링 (Sequential Sampling) 모드로 운용했을 경우 입니다.

첫 번째 변환결과가 나오는데 걸리는 시간
= $2.5T_{C(ADCCLK)}$ + $(1+Acqps)T_{C(ADCCLK)}$ + $4T_{C(ADCCLK)}$
두 번째 변환결과가 나오는데 걸리는 시간
= $(2+Acqps)T_{C(ADCCLK)}$
여기서, $T_{C(ADCCLK)}$는 ADCCLK 의 주기,
Acpps 는 ADCTRL1[11:8]- ACQ_PS 의 설정값 임.

ADCCLK: 25MHz 이고, Acqps: 0 라면,
$T_{C(ADCCLK)}$ = 40nsec 임.
트리거 신호 후 첫 번째 변환 시간
= 2.5*40ns + (1+0)*40ns + 4*40ns = 300ns
트리거 신호 후 두 번째 부터의 변환 시간
= (2+0)*40ns = 80ns

Sequential Sampling Mode Timing 도표



		SAMPLE n	SAMPLE n + 1	AT 25-MHz ADC CLOCK, $t_{c(ADCCLK)}$ = 40 ns	REMARKS
$t_{d(SH)}$	Delay time from event trigger to sampling	$2.5t_{c(ADCCLK)}$			
t_{SH}	Sample/Hold width/Acquisition Width	(1 + Acqps) * $t_{c(ADCCLK)}$		40 ns with Acqps = 0	Acqps value = 0-15 ADCTRL1[8:11]
$t_{d(schx_n)}$	Delay time for first result to appear in Result register	$4t_{c(ADCCLK)}$		160 ns	
$t_{d(schx_n+1)}$	Delay time for successive results to appear in Result register		(2 + Acqps) * $t_{c(ADCCLK)}$	80 ns	

첫번째 변환결과가 출력되는데 걸리는 시간

ADC 변환 시간 계산 (동시 샘플링 모드 일 때)

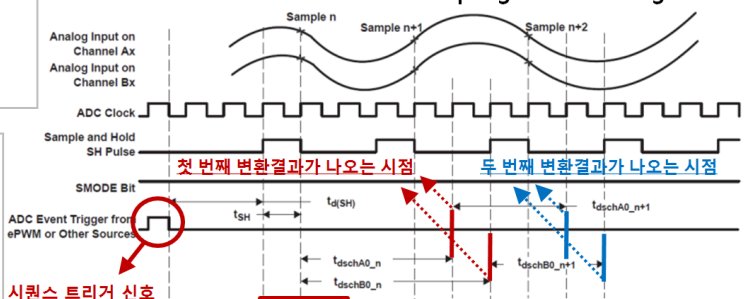
ADC 구동클럭(ADCCLK)과 S/H Width가 결정되면, ADC가 Analog 신호를 Digital 신호로 변환하는데 걸리는 시간을 계산할 수 있습니다. 여기서, 아래 Timing 도표는 ADC를 동시 샘플링(Simultaneous Sampling) 모드로 운용했을 경우 입니다. 동시 샘플링 모드에서는 A0/B0, A1/B1, A2/B2, …, A7/B7 와 같이 두 입력 채널이 짝(A0/B3 이런 짝은 안됨)을 이루워 동시에 샘플링이 이루어 집니다. 이는 ADC 모듈의 S/H 회로가 2개이기 때문에 가능한 것입니다. 하지만 ADC Core는 한 개 이기 때문에 변환결과는 ADCINBx 채널이 ADCINAx 채널 보다 1 ADCCLK 늦게 나옵니다

첫 번째 변환결과가 나오는데 걸리는 시간
ADCINAx 채널 = $2.5T_{C(ADCCLK)}$ + $(1+Acqps)T_{C(ADCCLK)}$ + $4T_{C(ADCCLK)}$
ADCINBx 채널 = $2.5T_{C(ADCCLK)}$ + $(1+Acqps)T_{C(ADCCLK)}$ + $5T_{C(ADCCLK)}$
두 번째 변환결과가 나오는데 걸리는 시간
= $(3+Acqps)T_{C(ADCCLK)}$

여기서, $T_{C(ADCCLK)}$는 ADCCLK 의 주기,
Acqps 는 ADCTRL1[11:8]- ACQ_PS 의 설정값 임.

ADCCLK: 25MHz 이고, Acqps: 0 라면,
$T_{C(ADCCLK)}$ = 40nsec 임.
트리거 신호 후 첫 번째 변환 시간
ADCINAx 채널 = 2.5*40ns + (1+0)*40ns + 4*40ns = 300ns
ADCINBx 채널 = 2.5*40ns + (1+0)*40ns + 5*40ns = 340ns
트리거 신호 후 두 번째 부터의 변환 시간
= (3+0)*40ns = 120ns

Simultaneous Sampling Mode Timing 도표



		SAMPLE n	SAMPLE n + 1	AT 25-MHz ADC CLOCK, $t_{c(ADCCLK)}$ = 40 ns	REMARKS
$t_{d(SH)}$	Delay time from event trigger to sampling	$2.5t_{c(ADCCLK)}$			
t_{SH}	Sample/Hold width/Acquisition Width	(1 + Acqps) * $t_{c(ADCCLK)}$		40 ns with Acqps = 0	Acqps value = 0-15 ADCTRL1[8:11]
$t_{d(schA0_n)}$	Delay time for first result to appear in Result register	$4t_{c(ADCCLK)}$		160 ns	
$t_{d(schB0_n)}$	Delay time for first result to appear in Result register	$5t_{c(ADCCLK)}$		200 ns	
$t_{d(schA0_n+1)}$	Delay time for successive results to appear in Result register		(3 + Acqps) * $t_{c(ADCCLK)}$	120 ns	
$t_{d(schB0_n+1)}$	Delay time for successive results to appear in Result register		(3 + Acqps) * $t_{c(ADCCLK)}$	120 ns	

첫번째 변환결과가 출력되는데 걸리는 시간

ADC 시퀀스 시작을 위한 트리거 소스

ADC 시퀀스를 시작하게 하는 방아쇠(Trigger)역할의 신호를 말합니다. 여기서 ADC 시퀀스의 시작은 줄여서 SOC(Start-Of-Conversion) 혹은 SOS(Start-Of-Sequence) 라고 부릅니다. ADC모듈의 ADC 시퀀서(SEQ1, SEQ2) 2개는 각각 다음과 같은 트리거 소스를 가지고 있습니다. 만일 직렬 시퀀서 모드를 사용한다면 시퀀서1, 2는 시퀀서 1로 통합됩니다. 각 시퀀서는 ADCTRL2 레지스터의 15, 8, 7, 0 번 비트에서 각 트리거 소스의 사용 여부를 설정할 수 있습니다.

◈ ADCTRL2 레지스터

ADC 시퀀스의 트리거 신호 선택, 시퀀스 리셋, ADC 인터럽트 모드 설정을 하는 레지스터입니다.

15	14	13	12	11	10	9	8
ePWM_SOCB_SEQ	RST_SEQ1	SOC_SEQ1	Reserved	INT_ENA_SEQ1	INT_MOD_SEQ1	Reserved	ePWM_SOCA_SEQ1
R/W-0	R/W-0	R/W-0	R-0	R/W-0	R/W-0	R-0	R/W-0
7	6	5	4	3	2	1	0
EXT_SOC_SEQ1	RST_SEQ2	SOC_SEQ2	Reserved	INT_ENA_SEQ2	INT_MOD_SEQ2	Reserved	ePWM_SOCB_SEQ2
R/W-0	R/W-0	R/W-0	R-0	R/W-0	R/W-0	R-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; W = Write access, S = Set only, C = Clear -*n* = value after reset

시퀀서1(SEQ1)의 트리거 소스

- **Software** : ADCTRL2[13]-SOC_SEQ1 비트를 소프트웨어에서 1로 설정하는 것으로 SEQ1을 SOC
- **ePWMx SOCA** : ePWM 모듈의 Event Trigger SOCA Pulse Generator에서 생성하는 신호로 SEQ1을 SOC
- **External Pin(GPIO/XINT2_ADCSOC)** : GPIO 포트 A(GPIO0~31)의 핀들 중 하나를 외부 인터럽트 핀으로 설정하고 외부 신호로 SEQ1을 SOC

시퀀서2(SEQ2)의 트리거 소스

- **Software** : ADCTRL2[5]-SOC_SEQ2 비트를 소프트웨어에서 1로 설정하는 것으로 SEQ2를 SOC
- **ePWMx SOCB** : ePWM 모듈의 Event Trigger SOCB Pulse Generator에서 생성하는 신호로 SEQ2를 SOC

직렬 시퀀서 모드시 트리거 소스

- **Software** : ADCTRL2[13]-SOC_SEQ1 비트를 소프트웨어에서 1로 설정하는 것으로 SOC
- **ePWMx SOCA** : ePWM 모듈의 Event Trigger SOCA Pulse Generator에서 생성하는 신호로 SOC
- **ePWMx SOCB** : ePWM 모듈의 Event Trigger SOCB Pulse Generator에서 생성하는 신호로 SOC
- **External Pin(GPIO/XINT2_ADCSOC)** : GPIO 포트 A(GPIO0~31)의 핀들 중 하나를 외부 인터럽트 핀으로 설정하고 외부 신호로 SOC

ADC 변환채널 수 설정

ADC 시퀀서는 각 시퀀서(SEQ1, SEQ2) 별로 8채널까지 변환채널 수를 설정할 수 있습니다. 만일 두 시퀀서를 직렬 시퀀서 모드로 운용한다면 하나의 시퀀서로 16채널까지 변환채널 수를 설정할 수 있습니다. 이러한 변환채널 수는 다음과 같은 ADCMAXCONV 레지스터에서 설정할 수 있습니다. 시퀀서는 ADCMAXCONV 설정에 따라서 한번의 시퀀스에 몇 개의 채널을 변환할 지 결정하게 됩니다.

◈ ADCMAXCONV 레지스터

ADC 시퀀스의 변환 채널 수를 설정하는 레지스터 입니다. 직렬 시퀀서 모드 일 때에는 3번~0번 비트까지 4비트로 설정할 수 있습니다. 병력 시퀀서 모드 일 때에는 2번~0번 비트는 시퀀서1의 변환 채널 수를 설정하고, 6번~4번 비트는 시퀀서2의 변환 채널 수를 설정합니다.

15				8
Reserved				
R-0				

7	6	4	3	0
Reserved	MAX_CONV2		MAX_CONV1	
R-0	R/W-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; x = undefined, *-n* = value after reset

시퀀서 직렬(Cascaded) 모드 운용 시

- 변환채널 수 = "ADCMAXCONV[3:0]-MAX_CONV1 설정값" + 1
- 예) 13개의 채널을 변환 한다면 "MAX_CONV1 = 12"

시퀀서 병렬(Dual) 모드 운용 시

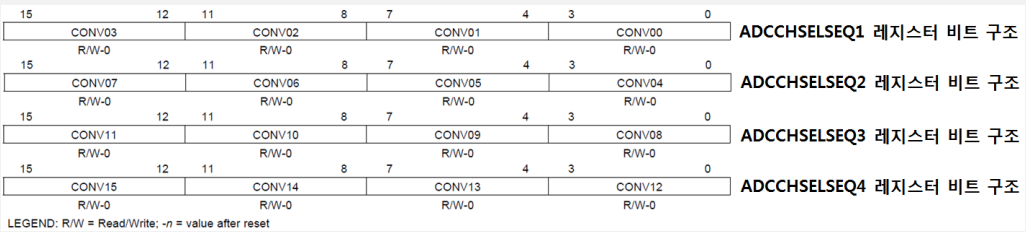
- 시퀀서1(SEQ1)의 변환채널 수 = "ADCMAXCONV[3:0]-MAX_CONV1 설정값" + 1
- 시퀀서2(SEQ2)의 변환채널 수 = "ADCMAXCONV[6:4]-MAX_CONV2 설정값" + 1
- 예) SEQ1에서 7개의 채널, SEQ2에서 1개 채널을 변환 한다면 "MAX_CONV1 = 6", "MAX_CONV2 = 0",

ADC 변환채널 순서 설정

각 시퀀서의 변환 채널 수가 설정됐으면, 이제 시퀀서가 어떤 순서로 채널을 변환하게 할지 설정해야 합니다. 시퀀서1(SEQ1)의 변환채널 순서는 ADCCHSELSEQ1, ADCCHSELSEQ2 두 레지스터에서 1~8번째까지 설정할 수 있습니다. 시퀀서2(SEQ2)의 변환채널 순서는 ADCCHSELSEQ3, ADCCHSELSEQ4 두 레지스터에서 1~8번째까지 설정할 수 있습니다. 만일 두 시퀀서를 직렬 시퀀서 모드로 운용한다면 ADCCHSELSEQ1~4의 4개 레지스터에서 1~16번째까지 변환채널 순서를 설정할 수 있습니다.

◈ ADCCHSELSEQ1~4 레지스터

ADC 시퀀스의 변환 채널 순서를 설정하는 레지스터 입니다.



시퀀서 직렬(Cascaded) 모드 운용 시

- 변환순서 : CONV00 → CONV01 → CONV02 → CONV03 → … CONV015
- 예) 4채널 변환 시 ADCINA2 → ADCINB0 → ADCINB6 → ADCINA1 채널 순서로 변환 한다면 "ADCCHSELSEQ1 = 0x1E82"

시퀀서 병렬(Dual) 모드 운용 시

- 시퀀서1(SEQ1)의 변환순서 : CONV00 → CONV01 → CONV02 → CONV03 → … CONV07
- 시퀀서2(SEQ2)의 변환순서 : CONV08 → CONV09 → CONV10 → CONV11 → … CONV15
- 예) SEQ1에서 2채널 변환 시 ADCINB2 → ADCINA5 채널 순서로 변환 한다면 "ADCCHSELSEQ1 = 0x??5A"
- 예) SEQ2에서 3채널 변환 시 ADCINA7 → ADCINB7 → ADCINB4 채널 순서로 변환 한다면 "ADCCHSELSEQ3 = 0x?CF7"

  * 설정 값에서 "?"는 Don't Care를 의미합니다.

ADC 결과 저장

ADC 시퀀스가 시작되면 시퀀서에 설정된 변환채널 수, 변환채널 순서에 따라 변환과정이 진행되고, 각 채널의 변환결과는 시퀀서 모드에 따라서 ADCRESULT0부터 ADCRESULT15까지의 16개 레지스터에 저장됩니다. 이때 변환결과는 ADCRESULTn 레지스터의 상위 12비트에 저장됩니다.

◈ **ADCRESULT0~15 레지스터**

ADC 시퀀스의 변환결과를 저장하는 레지스터 입니다.

15	14	13	12	11	10	9	8
D11	D10	D9	D8	D7	D6	D5	D4
R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0
7	6	5	4	3			0
D3	D2	D1	D0	Reserved			
R-0	R-0	R-0	R-0	R-0			

LEGEND: R/W = Read/Write; R = Read only; -*n* = value after reset

ADCRESULTn 레지스터 비트 구조 (n=0,1,2,...,15)

시퀀서 직렬(Cascaded) 모드 운용 시

- 변환채널 수와 순서에 따라서 ADCRESULT0부터 ADCRESULT15까지의 16개 레지스터에 순차적으로 저장됨.
- 예) 4채널 변환 시 ADCINA2 → ADCINB0 → ADCINB6 → ADCINA1의 순서로 변환한다면, 변환결과는 다음과 같이 저장됨.
  ADCINA2 → ADCRESULT0 / ADCINB0 → ADCRESULT1 / ADCINB6 → ADCRESULT2 / ADCINA1 → ADCRESULT03

시퀀서 병렬(Dual) 모드 운용 시

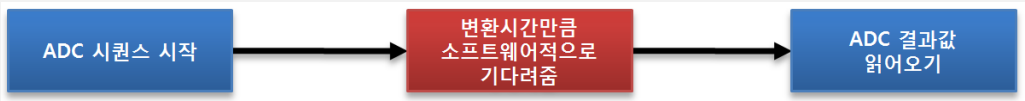
- 시퀀서1(SEQ1)의 변환채널 수와 순서에 따라서 ADCRESULT0부터 ADCRESULT7까지의 8개 레지스터에 순차적으로 저장됨.
- 시퀀서2(SEQ2)의 변환채널 수와 순서에 따라서 ADCRESULT8부터 ADCRESULT15까지의 8개 레지스터에 순차적으로 저장됨.
- 예) SEQ1에서 2채널 변환 시 ADCINB2 → ADCINA5 의 순서로 변환한다면, 변환결과는 다음과 같이 저장됨.
  ADCINB2 → ADCRESULT0 / ADCINA5 → ADCRESULT1
- 예) SEQ2에서 3채널 변환 시 ADCINA7 → ADCINB7 → ADCINB4 의 순서로 변환한다면, 변환결과는 다음과 같이 저장됨.
  ADCINA7 → ADCRESULT8 / ADCINB7 → ADCRESULT9 / ADCINB4 → ADCRESULT10

ADC 결과 읽어오기

사용자가 ADCRESULTn 레지스터에서 저장된 변환결과값을 올바로 읽기 위해서는 반드시 "ADC 시퀀스가 종료 된 후", "다음 ADC 시퀀스가 시작되기 전"의 적절한 타이밍에 변환결과값을 읽어야 합니다. 이를 위해서 다음의 2가지 방법 중 하나를 선택할 수 있습니다.

◈ **Polling 방식으로 결과를 읽는 방법.**

ADC 변환 시간은 구동클럭에 의해서 명확히 결정되므로, 이 시간을 계산해서 기다린 다음 결과값을 읽어들임.



◈ **Interrupt 방식으로 결과를 읽는 방법..**

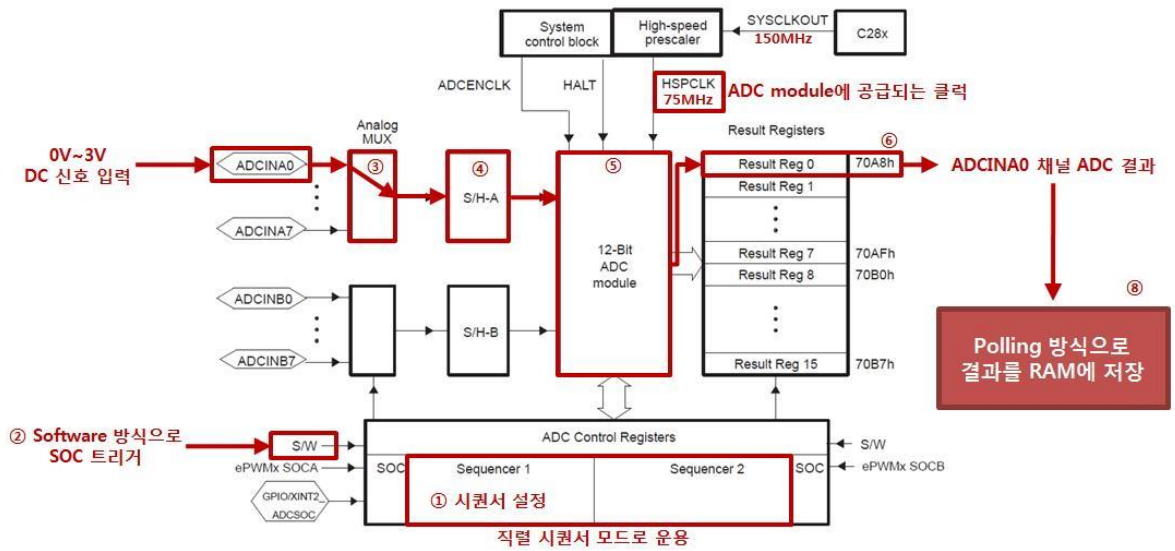
ADC 시퀀스가 종료되는 시점에서 인터럽트가 발생 되도록 설정하고, 이 인터럽트 서비스 루틴에서 결과값을 읽어들임



3.1 ADC 1채널, 비-인터럽트(Polling) 방식

실습 목적

Delfino EVM의 Potentiometer의 DC 출력 신호를 ADC하고, 결과를 CCS의 Watch window로 확인하는 예제입니다. Software SOC로 ADC 시퀀스를 시작하고, 비-인터럽트 방식으로 ADC 결과를 저장하는 과정으로, ADC 클럭 및 S/H(Sample-and-Hold) 시간, ADC 구동 속도에 대한 개념을 습득하실 수 있습니다.



실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC01_1ch_Softwere_SOC_Polling
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC01_1ch_Softwere_SOC_Polling
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC01_1ch_Softwere_SOC_Polling

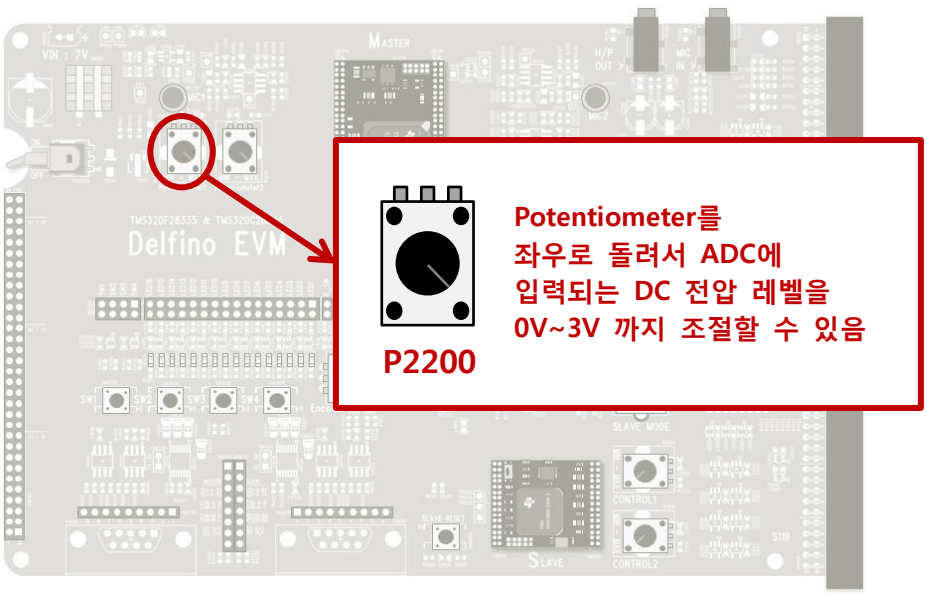
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-01] 1채널, 비-인터럽트(Polling) 방식으로 ADC 하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 1채널, 비-인터럽트(Polling) 방식으로 ADC 하기" 게시물

실습해 보기

Delfino EVM의 Potentiometer(P2200)의 노브를 좌우로 조절하면, ADCINA0 채널에 0.0V ~ 3.0V 사이의 가변 DC 전압 신호를 입력할 수 있습니다.



CCS에서 예제를 실행하고, Watch Window를 통해서 ADC_value01 변수를 관찰하면, 0.0V ~ 3.0V 사이의 가변 DC 전압이 ADC 된 결과를 확인할 수 있습니다.

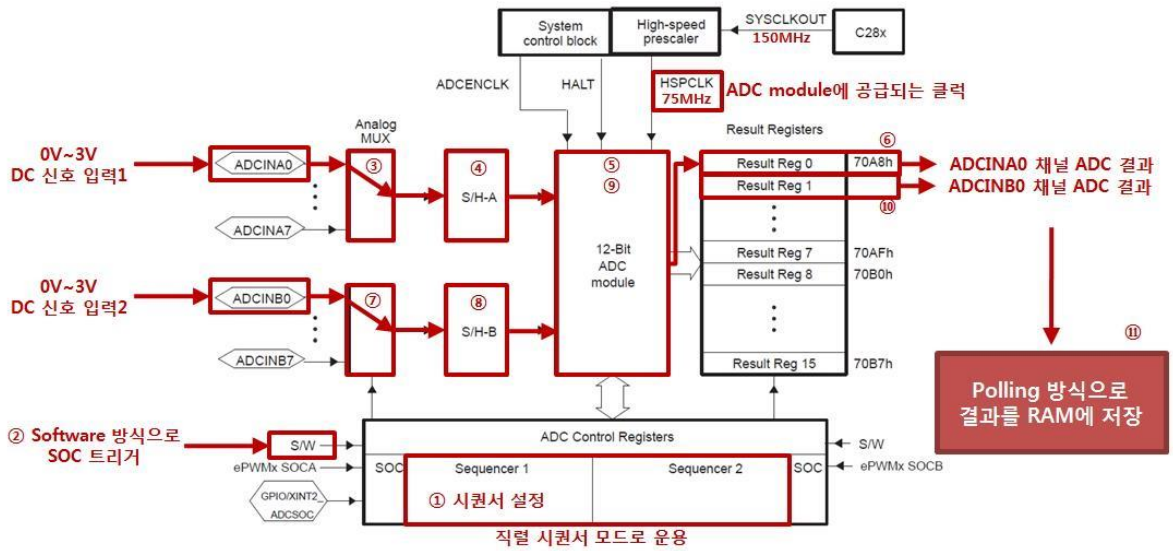
퀴즈!!

본 예제 실행 시 ADCINA0 채널의 샘플링 속도는 몇 Hz 인가요?

3.2 ADC 2채널, 비-인터럽트(Polling) 방식

실습 목적

Delfino EVM 의 2 개의 Potentiometer 의 DC 출력 신호를 2 채널 ADC 하고, 결과를 CCS 의 Watch window 로 확인하는 예제입니다. Software SOC 로 ADC 시퀀스를 시작하고, 비-인터럽트 방식으로 ADC 결과를 저장하는 과정입니다. 다채널 변환 시에 어떠한 것이 달라지는지 확인할 수 있습니다.



실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC02_2ch_Softwere_SOC_Polling
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC02_2ch_Softwere_SOC_Polling
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC02_2ch_Softwere_SOC_Polling

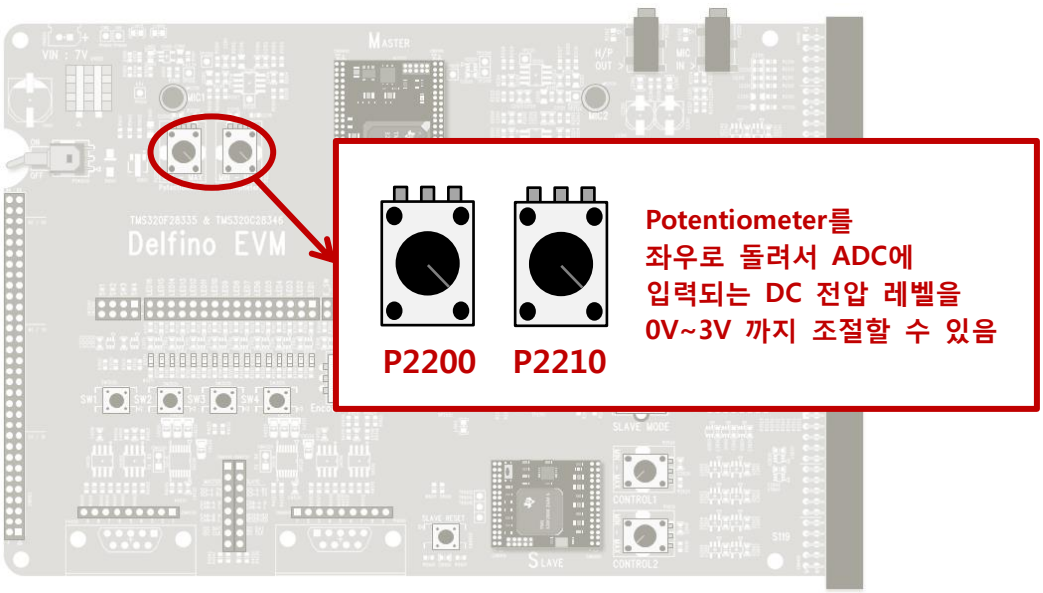
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-02] 2채널, 비-인터럽트(Polling) 방식으로 ADC 하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 2채널, 비-인터럽트(Polling) 방식으로 ADC 하기" 게시물

실습해 보기

Delfino EVM의 Potentiometer P2200와 P2210의 노브를 좌우로 조절하면, 각각 ADCINA0와 ADCINB0 채널에 0.0V ~ 3.0V 사이의 가변 DC 전압 신호를 입력할 수 있습니다.



CCS에서 예제를 실행하고, Watch Window를 통해서 ADC_value01, ADC_value02 변수를 관찰하면, 0.0V ~ 3.0V 사이의 가변 DC 전압이 ADC 된 결과를 확인할 수 있습니다.

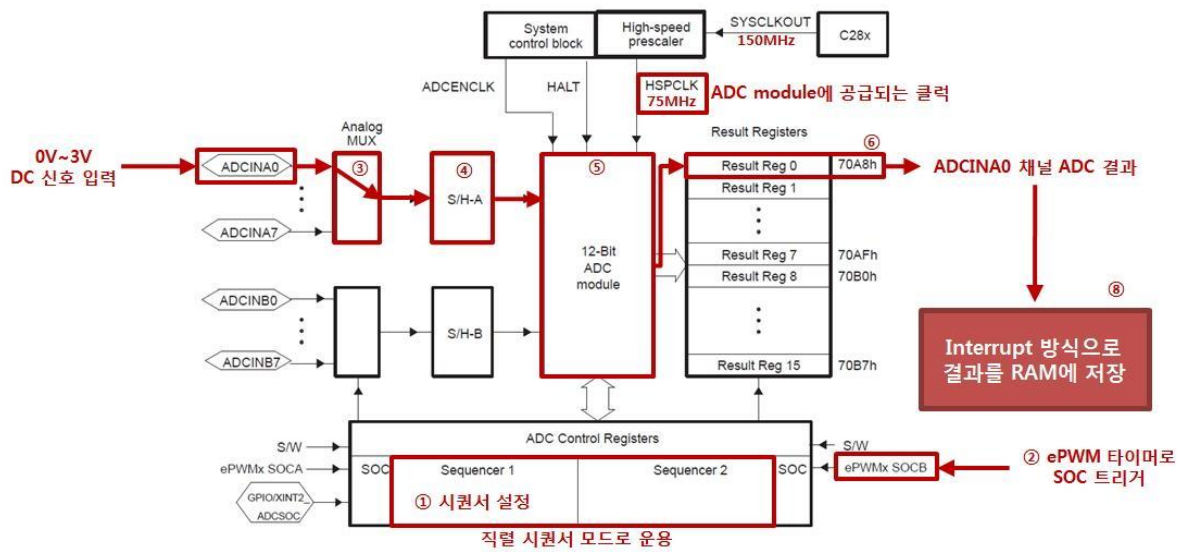
퀴즈!!

본 예제 실행 시 ADCINA0와 ADCINB0 채널의 샘플링 속도는 각각 몇 Hz 인가요?

3.3 ADC 1채널, 인터럽트(Interrupt) 방식

실습 목적

Delfino EVM의 Potentiometer의 DC 출력 신호를 ADC하고, 결과를 CCS의 Watch window로 확인하는 예제입니다. ePWM 모듈의 타이머를 사용해서 정확한 주기의 ePWM SOC로 ADC 시퀀스를 시작하고, 인터럽트 방식으로 ADC 결과를 저장하는 예제입니다. 가장 일반적인 ADC 사용 방법입니다.



실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC03_1ch_Timer_SOC_Interrupt
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC03_1ch_Timer_SOC_Interrupt
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC03_1ch_Timer_SOC_Interrupt

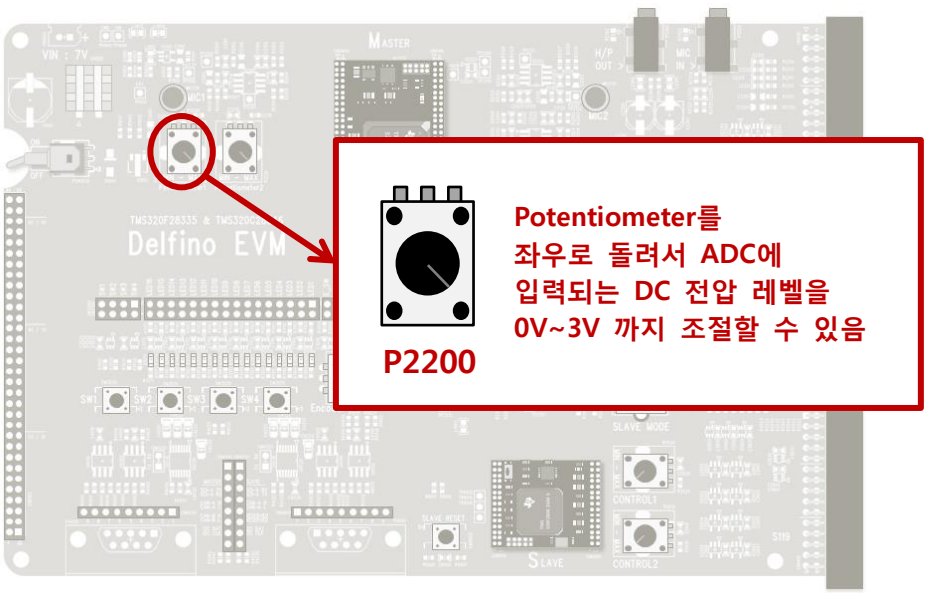
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-03] 1채널, 인터럽트(Interrupt) 방식으로 ADC 하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 1채널, 인터럽트(Interrupt) 방식으로 ADC 하기" 게시물

실습해 보기

Delfino EVM의 Potentiometer(P2200)의 노브를 좌우로 조절하면, ADCINA0 채널에 0.0V ~ 3.0V 사이의 가변 DC 전압 신호를 입력할 수 있습니다.



CCS에서 예제를 실행하고, Watch Window를 통해서 ADC_value01 변수를 관찰하면, 0.0V ~ 3.0V 사이의 가변 DC 전압이 ADC 된 결과를 확인할 수 있습니다.

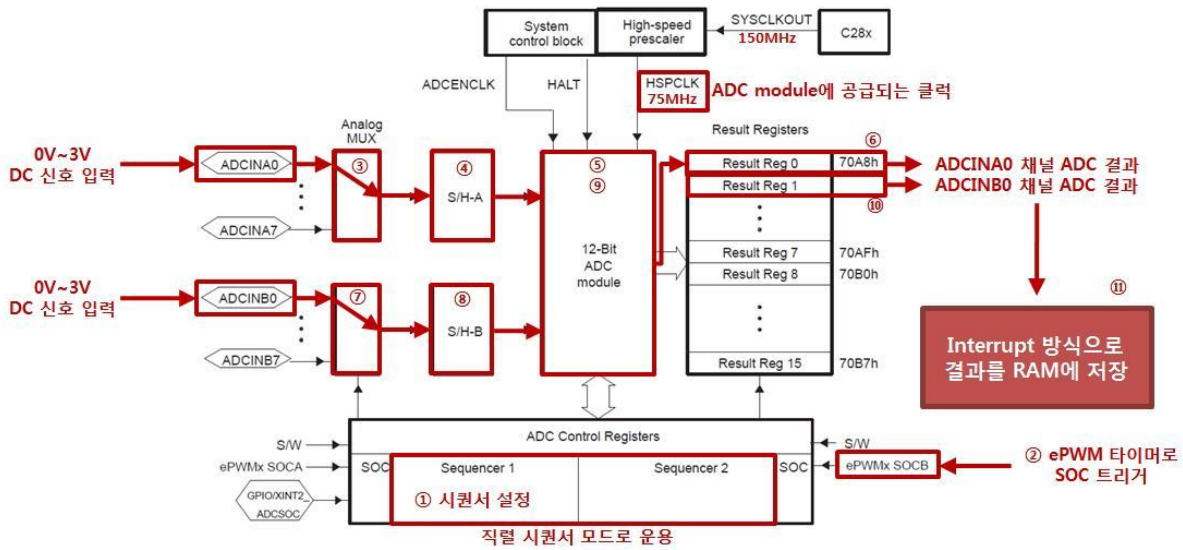
퀴즈!!

본 예제 실행 시 ADCINA0 채널의 샘플링 속도는 몇 Hz 인가요?

3.4 ADC 2채널, 인터럽트(Interrupt) 방식

실습 목적

Delfino EVM의 2개의 Potentiometer의 DC 출력 신호를 2채널 ADC하고, 결과를 CCS의 Watch window로 확인하는 예제입니다. ePWM 모듈의 타이머를 사용해서 정확한 주기의 ePWM SOC로 ADC 시퀀스를 시작하고, 인터럽트 방식으로 ADC 결과를 저장하는 예제입니다.



실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC04_2ch_Timer_SOC_Interrupt
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC04_2ch_Timer_SOC_Interrupt
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC04_2ch_Timer_SOC_Interrupt

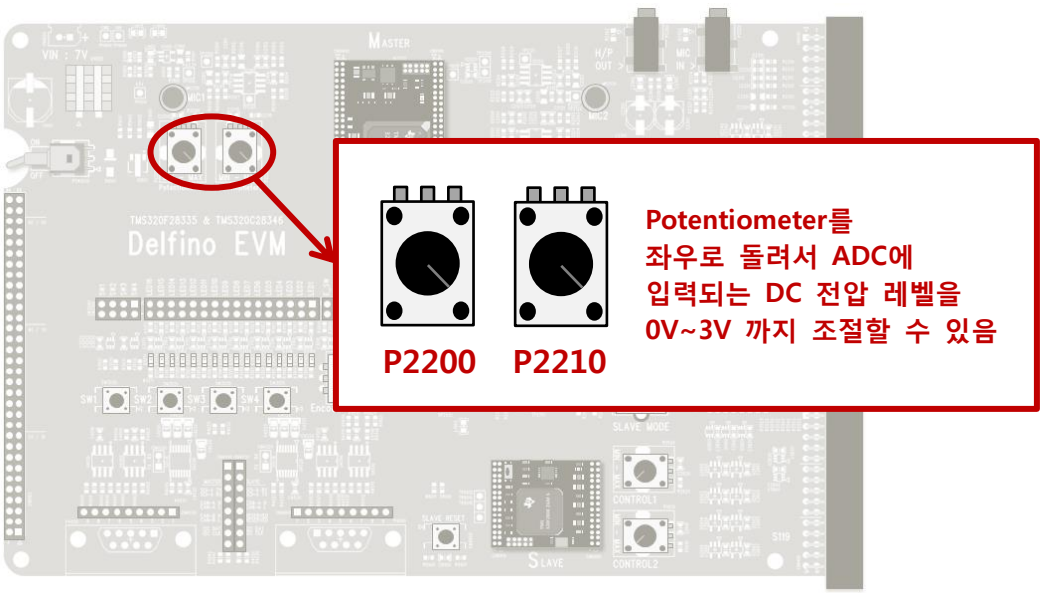
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-04] 2채널, 인터럽트(Interrupt) 방식으로 ADC 하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 2채널, 인터럽트(Interrupt) 방식으로 ADC 하기" 게시물

실습해 보기

Delfino EVM의 Potentiometer P2200와 P2210의 노브를 좌우로 조절하면, 각각 ADCINA0와 ADCINB0 채널에 0.0V ~ 3.0V 사이의 가변 DC 전압 신호를 입력할 수 있습니다.



CCS에서 예제를 실행하고, Watch Window를 통해서 ADC_value01, ADC_value02 변수를 관찰하면, 0.0V ~ 3.0V 사이의 가변 DC 전압이 ADC 된 결과를 확인할 수 있습니다.

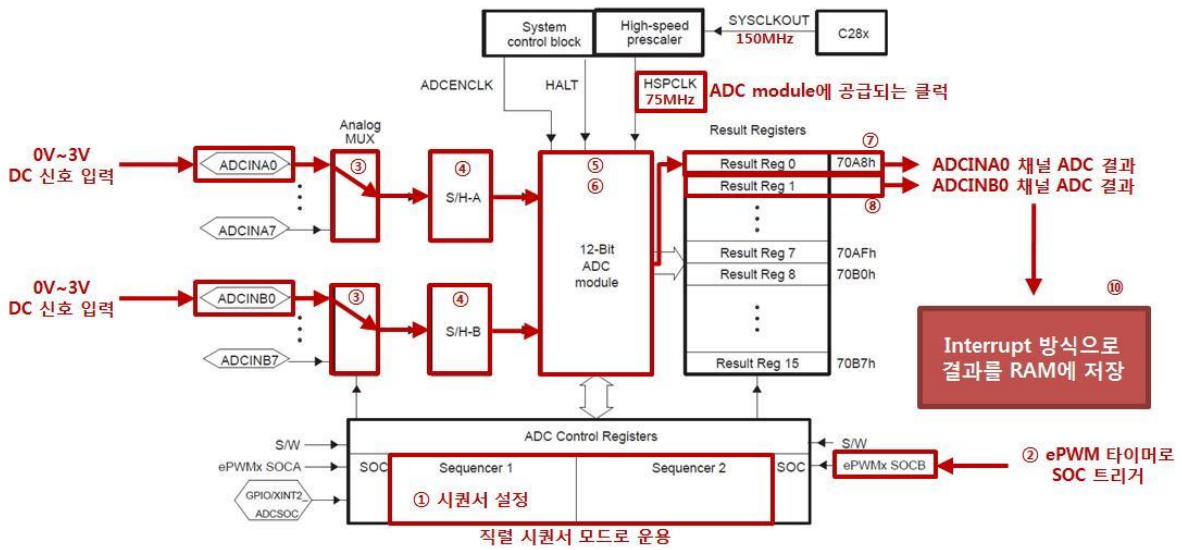
퀴즈!!

본 예제 실행 시 ADCINA0 채널의 샘플링 속도는 몇 Hz 인가요?

3.5 ADC 2채널, 동시(Simultaneous) 모드 운용

실습 목적

Delfino EVM 의 2 개의 Potentiometer 의 DC 출력 신호를 2 채널 ADC 하고, 결과를 CCS 의 Watch window 로 확인하는 예제입니다. TMS320F28x ADC 의 동시(Simultaneous) 모드를 사용해서 2 채널을 순차적으로 ADC 하는 것이 아닌, 동일한 시점에 ADC 하는 방식입니다.



실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC05_2ch_Simultaneus
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC05_2ch_Simultaneus
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC05_2ch_Simultaneus

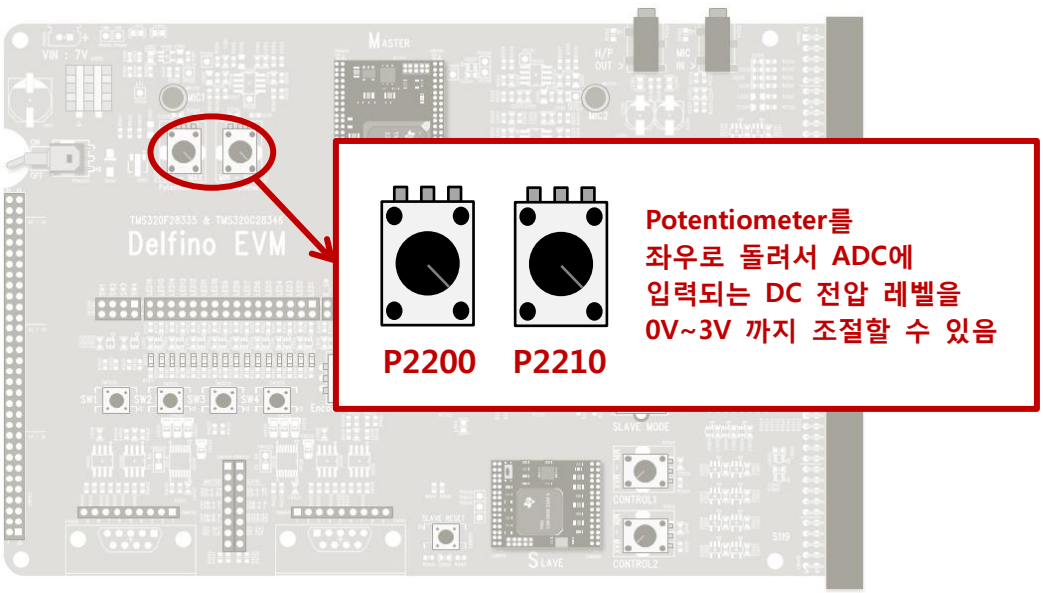
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-05] 2채널, 동시(Simultaneous) 모드 운용" 폴더

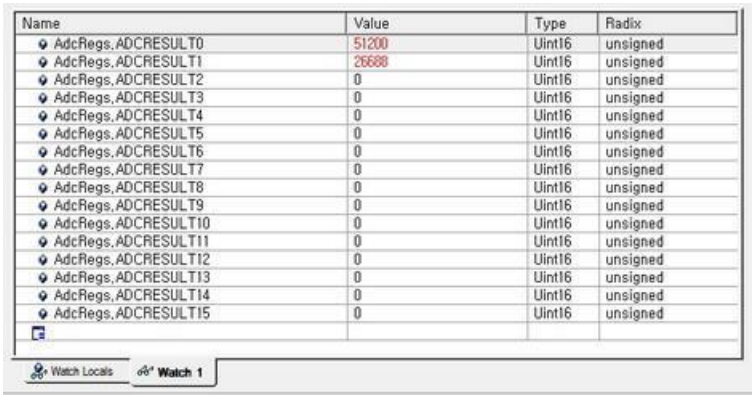
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM – 2채널, 동시(Simultaneous) 모드 운용" 게시물

실습해 보기

Delfino EVM의 Potentiometer P2200와 P2210의 노브를 좌우로 조절하면, 각각 ADCINA0와 ADCINB0 채널에 0.0V ~ 3.0V 사이의 가변 DC 전압 신호를 입력할 수 있습니다.



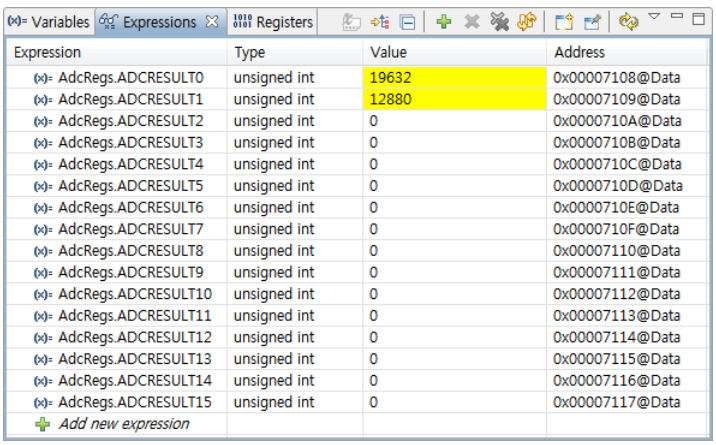
CCS에서 예제를 실행하고, CCSv3.3의 경우는 Watch Window를 통해서 다음과 같이 모든 ADC Result 레지스터를 관찰해서 몇 번 ADC Result 레지스터에 어떤 결과가 나타나는지 확인할 수 있습니다.



Name	Value	Type	Radix
AdcRegs.ADCRESULT0	51200	Uint16	unsigned
AdcRegs.ADCRESULT1	26688	Uint16	unsigned
AdcRegs.ADCRESULT2	0	Uint16	unsigned
AdcRegs.ADCRESULT3	0	Uint16	unsigned
AdcRegs.ADCRESULT4	0	Uint16	unsigned
AdcRegs.ADCRESULT5	0	Uint16	unsigned
AdcRegs.ADCRESULT6	0	Uint16	unsigned
AdcRegs.ADCRESULT7	0	Uint16	unsigned
AdcRegs.ADCRESULT8	0	Uint16	unsigned
AdcRegs.ADCRESULT9	0	Uint16	unsigned
AdcRegs.ADCRESULT10	0	Uint16	unsigned
AdcRegs.ADCRESULT11	0	Uint16	unsigned
AdcRegs.ADCRESULT12	0	Uint16	unsigned
AdcRegs.ADCRESULT13	0	Uint16	unsigned
AdcRegs.ADCRESULT14	0	Uint16	unsigned
AdcRegs.ADCRESULT15	0	Uint16	unsigned

Watch Locals | Watch 1

CCSv4, CCSv5의 경우에는 Expression Window를 통해서 결과를 확인할 수 있습니다.



Variables | Expressions | Registers

Expression	Type	Value	Address
AdcRegs.ADCRESULT0	unsigned int	19632	0x00007108@Data
AdcRegs.ADCRESULT1	unsigned int	12880	0x00007109@Data
AdcRegs.ADCRESULT2	unsigned int	0	0x0000710A@Data
AdcRegs.ADCRESULT3	unsigned int	0	0x0000710B@Data
AdcRegs.ADCRESULT4	unsigned int	0	0x0000710C@Data
AdcRegs.ADCRESULT5	unsigned int	0	0x0000710D@Data
AdcRegs.ADCRESULT6	unsigned int	0	0x0000710E@Data
AdcRegs.ADCRESULT7	unsigned int	0	0x0000710F@Data
AdcRegs.ADCRESULT8	unsigned int	0	0x00007110@Data
AdcRegs.ADCRESULT9	unsigned int	0	0x00007111@Data
AdcRegs.ADCRESULT10	unsigned int	0	0x00007112@Data
AdcRegs.ADCRESULT11	unsigned int	0	0x00007113@Data
AdcRegs.ADCRESULT12	unsigned int	0	0x00007114@Data
AdcRegs.ADCRESULT13	unsigned int	0	0x00007115@Data
AdcRegs.ADCRESULT14	unsigned int	0	0x00007116@Data
AdcRegs.ADCRESULT15	unsigned int	0	0x00007117@Data
Add new expression			

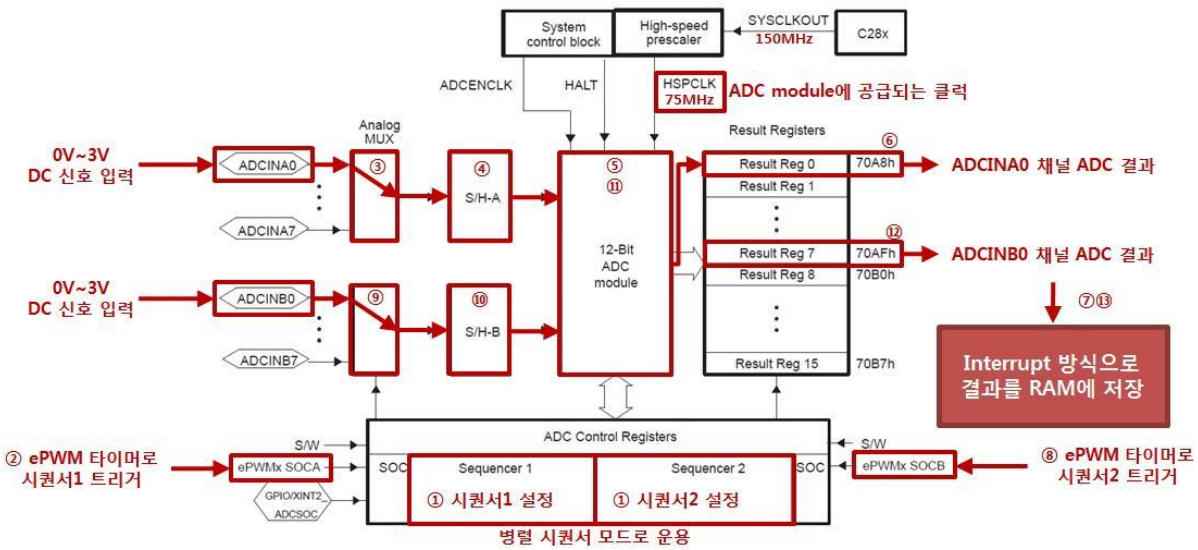
퀴즈!!

앞서 "2채널, 인터럽트 방식으로 ADC 하기 예제"와 ADC 초기화 코드에서 달라진 부분이 무엇인지 찾아보세요.

3.6 ADC 2채널, 병렬 시퀀서(Dual-Sequencer) 모드 운용

실습 목적

Delfino EVM 의 2 개의 Potentiometer 의 DC 출력 신호를 2 채널 ADC 하고, 결과를 CCS 의 Watch window 로 확인하는 예제입니다. TMS320F28x ADC 의 병렬-시퀀서(Dual-Sequencer) 모드를 사용해서 각각의 시퀀서(SEQ1, SEQ2)가 각각 1 채널씩 ADC 하는 방식입니다.



실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC06_2ch_DualSequencer
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC06_2ch_DualSequencer
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC06_2ch_DualSequencer

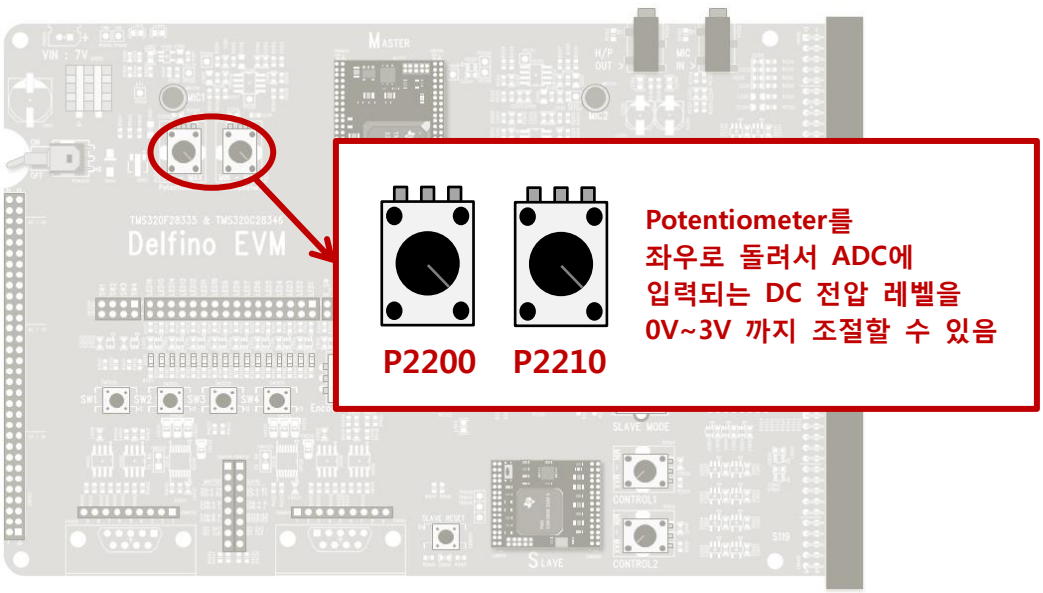
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-06] 2채널, 병렬 시퀀서(Dual-Sequencer) 모드 운용" 폴더

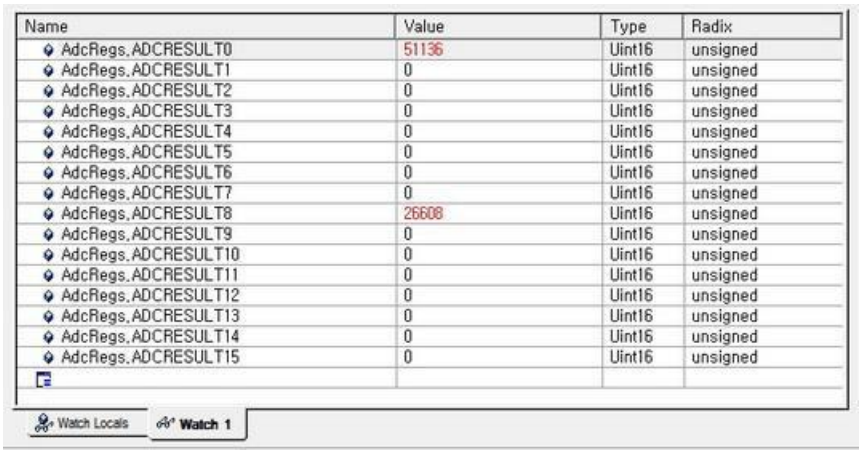
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM – 2채널, 병렬 시퀀서(Dual-Sequencer) 모드 운용" 게시물

실습해 보기

Delfino EVM의 Potentiometer P2200와 P2210의 노브를 좌우로 조절하면, 각각 ADCINA0와 ADCINB0 채널에 0.0V ~ 3.0V 사이의 가변 DC 전압 신호를 입력할 수 있습니다.



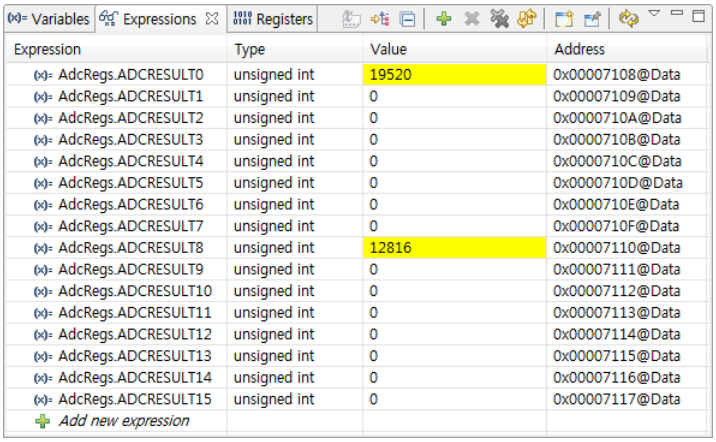
CCS에서 예제를 실행하고, CCSv3.3의 경우는 Watch Window를 통해서 ADC_value01, ADC_value02 변수를 관찰하면, 0.0V ~ 3.0V 사이의 가변 DC 전압이 ADC 된 결과를 확인할 수 있습니다.



Name	Value	Type	Radix
AdcRegs.ADCRESULT0	51136	Uint16	unsigned
AdcRegs.ADCRESULT1	0	Uint16	unsigned
AdcRegs.ADCRESULT2	0	Uint16	unsigned
AdcRegs.ADCRESULT3	0	Uint16	unsigned
AdcRegs.ADCRESULT4	0	Uint16	unsigned
AdcRegs.ADCRESULT5	0	Uint16	unsigned
AdcRegs.ADCRESULT6	0	Uint16	unsigned
AdcRegs.ADCRESULT7	0	Uint16	unsigned
AdcRegs.ADCRESULT8	26608	Uint16	unsigned
AdcRegs.ADCRESULT9	0	Uint16	unsigned
AdcRegs.ADCRESULT10	0	Uint16	unsigned
AdcRegs.ADCRESULT11	0	Uint16	unsigned
AdcRegs.ADCRESULT12	0	Uint16	unsigned
AdcRegs.ADCRESULT13	0	Uint16	unsigned
AdcRegs.ADCRESULT14	0	Uint16	unsigned
AdcRegs.ADCRESULT15	0	Uint16	unsigned

Watch Locals | Watch 1

CCSv4, CCSv5의 경우에는 Expression Window를 통해서 결과를 확인할 수 있습니다.



Variables | Expressions | Registers

Expression	Type	Value	Address
AdcRegs.ADCRESULT0	unsigned int	19520	0x00007108@Data
AdcRegs.ADCRESULT1	unsigned int	0	0x00007109@Data
AdcRegs.ADCRESULT2	unsigned int	0	0x0000710A@Data
AdcRegs.ADCRESULT3	unsigned int	0	0x0000710B@Data
AdcRegs.ADCRESULT4	unsigned int	0	0x0000710C@Data
AdcRegs.ADCRESULT5	unsigned int	0	0x0000710D@Data
AdcRegs.ADCRESULT6	unsigned int	0	0x0000710E@Data
AdcRegs.ADCRESULT7	unsigned int	0	0x0000710F@Data
AdcRegs.ADCRESULT8	unsigned int	12816	0x00007110@Data
AdcRegs.ADCRESULT9	unsigned int	0	0x00007111@Data
AdcRegs.ADCRESULT10	unsigned int	0	0x00007112@Data
AdcRegs.ADCRESULT11	unsigned int	0	0x00007113@Data
AdcRegs.ADCRESULT12	unsigned int	0	0x00007114@Data
AdcRegs.ADCRESULT13	unsigned int	0	0x00007115@Data
AdcRegs.ADCRESULT14	unsigned int	0	0x00007116@Data
AdcRegs.ADCRESULT15	unsigned int	0	0x00007117@Data
Add new expression			

퀴즈!!

앞서 "2채널, 인터럽트 방식으로 ADC 하기 예제"와 ADC 초기화 코드에서 달라진 부분이 무엇인지 찾아보세요.

3.7 8채널, 순차(Sequential) ADC 하기

실습 목적

Delfino EVM에는 Potentiometer를 통한 DC 신호 2채널, 외부 마이크 회로를 통한 Sound 신호 2채널, 슬레이브 모듈 쪽에서 보내는 Sine 신호 2채널, 자체 생성한 PWM 신호 2채널과 같은 총 8채널의 다양한 신호가 다음과 같이 마스터 모듈의 ADC에 연결되어 있습니다. 본 예제는 이러한 8개의 신호를 순차적으로 ADC 하는 예제입니다.

ADCINA0	Potentiometer1(P2200) 회로를 통한 0V~3V 가변 DC 신호
ADCINA1	MIC1(Left) 회로를 통한 Sound 신호
ADCINA2	Slave의 PWM-DAC1 회로를 통한 100Hz~5KHz의 Sine 신호
ADCINA3	자체 ePWM1B 채널에서 생성한 PWM 신호
ADCINA4	외부 커넥터 인출
ADCINA5	외부 커넥터 인출
ADCINA6	외부 커넥터 인출
ADCINA7	외부 커넥터 인출
ADCINB0	Potentiometer2 회로를 통한 0V~3V 가변 DC 신호
ADCINB1	MIC2(Left) 회로를 통한 Sound 신호
ADCINB2	Slave의 PWM-DAC2 회로를 통한 100Hz~5KHz의 Sine 신호
ADCINB3	자체 ePWM1A 채널에서 생성한 PWM 신호
ADCINB4	외부 커넥터 인출
ADCINB5	외부 커넥터 인출
ADCINB6	외부 커넥터 인출
ADCINB7	외부 커넥터 인출

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC07_8ch_Sequential
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC07_8ch_Sequential
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC07_8ch_Sequential

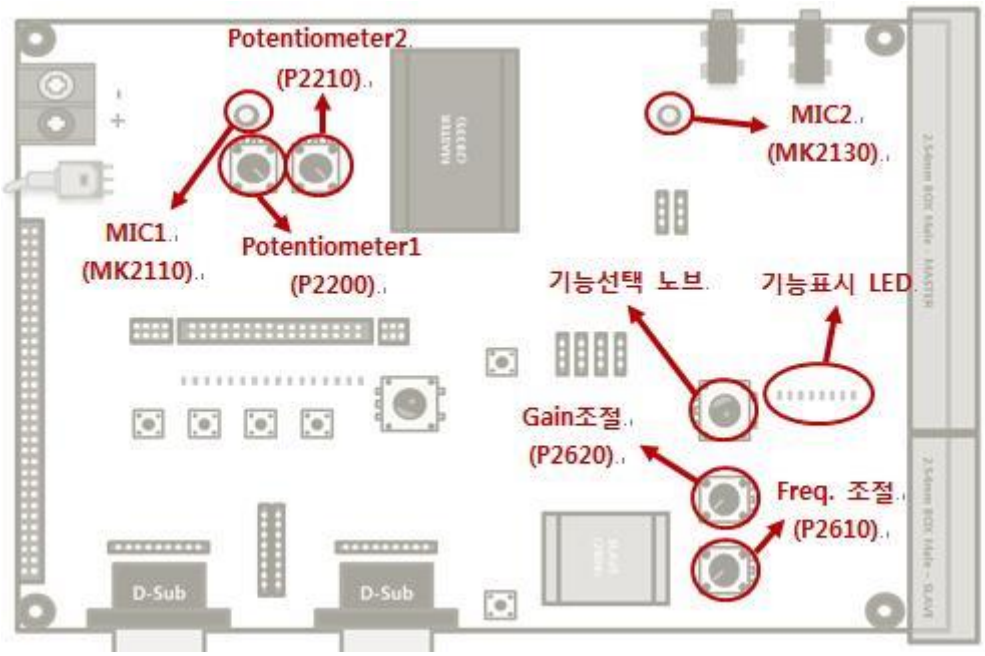
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-07] 8채널, 순차(Sequential) ADC 하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM – 8채널, 순차(Sequential) ADC 하기" 게시물

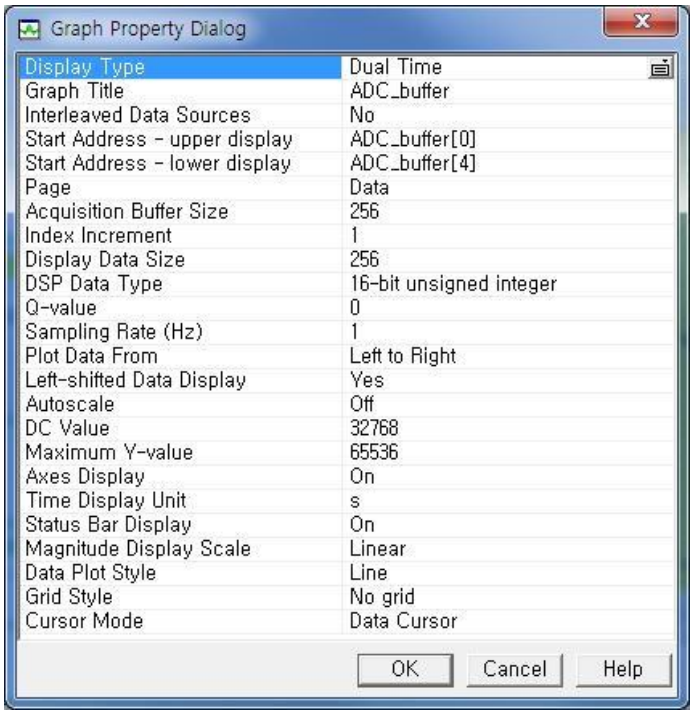
실습해 보기

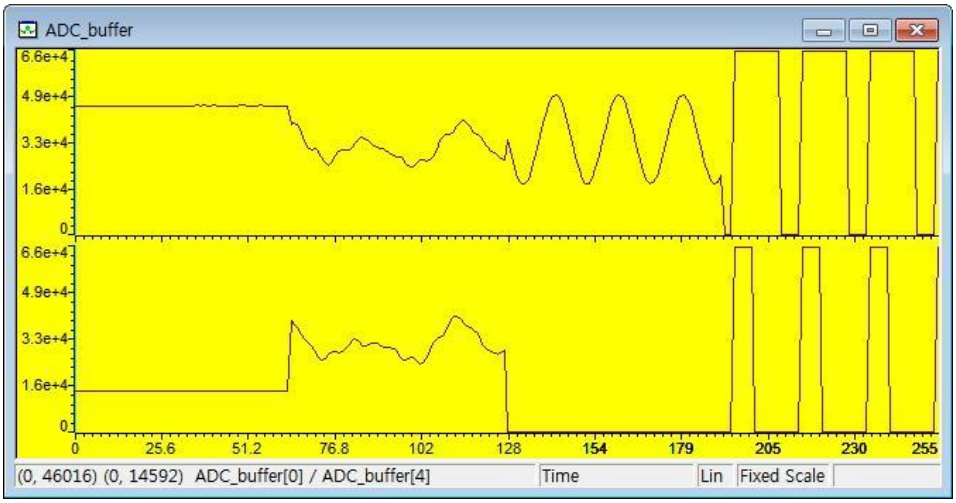
Delfino EVM의 마스터 모듈 ADC에 입력되는 신호는 다음 그림을 참고하여 조절하시기 바랍니다.



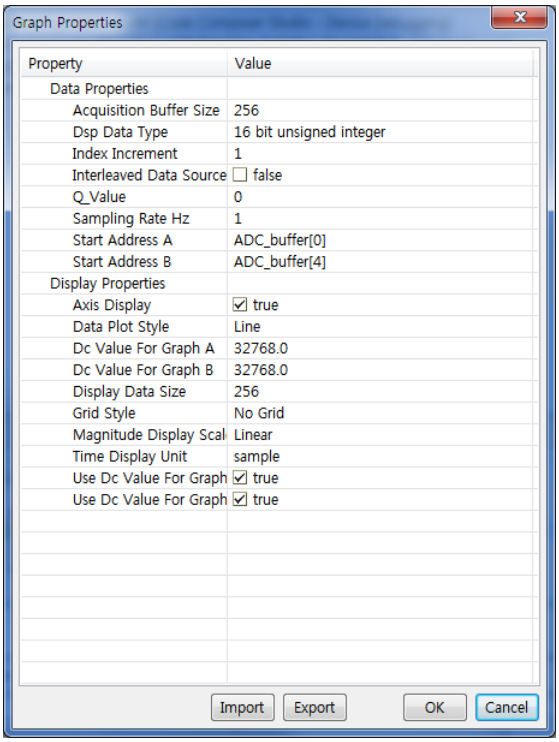
Delfino EVM의 마스터 모듈 ADC에 입력되는 신호들 중 DC 신호 2채널은 Potentiometer1 (P2200)와 Potentiometer2 (P2210) 노브를 좌우로 조절해서 0V~3V까지 가변할 수 있습니다. 다음 Sound 신호 2채널은 MIC1, MIC2를 통해서 입력할 수 있습니다. 다음 Sine 신호 2채널은 기능선택 노브를 좌우로 돌려 기능표시 LED의 왼쪽에서 3번째 혹은 4번째 LED가 켜지게 조절하고, 기능선택 노브를 눌러야 합니다. 출력되는 Sine 신호의 주파수와 게인은 P2620과 P2610 노브를 통해서 조절할 수 있습니다.

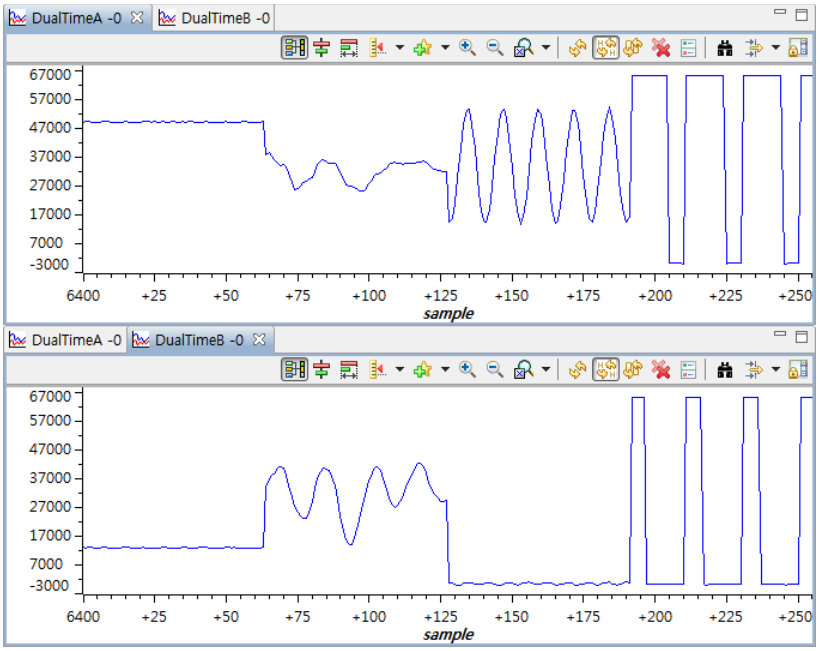
CCS에서 예제를 실행하고, CCS의 Graph 기능을 통해서 ADC 결과를 시각적으로 관찰할 수 있습니다. 다음은 CCSv3.3의 그래프 관찰 예 입니다.





다음은 CCSv5 의 그래프 관찰 예 입니다.





3.8 8채널, 동시(Simultaneous), 병렬(Dual) ADC 하기

실습 목적

Delfino EVM에는 Potentiometer를 통한 DC 신호 2채널, 외부 마이크 회로를 통한 Sound 신호 2채널, 슬레이브 모듈 쪽에서 보내는 Sine 신호 2채널, 자체 생성한 PWM 신호 2채널과 같은 총 8채널의 다양한 신호가 다음과 같이 마스터 모듈의 ADC에 연결되어 있습니다. 본 예제는 이러한 8개의 신호를 동시(Simultaneous) 샘플링 모드와 병렬(Dual) 시퀀서 모드를 함께 사용하여 ADC 해보는 예제 입니다.

ADCINA0	Potentiometer1(P2200) 회로를 통한 0V~3V 가변 DC 신호
ADCINA1	MIC1(Left) 회로를 통한 Sound 신호
ADCINA2	Slave의 PWM-DAC1 회로를 통한 100Hz~5KHz의 Sine 신호
ADCINA3	자체 ePWM1B 채널에서 생성한 PWM 신호
ADCINA4	외부 커넥터 인출
ADCINA5	외부 커넥터 인출
ADCINA6	외부 커넥터 인출
ADCINA7	외부 커넥터 인출
ADCINB0	Potentiometer2 회로를 통한 0V~3V 가변 DC 신호
ADCINB1	MIC2(Left) 회로를 통한 Sound 신호
ADCINB2	Slave의 PWM-DAC2 회로를 통한 100Hz~5KHz의 Sine 신호
ADCINB3	자체 ePWM1A 채널에서 생성한 PWM 신호
ADCINB4	외부 커넥터 인출
ADCINB5	외부 커넥터 인출
ADCINB6	외부 커넥터 인출
ADCINB7	외부 커넥터 인출

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC08_8ch_Simultaneus_Dual
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC08_8ch_Simultaneus_Dual
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC08_8ch_Simultaneus_Dual

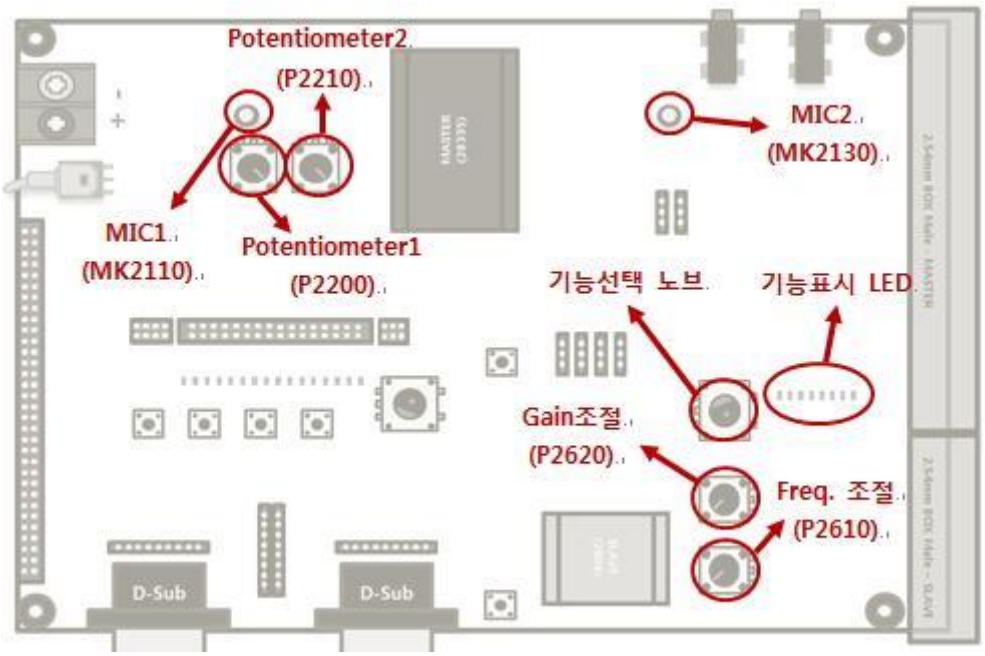
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-08] 8채널, 동시(Simultaneous), 병렬(Dual) ADC 하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM – 8채널, 동시(Simultaneous), 병렬(Dual) ADC 하기" 게시물

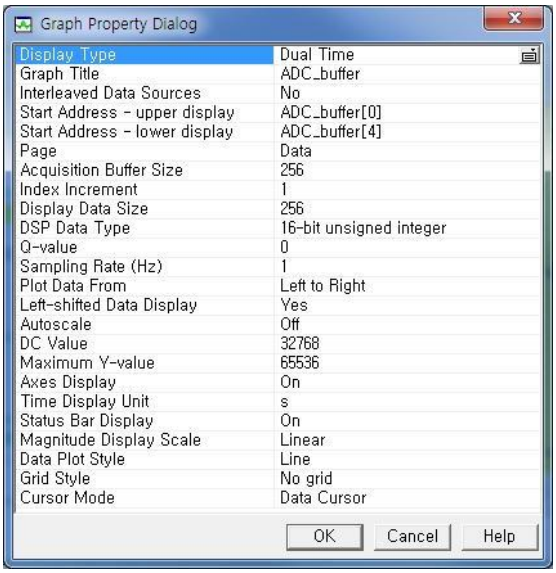
실습해 보기

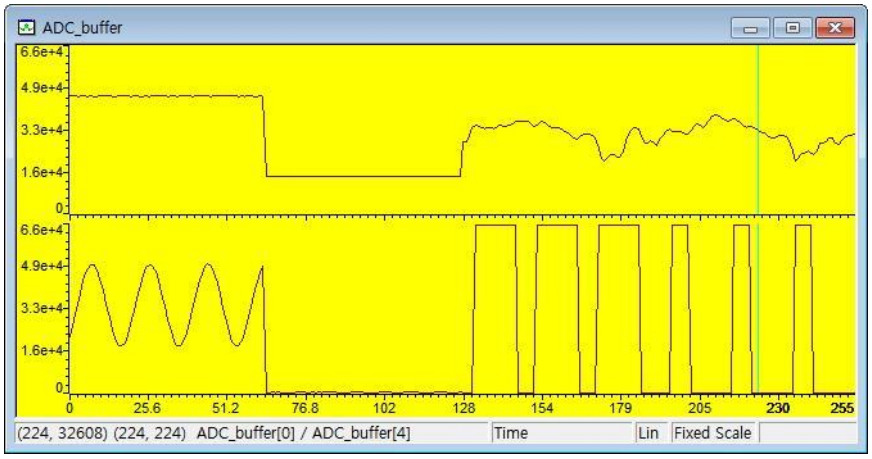
Delfino EVM의 마스터 모듈 ADC에 입력되는 신호는 다음 그림을 참고하여 조절하시기 바랍니다.



Delfino EVM의 마스터 모듈 ADC에 입력되는 신호들 중 DC 신호 2채널은 Potentiometer1 (P2200)와 Potentiometer2 (P2210) 노브를 좌우로 조절해서 0V~3V까지 가변할 수 있습니다. 다음 Sound 신호 2채널은 MIC1, MIC2를 통해서 입력할 수 있습니다. 다음 Sine 신호 2채널은 기능선택 노브를 좌우로 돌려 기능표시 LED의 왼쪽에서 3번째 혹은 4번째 LED가 켜지게 조절하고, 기능선택 노브를 눌러야 합니다. 출력되는 Sine 신호의 주파수와 게인은 P2620과 P2610 노브를 통해서 조절할 수 있습니다.

CCS에서 예제를 실행하고, CCS의 Graph 기능을 통해서 ADC 결과를 시각적으로 관찰할 수 있습니다. 다음은 CCSv3.3의 그래프 관찰 예 입니다

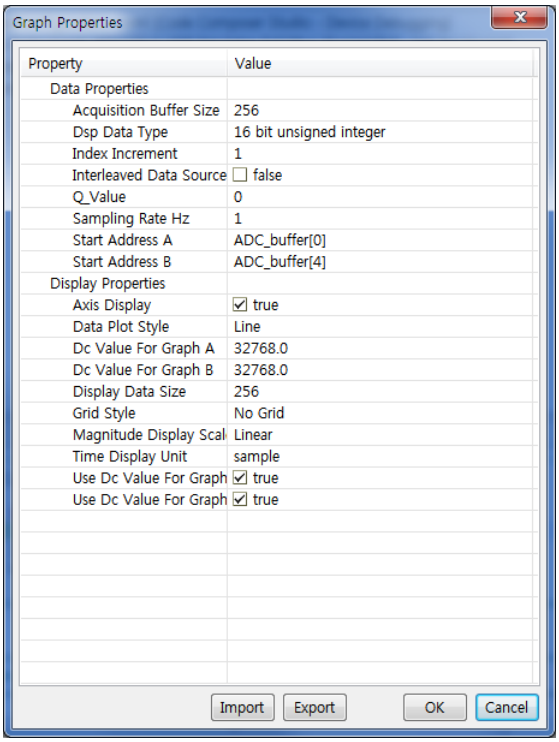


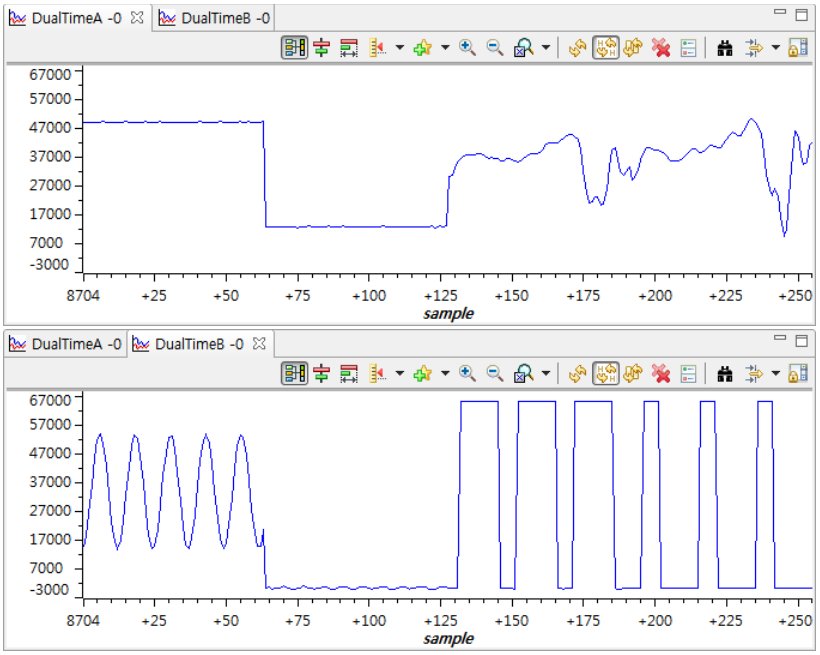


퀴즈!!

앞서 "8채널 순차(Sequential) ADC 하기"와 ADC 초기화 코드에서 달라진 부분이 무엇인지 찾아보세요.

다음은 CCSv5 의 그래프 관찰 예 입니다.





3.9 ADC 8채널, Mode 2 인터럽트 결과저장

실습 목적

Delfino EVM 에는 Potentiometer 를 통한 DC 신호 2 채널, 외부 마이크 회로를 통한 Sound 신호 2 채널, 슬레이브 모듈 쪽에서 보내는 Sine 신호 2 채널, 자체 생성한 PWM 신호 2 채널과 같은 총 8 채널의 다양한 신호가 다음과 같이 마스터 모듈의 ADC 에 연결되어 있습니다.

ADCINA0	Potentiometer1(P2200) 회로를 통한 0V~3V 가변 DC 신호
ADCINA1	MIC1(Left) 회로를 통한 Sound 신호
ADCINA2	Slave의 PWM-DAC1 회로를 통한 100Hz~5KHz의 Sine 신호
ADCINA3	자체 ePWM1B 채널에서 생성한 PWM 신호
ADCINA4	외부 커넥터 인출
ADCINA5	외부 커넥터 인출
ADCINA6	외부 커넥터 인출
ADCINA7	외부 커넥터 인출
ADCINB0	Potentiometer2 회로를 통한 0V~3V 가변 DC 신호
ADCINB1	MIC2(Left) 회로를 통한 Sound 신호
ADCINB2	Slave의 PWM-DAC2 회로를 통한 100Hz~5KHz의 Sine 신호
ADCINB3	자체 ePWM1A 채널에서 생성한 PWM 신호
ADCINB4	외부 커넥터 인출
ADCINB5	외부 커넥터 인출
ADCINB6	외부 커넥터 인출
ADCINB7	외부 커넥터 인출

본 예제는 이러한 8개의 신호를 첫 번째로 4채널 순차적으로 ADC하고, 두 번째로 나머지 4채널을 ADC한 다음 총 8채널의 ADC 결과를 저장하는 예제입니다. 즉, 매 2번째 ADC 시퀀스가 종료될 때 ADC 인터럽트가 발생하고, 인터럽트 서비스 루틴에서 2번의 ADC 시퀀스로 축적된 ADC 결과를 저장하는 방식입니다. 이러한 방식으로 인터럽트 처리를 위한 CPU 부하를 2배 줄일 수 있습니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC09_8ch_INT_MOD
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC09_8ch_INT_MOD
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC09_8ch_INT_MOD

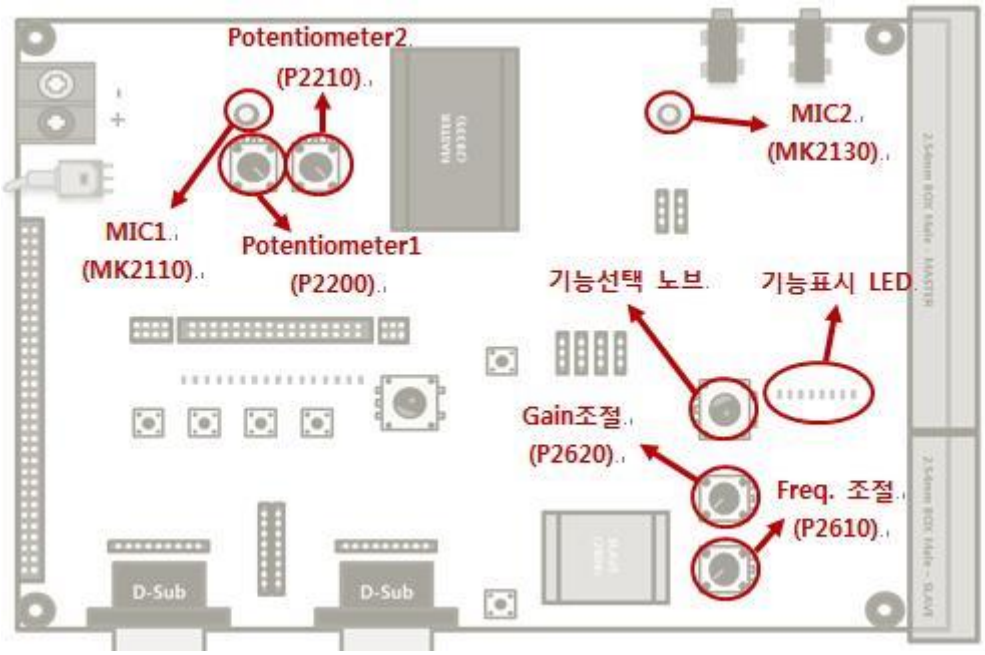
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-09] 8채널, Mode 2 인터럽트 결과저장" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM – 8채널, ADC, Mode 2 인터럽트 결과저장" 게시물

실습해 보기

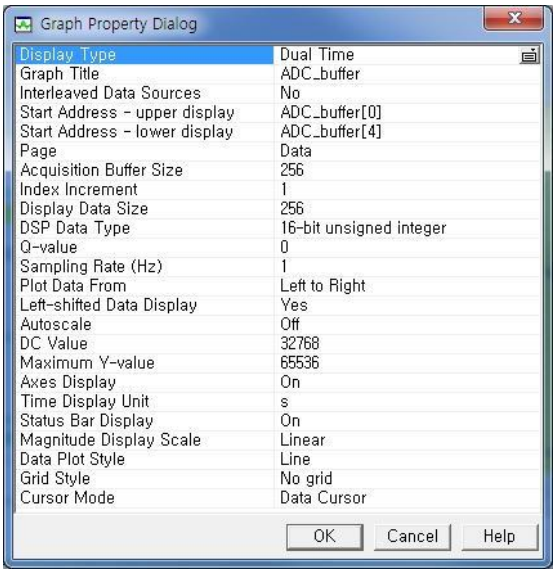
Delfino EVM의 마스터 모듈 ADC에 입력되는 신호는 다음 그림을 참고하여 조절하시기 바랍니다.

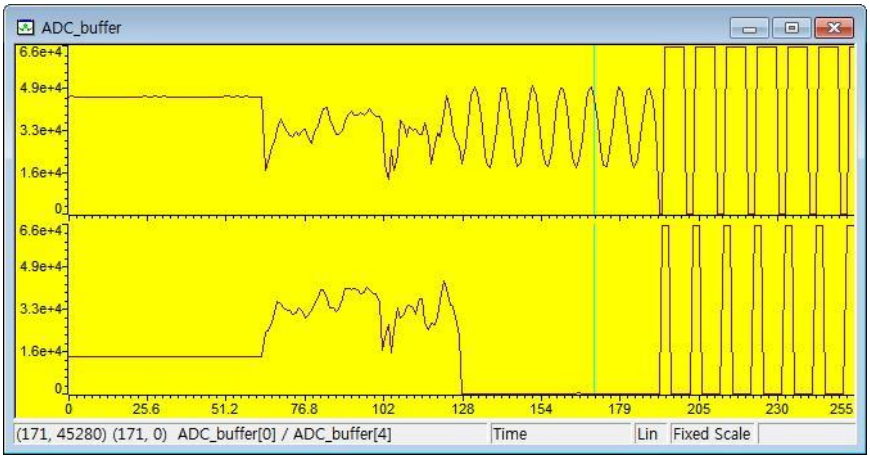


Delfino EVM의 마스터 모듈 ADC에 입력되는 신호들 중 DC 신호 2채널은 Potentiometer1 (P2200)와 Potentiometer2 (P2210) 노브를 좌우로 조절해서 0V~3V까지 가변할 수 있습니다. 다음 Sound 신호 2채널은 MIC1, MIC2를 통해서 입력할 수 있습니다. 다음 Sine 신호 2채널은 기능선택 노브를 좌우로 돌려 기능표시 LED의 왼쪽에서 3번째 혹은 4번째 LED가 켜지게 조절하고, 기능선택 노브를 눌러야 합니다. 출력되는 Sine 신호의 주파수와 게인은 P2620과 P2610 노브를 통해서 조절할 수 있습니다.

CCS에서 예제를 실행하고, CCS의 Graph 기능을 통해서 ADC 결과를 시각적으로 관찰할 수 있습니다.

다음은 CCSv3.3의 그래프 관찰 예 입니다.

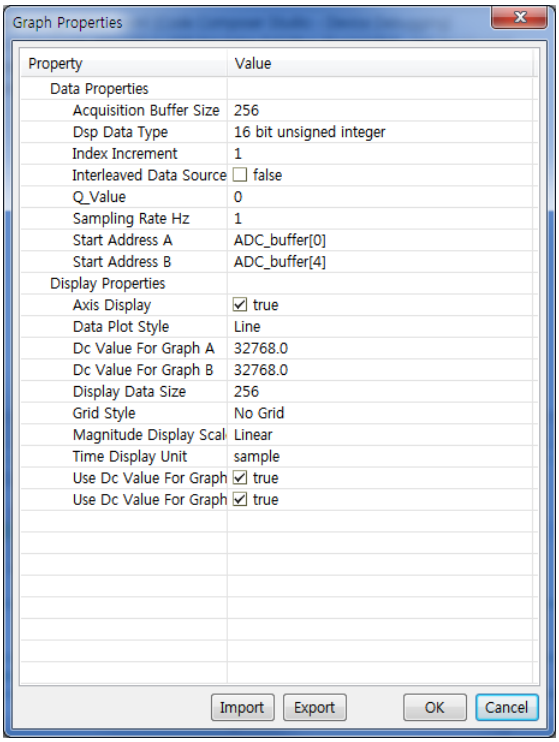


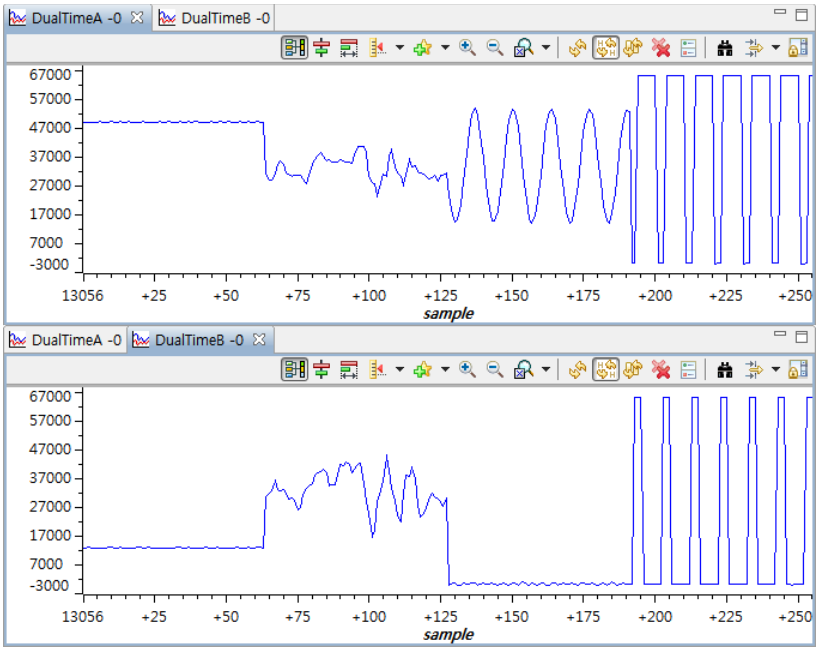


퀴즈!!

앞서 "8채널 순차(Sequential) ADC 하기"와 ADC 초기화 코드에서 달라진 부분이 무엇인지 찾아보세요..

다음은 CCSv5 의 그래프 관찰 예 입니다.





3.10 ADC 1채널, 오버샘플링(Over-Sampling) 하기

실습 목적

예제는 Delfino EVM 의 슬레이브 모듈 쪽에서 보내는 Sine 신호 1 채널을 20KHz 의 샘플링 속도로 ADC 하지만, 한번의 ADC 시퀀스에서 3.125MHz 의 샘플링 속도로 16 개의 ADC 결과를 수집하여 평균값을 취하여 결과를 저장하는 방식의 오버 샘플링 기법을 실습합니다. 이러한 오버 샘플링에 의해서 ADC 정밀도(유효비트)를 높일 수 있습니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ADC10_1ch_Over_Sampling
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ADC10_1ch_Over_Sampling
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ADC10_1ch_Over_Sampling

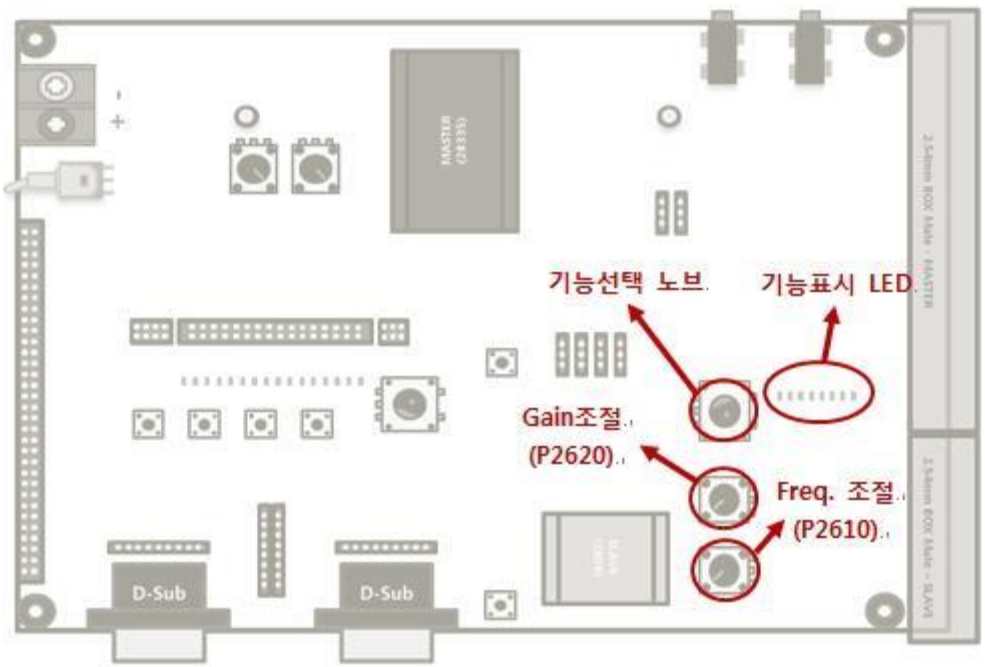
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[ADC-10] 1채널, 오버샘플링(Over-Sampling) ADC" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM – 1채널, 오버샘플링(Over-Sampling) ADC" 게시물

실습해 보기

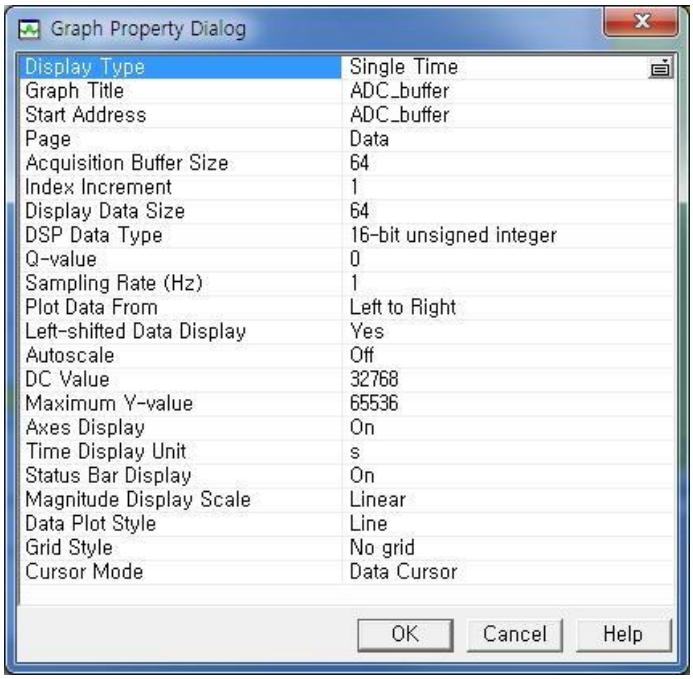
Delfino EVM의 마스터 모듈 ADC에 입력되는 신호는 다음 그림을 참고하여 조절하시기 바랍니다.

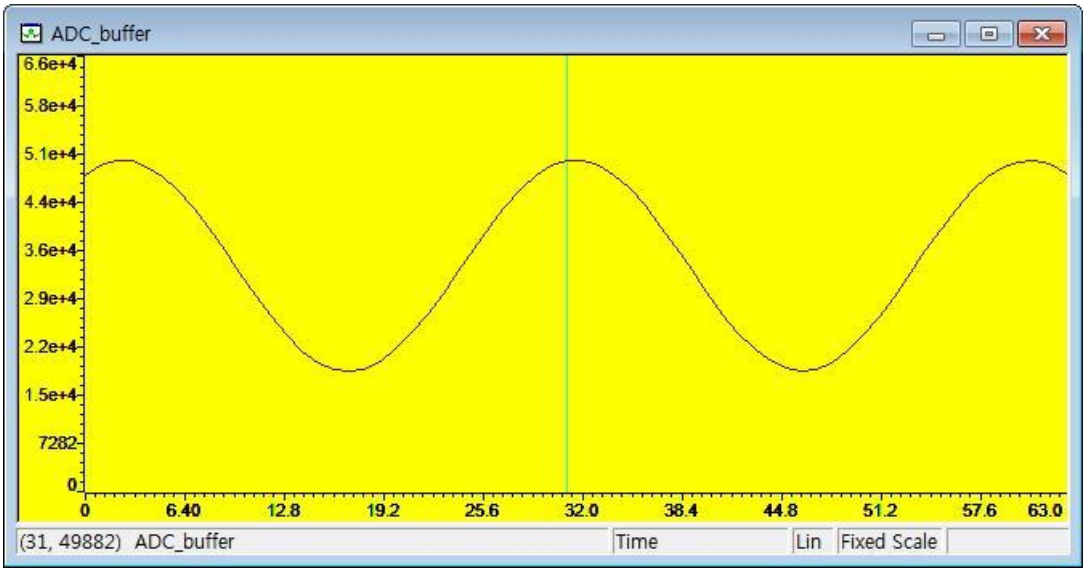


Delfino EVM의 마스터 모듈 ADC에 입력되는 신호들 중 Sine 신호는 기능선택 노브를 좌우로 돌려 기능표시 LED의 왼쪽에서 3번째 LED가 켜지게 조절하고, 기능선택 노브를 눌러야 출력됩니다. 출력되는 Sine 신호의 주파수와 게인은 P2620과 P2610 노브를 통해서 조절할 수 있습니다

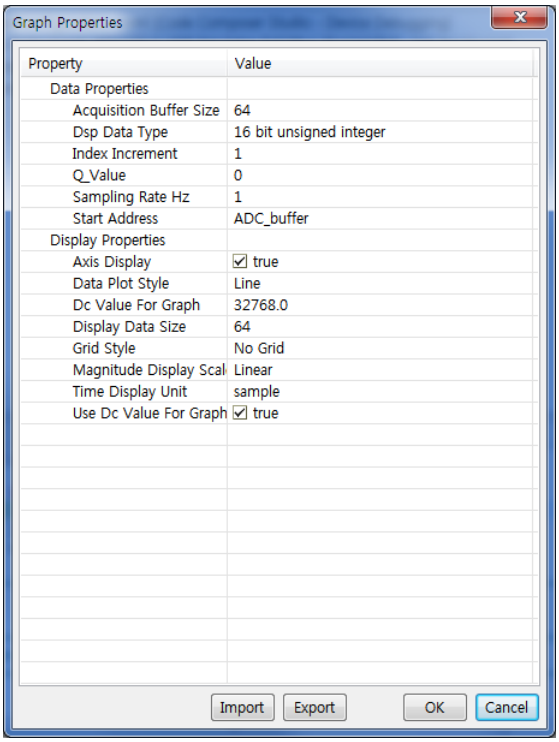
CCS에서 예제를 실행하고, CCS의 Graph 기능을 통해서 ADC 결과를 시각적으로 관찰할 수 있습니다

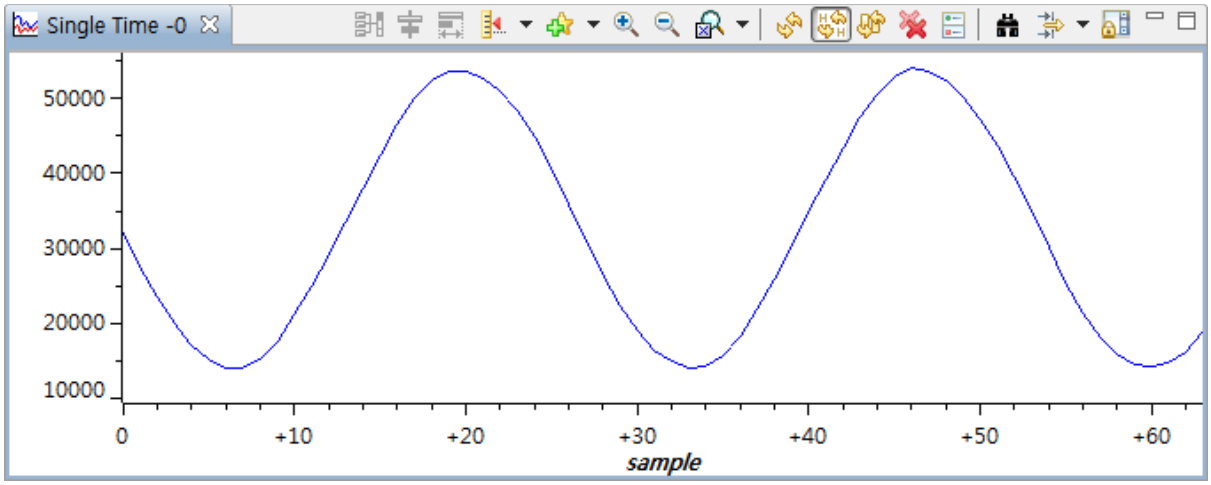
다음은 CCSv3.3의 그래프 관찰 예 입니다.





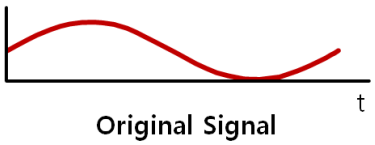
다음은 CCSv5 의 그래프 관찰 예 입니다.

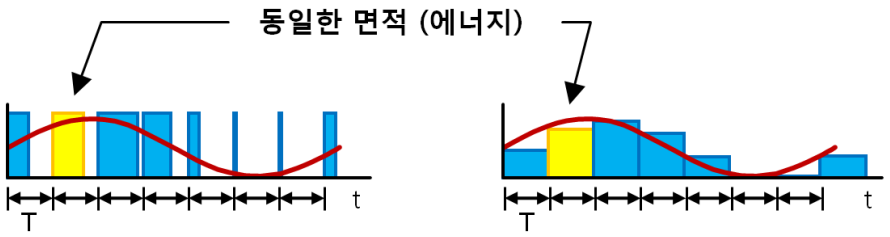




4 EPWM

PWM 이라는 것은 Pulse Width Modulation의 약자 입니다. 무언가 펄스 폭으로 변조한다는 것 입니다. 즉, PWM 신호는 신호의 정보를 펄스의 폭으로 나타낸다는 의미 입니다. 강한 신호는 넓은 펄스 폭으로 약한 신호는 좁은 펄스 폭으로 나타내게 됩니다. 아래 그림을 보면 PWM 이 어떠한 방식으로 원본 신호를 표현하는지 PAM과 비교해서 좀더 쉽게 이해 할 수 있습니다.





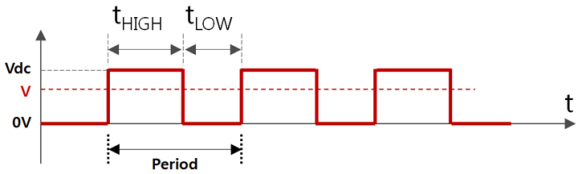
PWM (1/T = Carrier Freq.)

PWM은?
- 디지털 시스템에서 구현 용이
- "0" 또는 "1"의 이진(Binary) 신호
- 별도의 신호 가공용 회로 없이 MCU의 디지털 출력으로 아날로그 시스템을 직접 제어
- MCU부터 제어 시스템에 이르는 경로가 모두 디지털
  - ✓ 노이즈 유입에 대한 영향이 적음
- 외부 TR 구동 시, TR을 스위치 모드로 이용
  - ✓ On 저항 값 최소영역 → 전력손실 최소화

PAM (1/T = Sampling Freq.)

PAM은?
- MCU와 같은 디지털 시스템에서 직접 생성 불가
  - ✓ 아날로그 시스템으로 구현
- MCU의 디지털 출력을 아날로그로 변환하는 부가회로 필요
  - ✓ 제어를 위해 회로에서 사용되는 전류가 발열로 나타남
  - ✓ 경우에 따라 부가적인 방열대책 필요
  - ✓ 시스템의 크기, 비용 증대 / 효율 감소
- 시간에 대해 연속적인 아날로그 신호 → 노이즈 유입에 취약

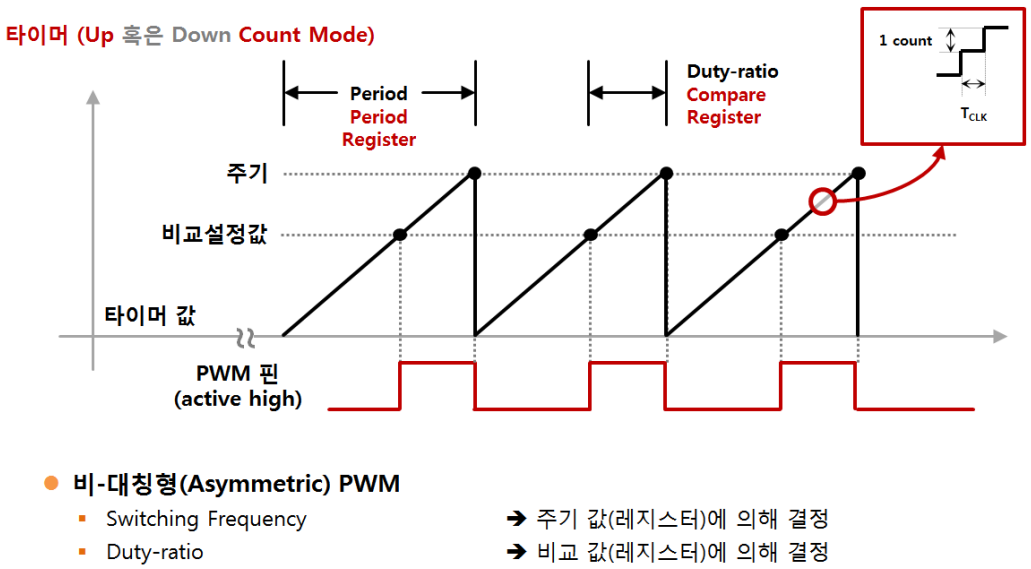
좀더 명확하게 PWM 신호를 설명하면 다음 그림과 같습니다. 즉, t_{HIGH}와 t_{LOW} 주기의 비율이 신호의 크기(평균전압) V를 결정하고 있습니다.



$$V = \frac{t_{HIGH}}{t_{HIGH} + t_{LOW}} \cdot Vdc$$

디지털 PWM 생성방법1 (Asymmetric PWM)

다음 그림은 PWM 생성 원리는 보여주는 그림입니다. 디지털 시스템의 PWM 생성에는 기본적으로 타이머를 사용하게 됩니다. 타이머의 주기값과 비교설정값을 설정해서 PWM 신호를 생성하게 됩니다.



- **비-대칭형(Asymmetric) PWM**
  - Switching Frequency ➔ 주기 값(레지스터)에 의해 결정
  - Duty-ratio ➔ 비교 값(레지스터)에 의해 결정

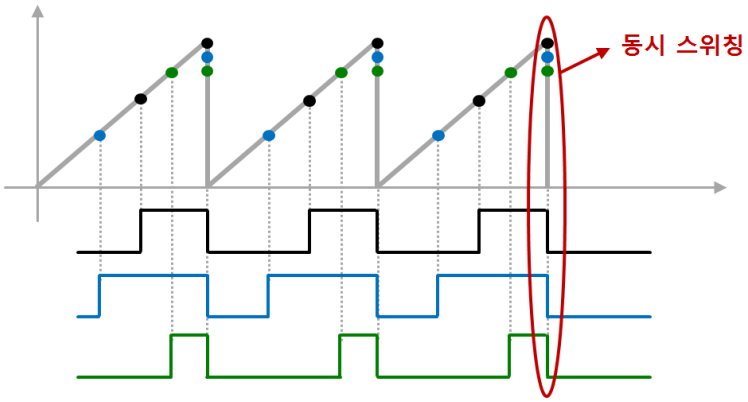
예를 들어서 16비트 타이머가 있다고 가정해보겠습니다. 이 16비트 타이머에서 설정할 수 있는 최대 주기값은 65535입니다. 즉, 0~65535 사이의 아무 값이 나 주기값으로 설정할 수 있다는 얘기 입니다. 그리고 이 타이머는 설정한 주기 값에 도달하면, 다시 0부터 카운팅 된다고 가정하겠습니다. 그럼 여기서 주기값을 9999로 설정하고, 타이머에 10MHz의 클럭이 공급된다면, 타이머의 카운터가 0부터 9999까지 증가하는데 걸리는 시간은 1ms 입니다. 주파수로 표현하면 1KHz입니다. 이것이 PWM신호의 캐리어 주파수가 됩니다.

그러면, PWM 신호의 신호정보를 표현하는 펄스 폭은 어떻게 생성될까요? 이때 필요한 것이 비교설정값입니다. 아까와 같은 조건에서 비교설정값을 499로 설정한다고 가정하고, 타이머 카운터 값이 비교설정값 보다 작을 때는 PWM 핀이 Low를 유지하고, 비교설정값 보다 클 때는 High를 유지한다면, 실제 PWM 핀에서 출력되는 신호는 1KHz의 캐리어 주파수에 5%의 듀티를 가지는 PWM 신호가 됩니다.

비-대칭형 PWM의 단점

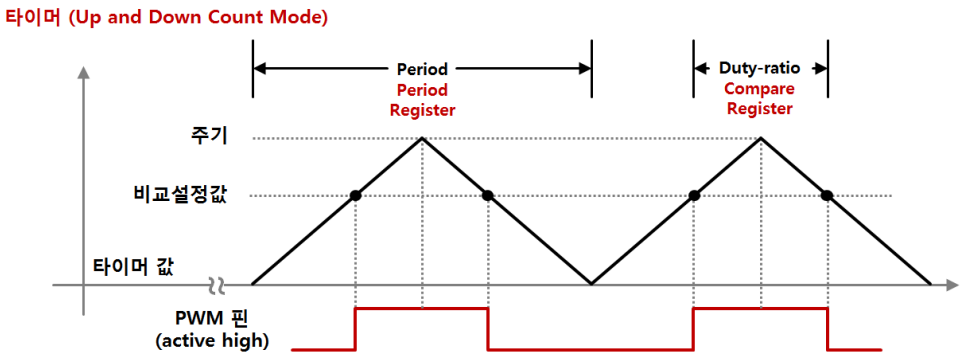
앞서 설명했던 PWM 생성 방법은 비 대칭형 PWM을 생성하는 방법입니다. 이 비 대칭형 PWM은 아래 그림과 같은 단점을 가지고 있습니다. 비 대칭형 PWM의 단점으로 인해 문제가 생긴다면 대칭형 PWM 사용을 고려할 수 있습니다.

- **디지털, 아날로그 복합 시스템에서 다-상으로 외부 스위치 조작 시...**
  - 동일 시점에서 PWM이 제어하는 외부 스위치 모두가 동시에 Turn ON or OFF 동작
  - 아날로그 회로에 대한 스위칭 노이즈 영향 증대



디지털 PWM 생성방법1 (Symmetric PWM)

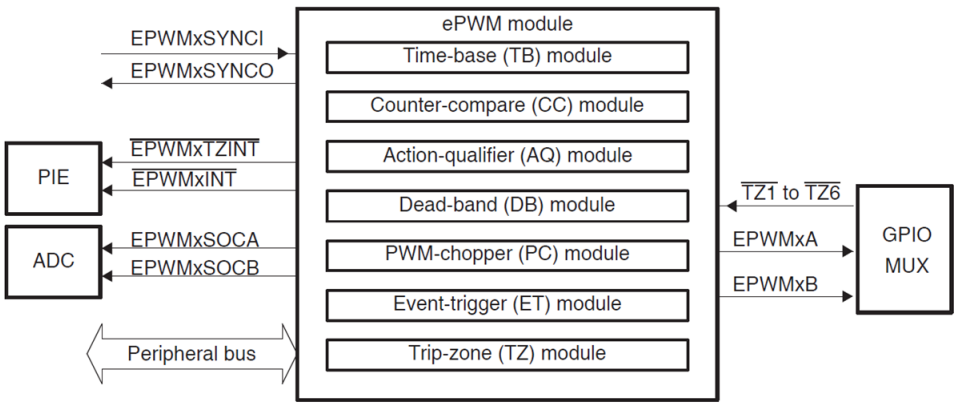
대칭형 PWM은 아래 그림처럼 타이머 카운터가 대칭형으로 증감하기 때문에 여러 채널을 사용하더라도 비 대칭형 PWM과 같은 동시 스위칭 문제가 없습니다.



- **대칭형(Symmetric) PWM**
  - 다-상 PWM으로 활용 시, 스위칭 시간 분산으로 파워 시스템 운영이 안정적
  - 아날로그 회로(예, ADC)에 스위칭 노이즈 최소화 가능

Delfino 프로세서의 EPWM 특징

Delfino 프로세서는 PWM은 EPWM 라고 표시되어 있는데, 이는 Enhanced PWM라는 의미로 기존 PWM 보다 진보된, 향상된 PWM 라고 보시면 됩니다

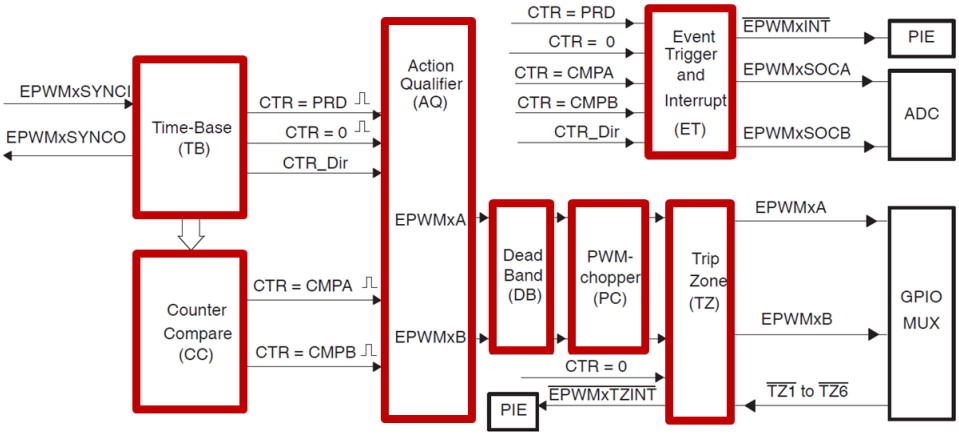


- 1개의 16-Bit 타이머 기반
  - 모듈 당 2개의 PWM 핀 보유 (EPWMxA, EPWMxB)
- 독립적인 Duty-ratio 제어 가능 (비교 레지스터 2개)
- 모든 이벤트들을 인터럽트로 활용 가능
- 모든 이벤트들을 ADC의 SOC로 연결 가능
- 이벤트 Prescale 기능 제공
- EPWM 모듈 간 동기(Sync.) 가능
- EPWM 모듈 간 위상-차 제어 가능
  - H/W(레지스터)로 Phase Interleaved PWM 구현
- Dead-Band Unit 보유
  - Rising Edge와 Falling Edge의 지연을 독립적으로 조절 가능
- Trip-Zone Unit 보유
  - 모듈 별로 Trip 핀 선택 가능
  - Trip 조건 발생 시, PWM 핀 상태를 High / Low / Hi-z 중 택일
- Pulse Transformer-based Gate 구동용 Chopper Unit 보유

Delfino 프로세서의 EPWM 구조

Delfino 프로세서의 PWM 모듈은 아래 그림과 같이 총 7개의 서브 모듈로 구성되어 있습니다. PWM 신호는 이러한 서브모듈들 통해서 생성되게 됩니다.

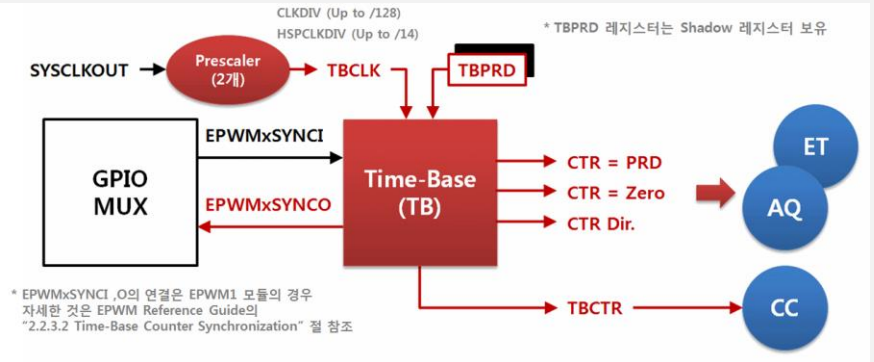
- **총 7개의 서브 모듈들로 하나의 EPWM 모듈이 구성됨**
  - TB ➔ CC ➔ AQ ➔ DB ➔ PC ➔ TZ ➔ **EPWMxA, B Out**
  - All Events ➔ ET



◈ **Time-Base(TB) 서브 모듈**

PWM Carrier 신호 생성을 위한 16-Bit 타이머

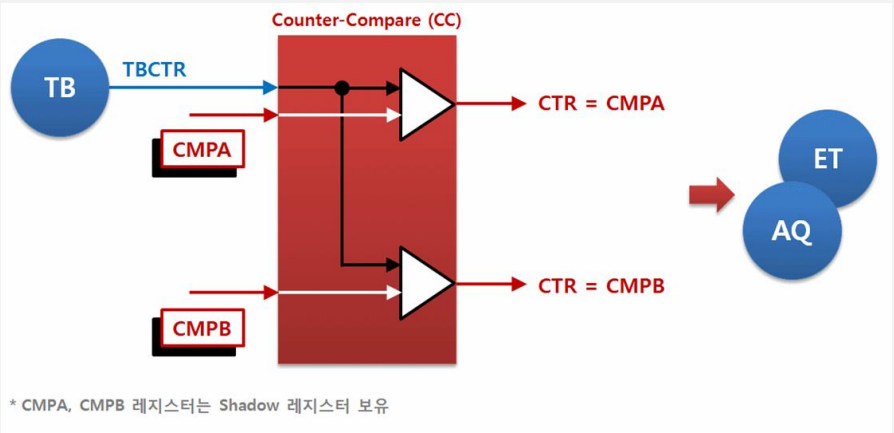
- 시스템 클럭을 분주한 TBCLK 기반으로 동작 (최대 /1792 분주 가능)
- PWM 생성 외 범용으로도 활용 가능 (주기적인 인터럽트 발생원)
- Counter-Compare(CC) 서브모듈에 16-Bit 타이머 값 전달
- Action-Qualifier(AQ), Event-Trigger(ET) 서브모듈에 타이머의 계수방향(Dir.)과
- CTR=PRD / CTR=Zero 이벤트 전달



◈ Counter-Compare(CC) 서브 모듈

PWM의 Duty-ratio를 결정짓는 비교 레지스터와 컨트롤 레지스터

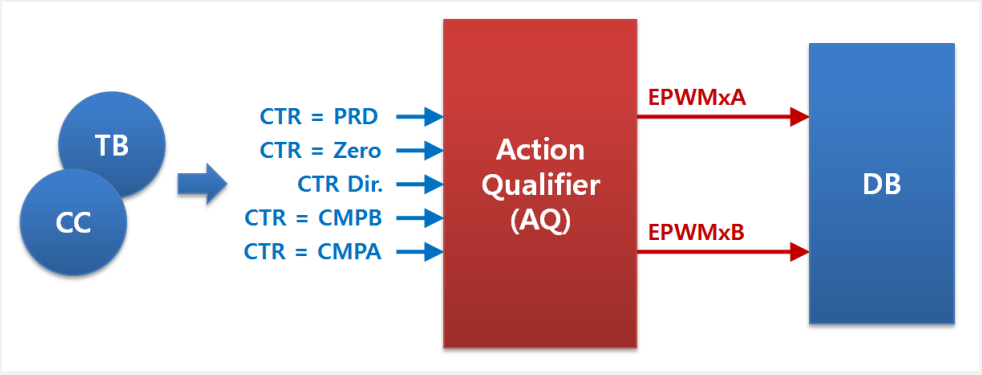
- Time-Base 서브모듈로부터 16-Bit 타이머 값을 전달받음
- 설정된 비교 레지스터와 타이머 값을 비교하여, CTR=CMPA or CMPB 이벤트 생성
- 이벤트를 Action-Qualifier(AQ), Event-Trigger(ET) 서브모듈에 전달



◈ Action-Qualifier(AQ) 서브 모듈

TB, CC 서브모듈의 이벤트들을 입력 받아, 해당 이벤트들에서의 PWM 출력 로직상태를 규정

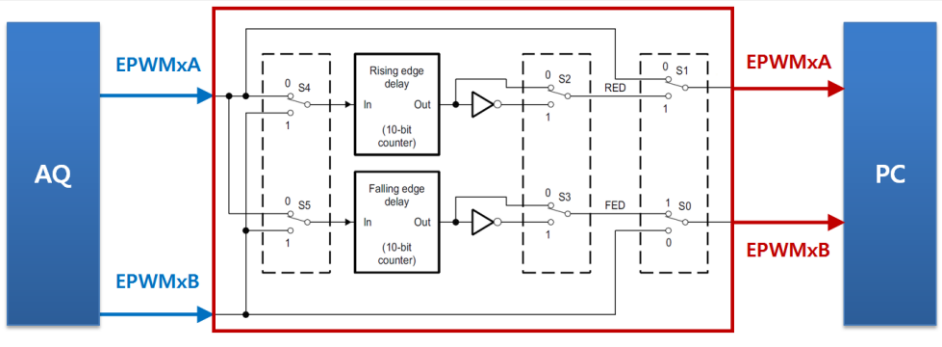
- AQ 서브모듈로 입력되는 이벤트들
  - ✓ CTR=PRD / CTR=Zero / CTR=CMPA on Up-Count / CTR=CMPA on Down-Count
  - ✓ CTR=CMPB on Up-Count / CTR=CMPB on Down-Count
- Set / Clear / Toggle / Do Nothing 4가지의 동작 규정 가능
- S/W Forcing 기능 지원



◈ Dead-Band(DB) 서브 모듈

AQ 모듈이 규정한 PWM에 Dead-Time을 추가

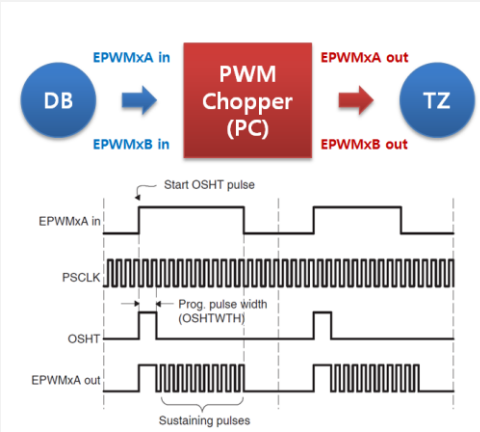
- AQ 서브모듈로 입력되는 이벤트들
- 스위칭 소자의 On/Off 시간차로 발생하는 소손 방지
- TBCLK를 기반으로 동작하는 10-Bit 타이머 2개 보유
- 각 타이머로 Rising Edge와 Falling Edge에 독립적인 지연 추가
- 입력소스 선택 / 신호 반전 가능 / Bypass 제공



◈ PWM-Chopper(PC) 서브 모듈

DB 모듈로부터 입력된 EPWMxA, B 신호를 고-주파수 Carrier 신호로 재-가공(Chopping)한 후, TZ 서브모듈에 전달

- Pulse Transformer-based Gate Driver 구동용
- Chopper 출력 신호의 첫 번째 펄스-폭은 별도로 임의 조정 가능 (16가지)
- 두 번째와 그 이후, Chopper 신호들의 주파수 및 펄스-폭(Duty) 조절 가능
- Bypass 가능

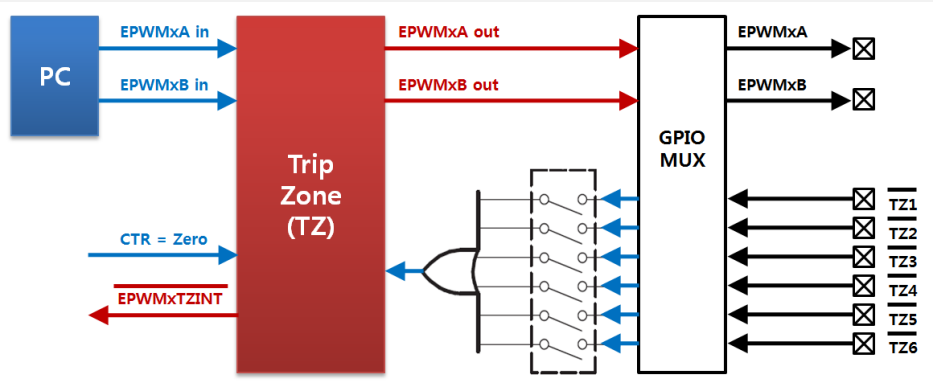


OSHTWTH (hex)	Pulse width (nS)
0	80
1	160
2	240
3	320
4	400
5	480
6	460
7	640
8	720
9	800
A	880
B	960
C	1040
D	1120
E	1200
F	1280

◈ Trip-Zone(TZ) 서브 모듈

Trip 핀에 Active 신호(Low) 입력 시, 연결된 PWM 핀 출력을 미리 설정된 상태로 전환

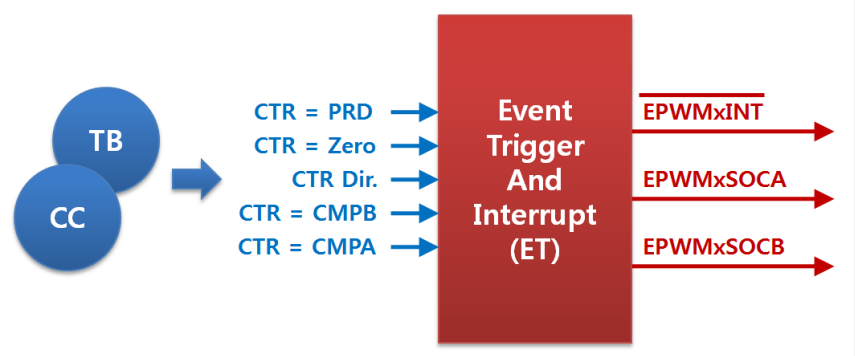
- 오류나 위험 상황에 대한 PWM 로직 상태 결정
- One-Shot / Continuous 동작 지원
  - ✓ One-Shot Trip → Short circuit or Over-current 검출용
    (Trip 발생 후, 사용자의 S/W적 Clear 전까지 Trip 상태 유지)
  - ✓ Cycle-by-Cycle(Continuous) Trip → Current limiting 지원용
    (CTR=Zero 이벤트에서 자동으로 Trip 상태 해제)
- Trip 상황에서의 PWM 핀 상태를 High / Low / Hi-z / No Action 의 4가지로 규정
- 인터럽트 지원, S/W Tripping, Bypass 지원



◈ Event-Trigger(ET) 서브 모듈

TB, CC 모듈이 생성하는 이벤트들을 입력 받아, 인터럽트 혹은 ADC 변환시작(SOC) 신호를 생성

- 이벤트 Prescale 가능 (Every / Every 2nd / Every 3rd)
- 인터럽트나 ADC SOC의 S/W Trigger 지원



4.1 PWM 1채널 출력하기 (Asymmetric, Single Edge)

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 1 채널의 비-대칭형 PWM 파형을 출력하고, 이를 계측장비로 확인합니다. (Asymmetric Single Edge Control)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM01_1Ch_PWM_Asymmetric_Single_Edge_Control
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM01_1Ch_PWM_Asymmetric_Single_Edge_Control
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM01_1Ch_PWM_Asymmetric_Single_Edge_Control

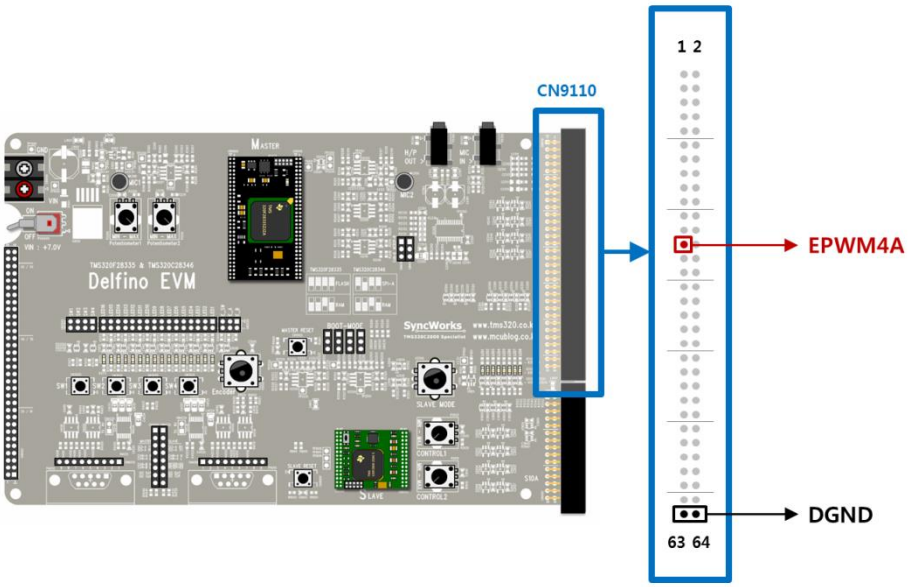
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

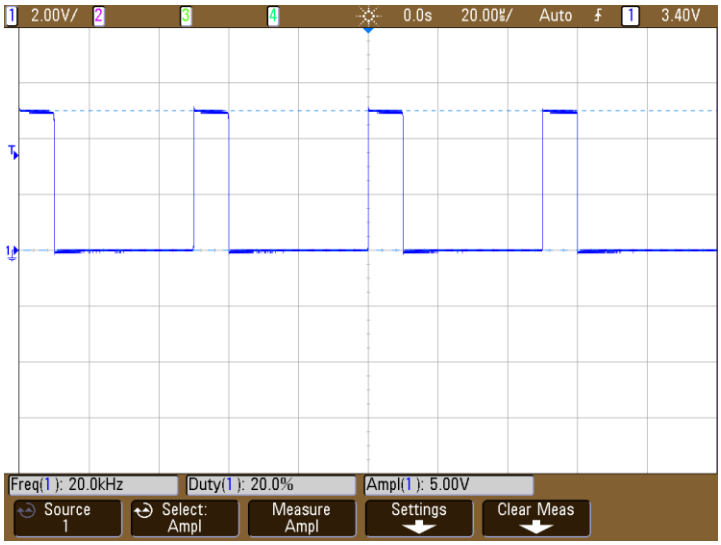
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-01] 1채널, PWM 출력하기 (Asymmetric, Single Edge)" 폴더

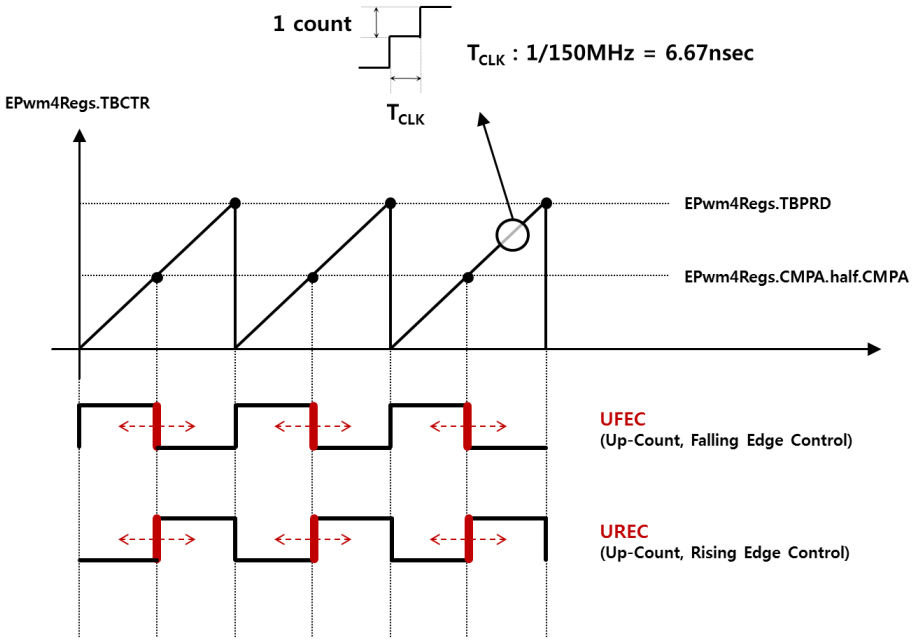
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 1채널 PWM 출력하기 (Asymmetric, Single Edge)" 게시물

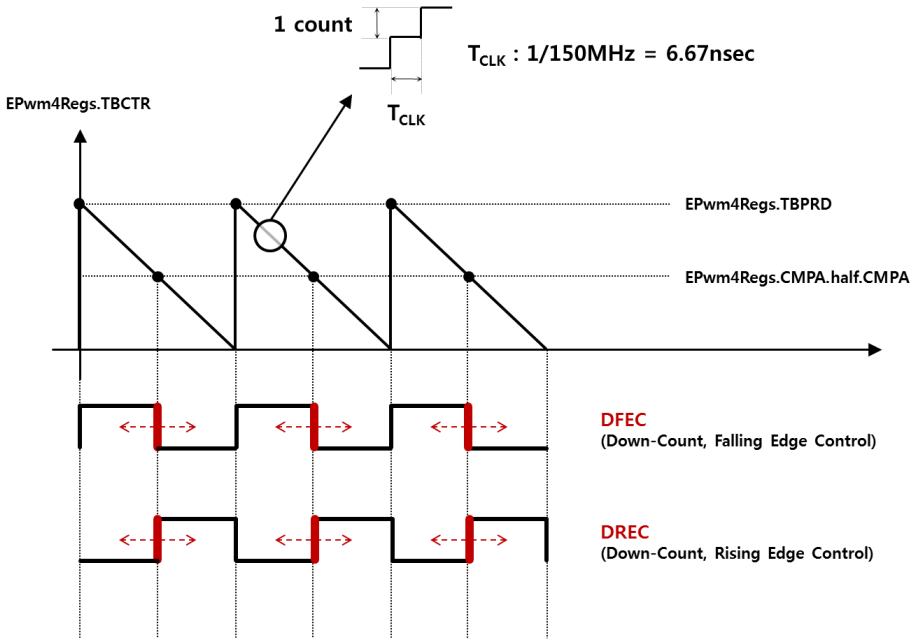
실습해 보기

Code Composer Studio에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25번 핀을 계측장비(Oscilloscope)로 관찰하면, PWM 출력 파형을 관찰하실 수 있습니다.






1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPA.half.CMPA
UFEC
(Up-Count, Falling Edge Control)
UREC
(Up-Count, Rising Edge Control)


1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPA.half.CMPA
DFEC
(Down-Count, Falling Edge Control)
DREC
(Down-Count, Rising Edge Control)

4.2 PWM 1채널 출력하기 (Symmetric, Dual Edge)

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 1 채널의 대칭형 PWM 파형을 출력하고, 이를 계측장비로 확인합니다. (Symmetric Dual Edge Control)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM02_1Ch_PWM_Symmetric_Dual_Edge_Control
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM02_1Ch_PWM_Symmetric_Dual_Edge_Control
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM02_1Ch_PWM_Symmetric_Dual_Edge_Control

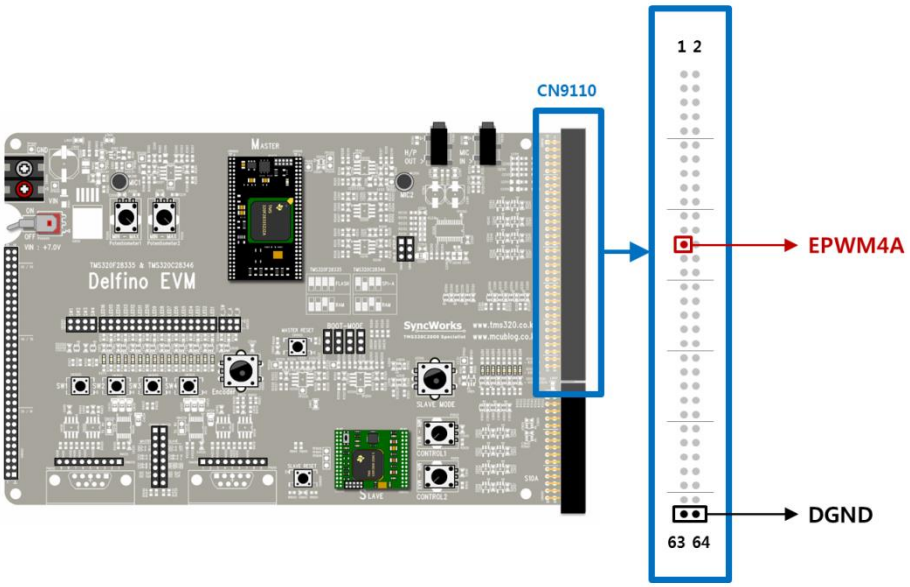
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

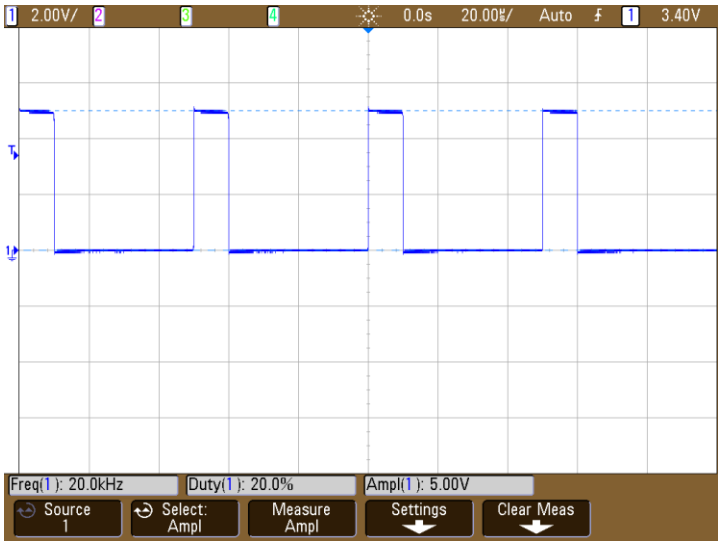
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-02] 1채널, PWM 출력하기 (Symmetric, Single Edge)" 폴더

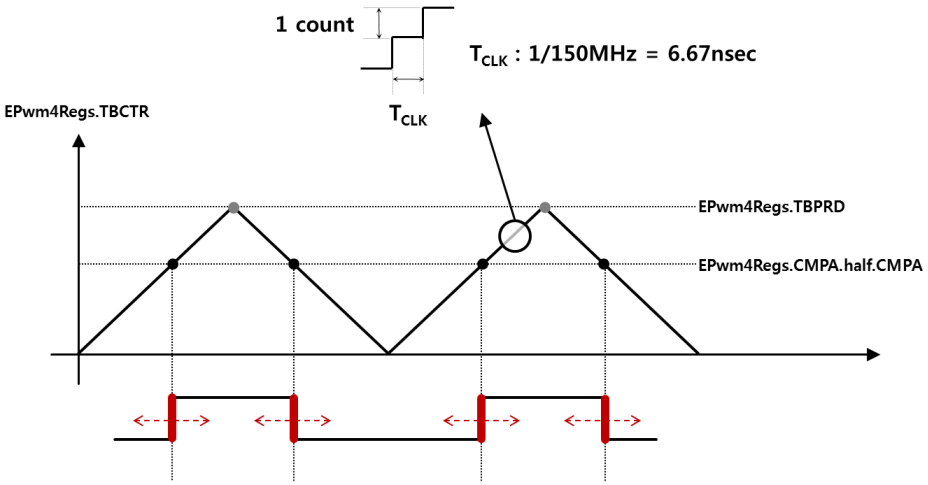
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 1채널 PWM 출력하기 (Symmetric, Single Edge)" 게시물

실습해 보기

Code Composer Studio에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25번 핀을 계측장비(Oscilloscope)로 관찰하면, PWM 출력 파형을 관찰하실 수 있습니다.







Up/Down-Count, Symmetric Dual-Edge Control

4.3 PWM 1채널 출력하기 (Asymmetric, Dual Edge)

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 1 채널의 비-대칭형 PWM 파형을 출력하고, 이를 계측장비로 확인합니다. (Asymmetric Dual Edge Control)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM03_1Ch_PWM_Asymmetric_Dual_Edge_Control
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM03_1Ch_PWM_Asymmetric_Dual_Edge_Control
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM03_1Ch_PWM_Asymmetric_Dual_Edge_Control

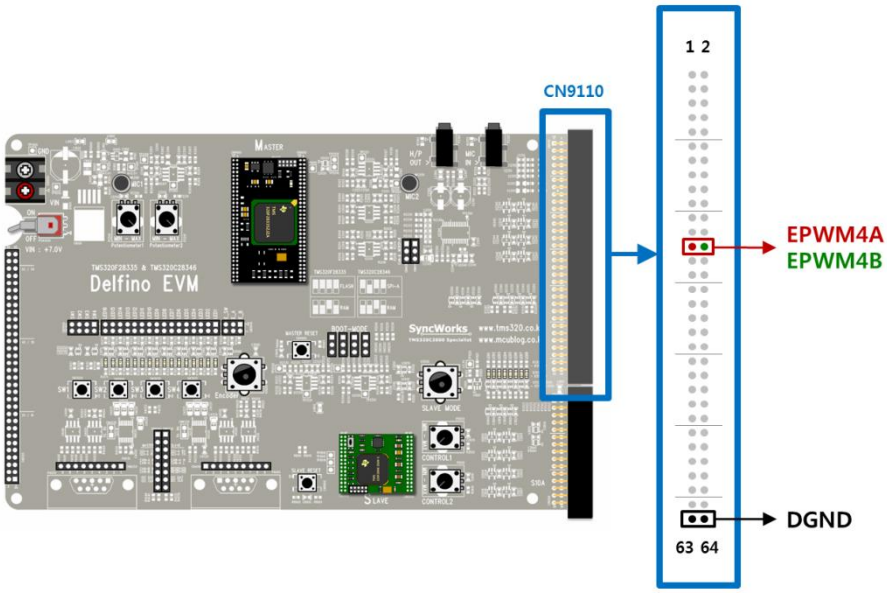
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

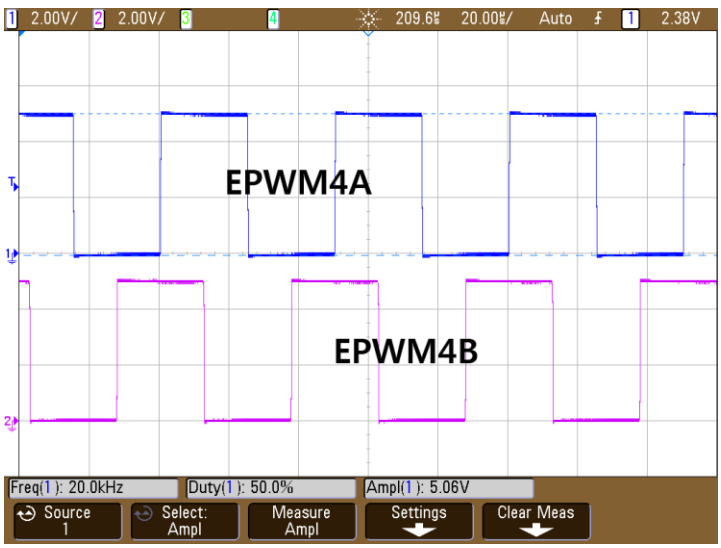
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-03] 1채널, PWM 출력하기 (Asymmetric, Dual Edge)" 폴더

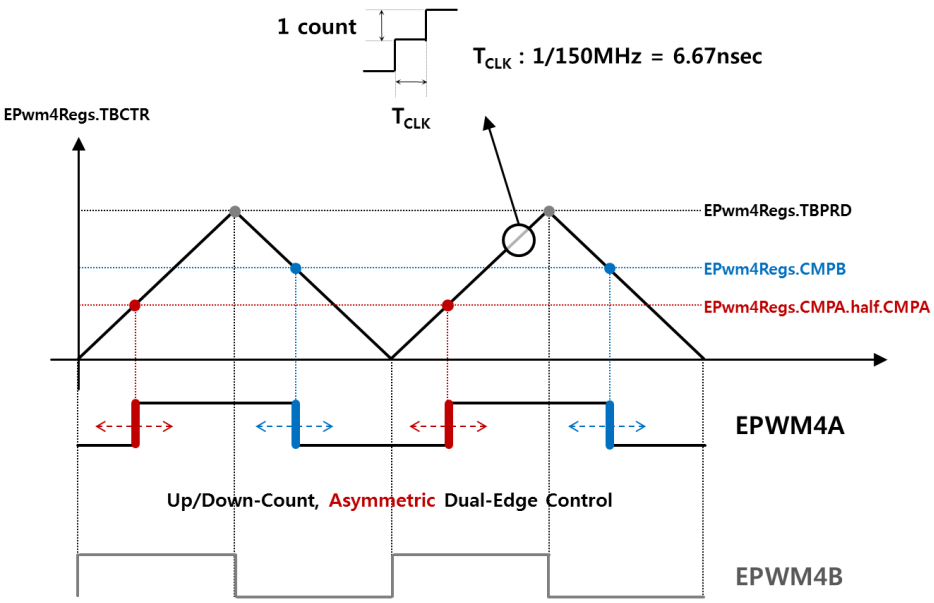
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 1채널 PWM 출력하기 (Asymmetric, Dual Edge)" 게시물

실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B) 핀을 계측장비(Oscilloscope)로 관찰하면, PWM 출력 파형을 관찰하실 수 있습니다.






1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPB
EPwm4Regs.CMPA.half.CMPA
EPWM4A
Up/Down-Count, Asymmetric Dual-Edge Control
EPWM4B

4.4 PWM 주기적인 인터럽트 생성과 이벤트 분주

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 주기적인 인터럽트를 발생시켜보고, Enhanced PWM Module 의 인터럽트 소스들과 이벤트 분주(Prescale) 기능에 대해 이해합니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM04_1Ch_PWM_Periodic_Interrupts
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM04_1Ch_PWM_Periodic_Interrupts
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM04_1Ch_PWM_Periodic_Interrupts

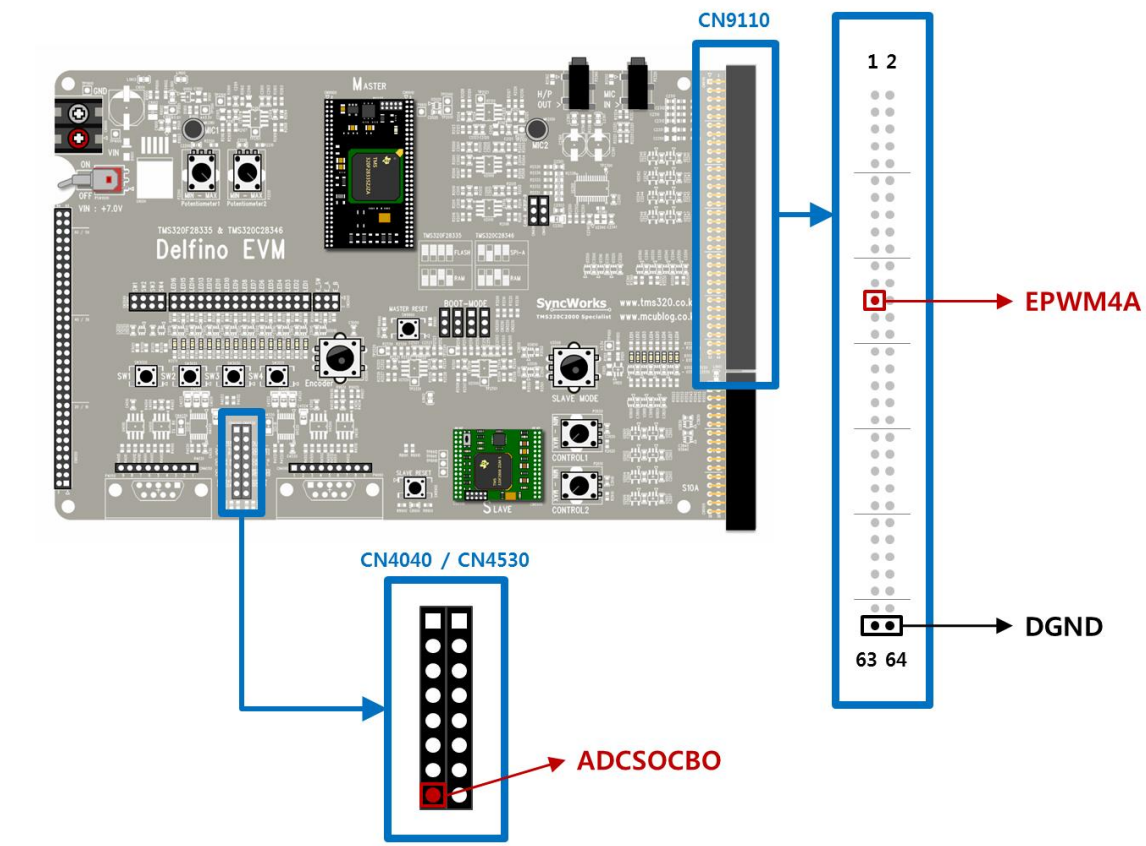
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

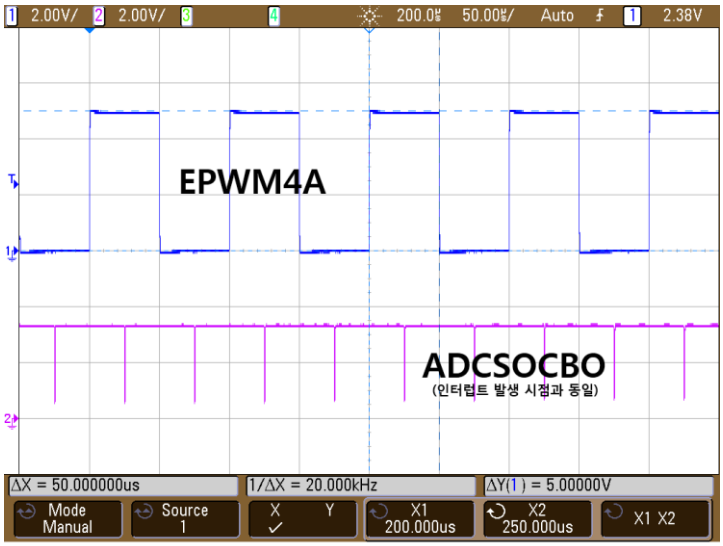
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-04] 주기적인 인터럽트 생성과 이벤트 분주" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 주기적인 인터럽트 생성과 이벤트 분주" 게시물

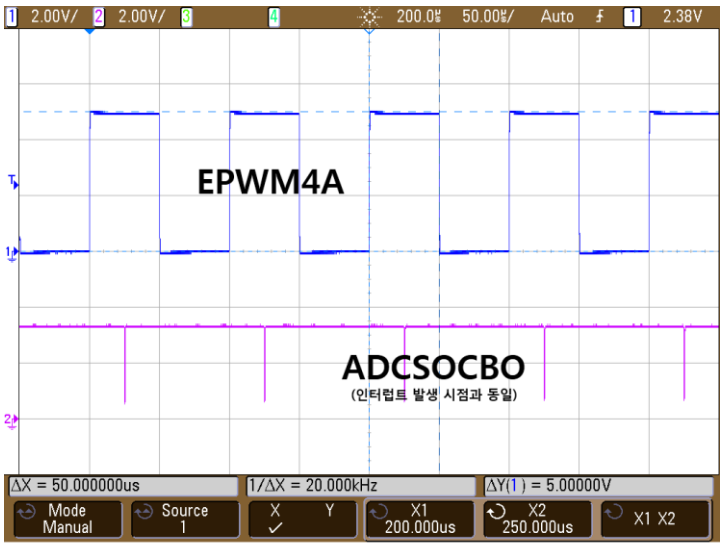
실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A) 핀과 CN4040 핀-헤더의 8 번(ADCSOCBO, 인터럽트 발생 시점 확인용 출력) 핀을 계측장비(Oscilloscope)로 관찰하면, PWM 주파수와 동일한 주기로 반전(Toggle)되는 파형과, 인터럽트 발생 시점과 일치하는 SOC 출력 파형을 관찰할 수 있습니다.

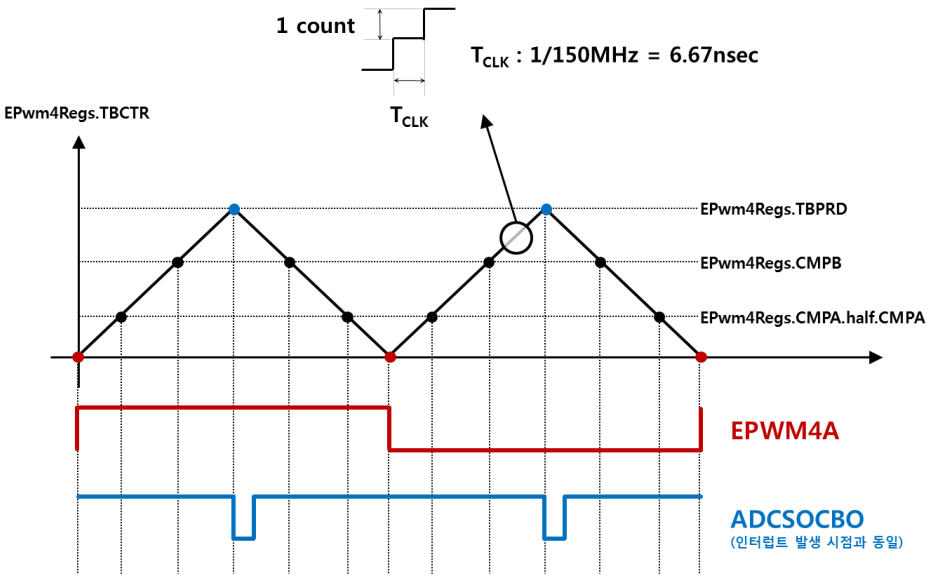




EPwm4Regs.ETSEL.bit.INTSEL = 2
--> 타이머 카운터가 주기값과 일치하는 이벤트를 인터럽트 소스로 사용
EPwm4Regs.ETPS.bit.INTPRD = 1
--> 매 이벤트(1st) 마다, 인터럽트 요청



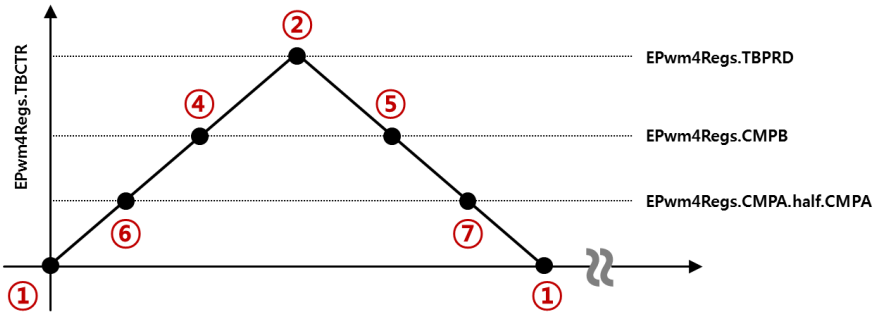
EPwm4Regs.ETSEL.bit.INTSEL = 2
--> 타이머 카운터가 주기값과 일치하는 이벤트를 인터럽트 소스로 사용
EPwm4Regs.ETPS.bit.INTPRD = 2
--> 매 두번째(2nd) 이벤트 마다, 인터럽트 요청



EPwm4Regs.ETSEL.bit.INTSEL = 2
EPwm4Regs.ETPS.bit.INTPRD = 1] 설정의 예

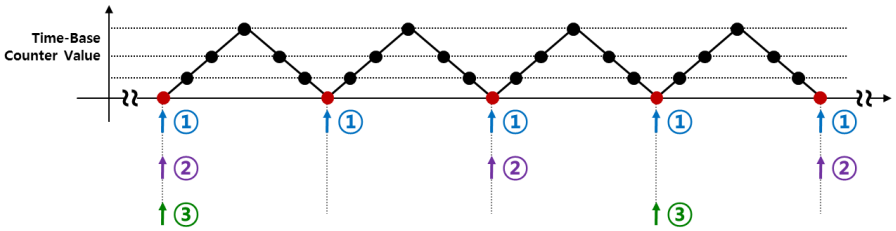
EPwm4Regs.ETSEL.bit.INTSEL 레지스터 값에 따른 인터럽트 소스 선택

INTSEL = 1 → 타이머 카운트 값이 0(Zero)과 일치할 경우 ①
INTSEL = 2 → 타이머 카운트 값이 주기 값(Period)과 일치할 경우 ②
INTSEL = 3 → Reserved
INTSEL = 4 → Up-Count 도중 타이머 카운트 값이 CMPA 값과 일치할 경우 ④
INTSEL = 5 → Down-Count 도중 타이머 카운트 값이 CMPA 값과 일치할 경우 ⑤
INTSEL = 6 → Up-Count 도중 타이머 카운트 값이 CMPB 값과 일치할 경우 ⑥
INTSEL = 7 → Down-Count 도중 타이머 카운트 값이 CMPB 값과 일치할 경우 ⑦



EPwm4Regs.ETPS.bit.INTPRD 레지스터 값에 따른 이벤트 Prescale 예
(TBCTR = Zero 이벤트를 인터럽트 소스로 사용하는 경우를 가정한 예)

INTPRD = 1 → 매 이벤트마다 인터럽트 생성 ..... ①
INTPRD = 2 → 매 두 번째 이벤트마다 인터럽트 생성 ..... ②
INTPRD = 3 → 매 세 번째 이벤트마다 인터럽트 생성 ..... ③



4.5 PWM 2채널 출력하기 (Independent Duty Cycle)

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 독립적으로 듀티비율이 제어되는 2 채널의 PWM 을 출력해봅니다. (Asymmetric Single Edge Control)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM05_2Ch_PWM_Independent_Duty_Cycle_Control
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM05_2Ch_PWM_Independent_Duty_Cycle_Control
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM05_2Ch_PWM_Independent_Duty_Cycle_Control

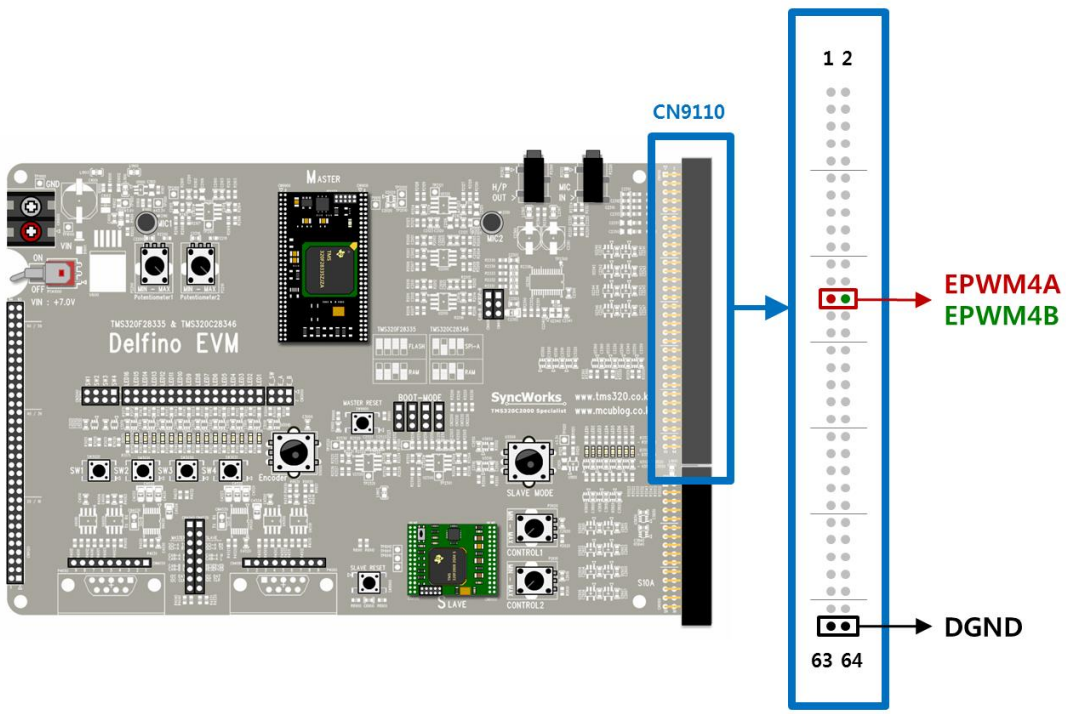
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

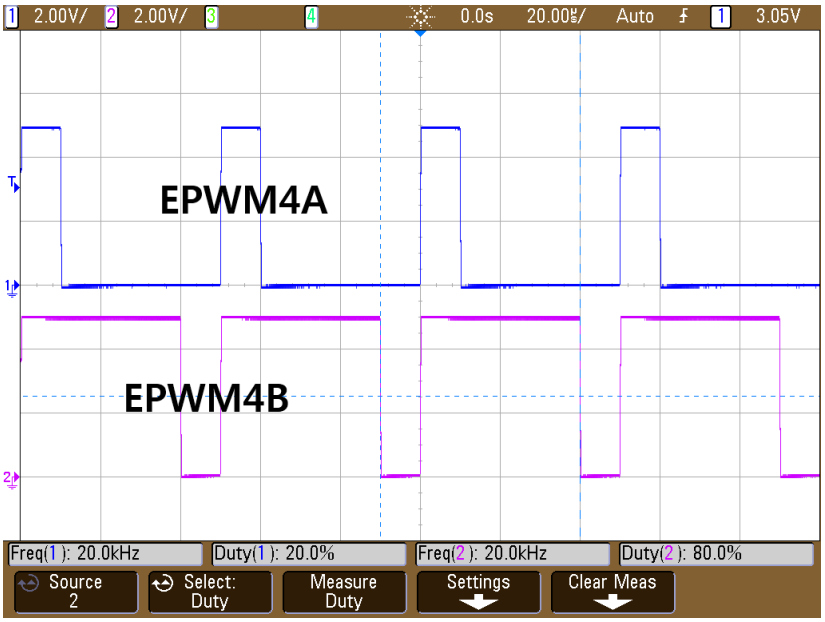
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-05] 2채널, PWM 출력하기 (Independent Duty Cycle)" 폴더

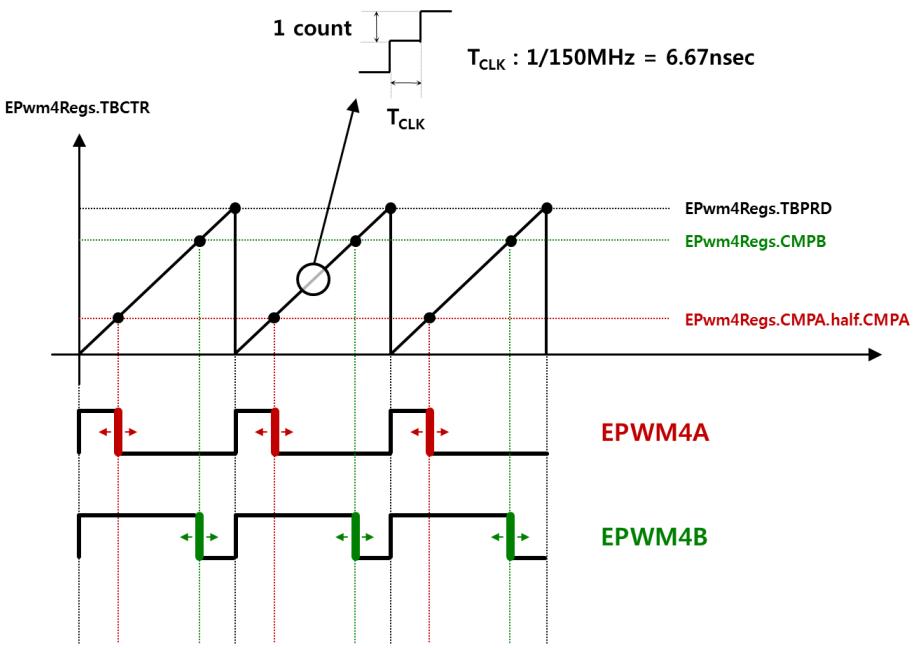
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 2채널 PWM 출력하기 (Independent Duty Cycle)" 게시물

실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B) 핀을 계측장비(Oscilloscope)로 관찰하면, PWM 출력 파형을 관찰하실 수 있습니다.




1 2.00V/ 2 2.00V/ 3 4 0.0s 20.00ns/ Auto 1 3.05V
EPWM4A
EPWM4B
Freq(1): 20.0kHz
Duty(1): 20.0%
Freq(2): 20.0kHz
Duty(2): 80.0%
Source 2
Select: Duty
Measure Duty
Settings
Clear Meas


1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPB
EPwm4Regs.CMPA.half.CMPA
EPWM4A
EPWM4B

4.6 PWM 2채널 출력하기 (Complementary)

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 서로 상보적인(Complementary) 2 채널의 PWM 을 출력해봅니다. (Asymmetric Single Edge Control)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM06_2Ch_PWM_Complementary
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM06_2Ch_PWM_Complementary
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM06_2Ch_PWM_Complementary

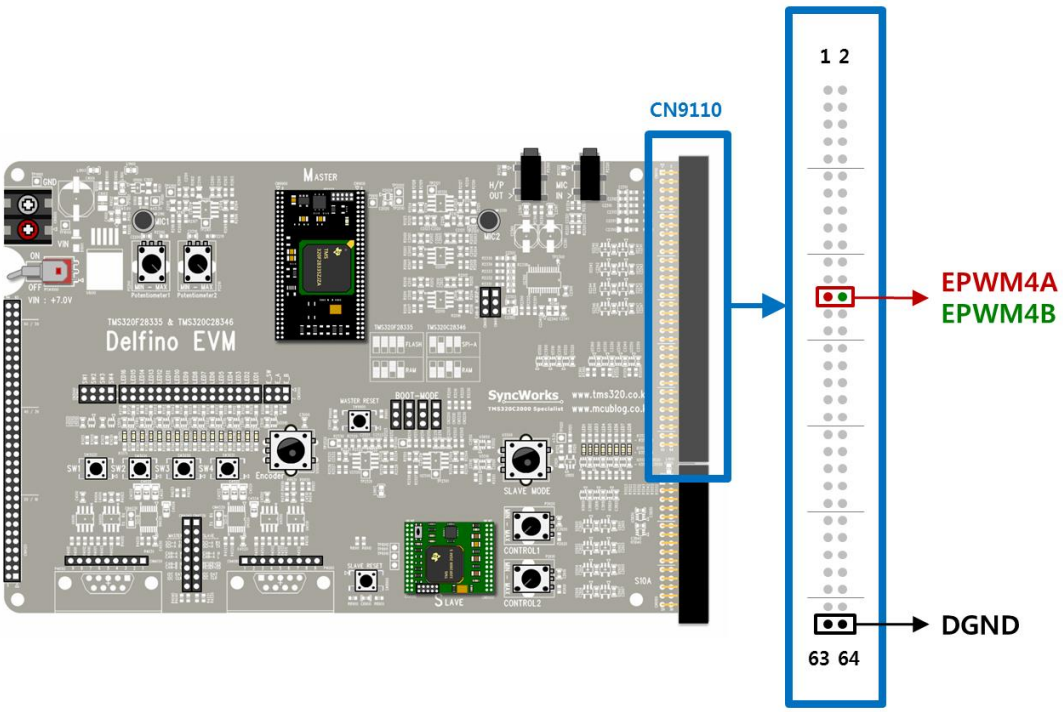
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

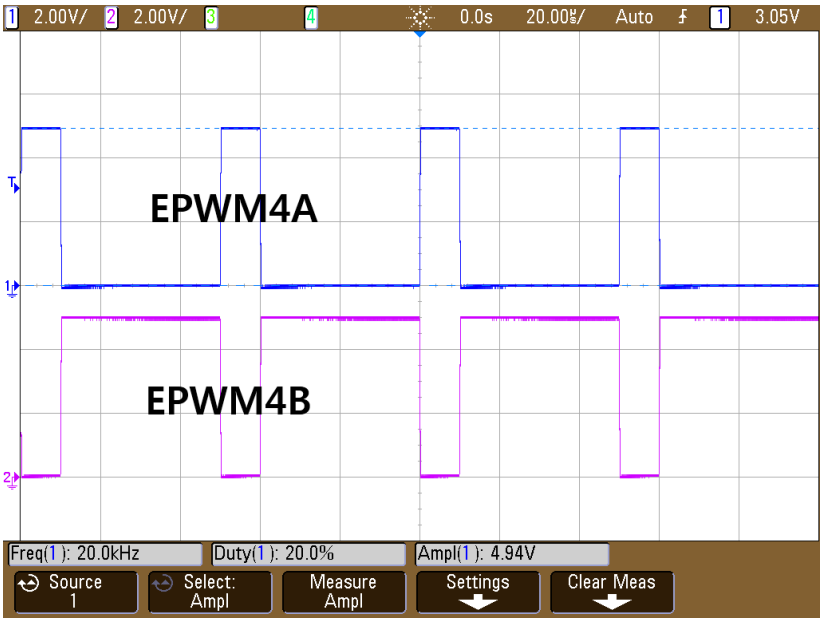
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-06] 2채널, PWM 출력하기 (Complementary)" 폴더

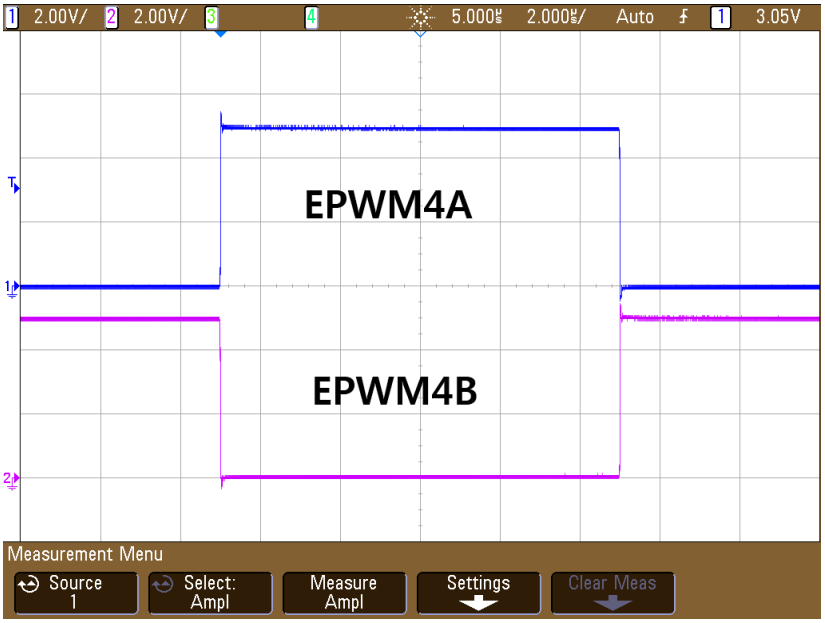
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 2채널 PWM 출력하기 (Complementary)" 게시물

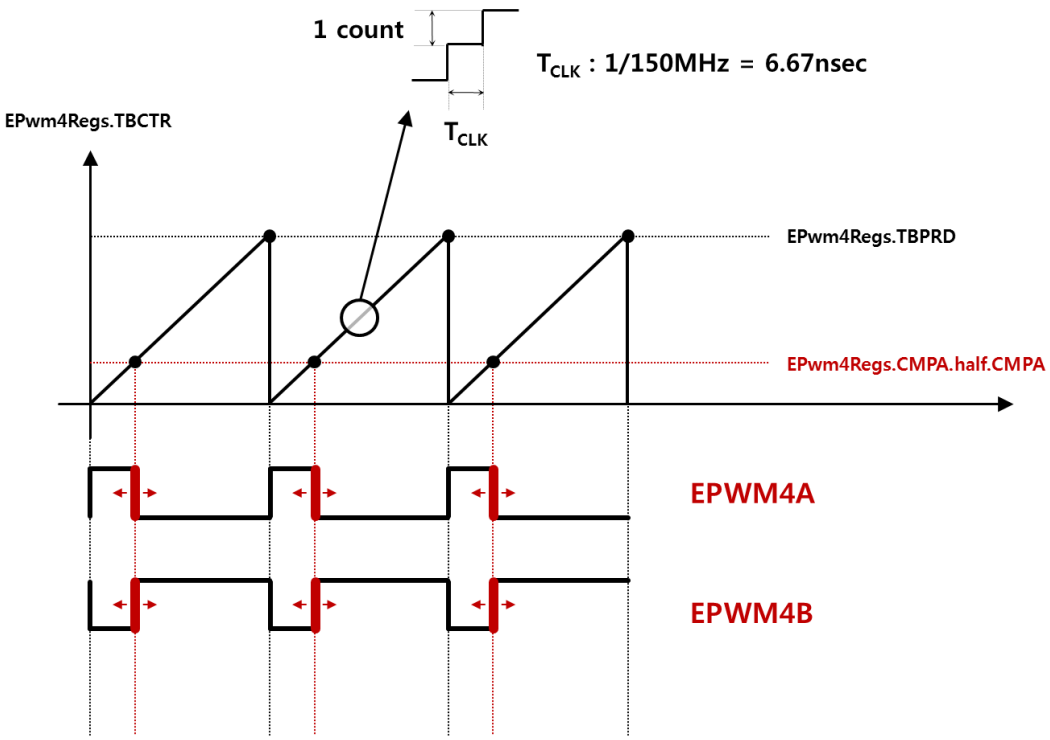
실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B) 핀을 계측장비(Oscilloscope)로 관찰하면, PWM 출력 파형을 관찰하실 수 있습니다.




1 2.00V/ 2 2.00V/ 3 4 0.0s 20.00ns/ Auto 1 3.05V
EPWM4A
EPWM4B
Freq(1): 20.0kHz
Duty(1): 20.0%
Ampl(1): 4.94V
Source 1
Select: Ampl
Measure Ampl
Settings
Clear Meas


1 2.00V/ 2 2.00V/ 3 4 5.000ns 2.000ns/ Auto 1 3.05V
EPWM4A
EPWM4B
Measurement Menu
Source 1
Select: Ampl
Measure Ampl
Settings
Clear Meas


1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPA.half.CMPA
EPWM4A
EPWM4B

4.7 PWM 2채널에 Dead-Band 추가하기

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 서로 상보적인(Complementary) 2 채널의 PWM 을 출력해봅니다. - EPWMxA, EPWMxB 채널 간 Dead-Band 적용 (Asymmetric Single Edge Control)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM07_2Ch_PWM_Dead_Time
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM07_2Ch_PWM_Dead_Time
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM07_2Ch_PWM_Dead_Time

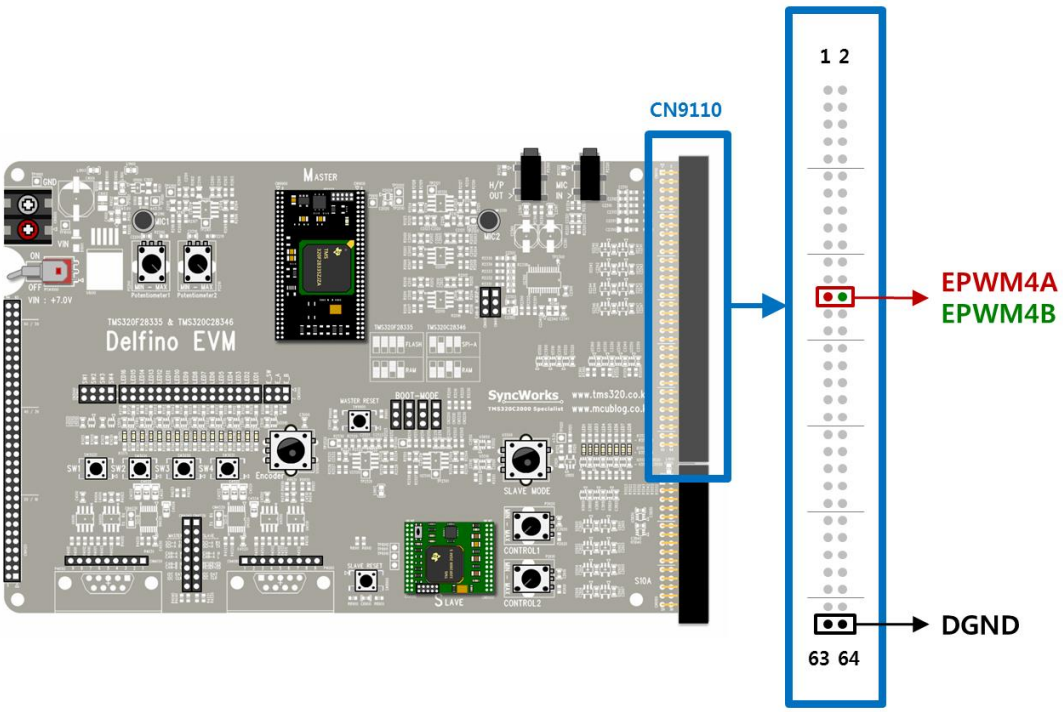
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

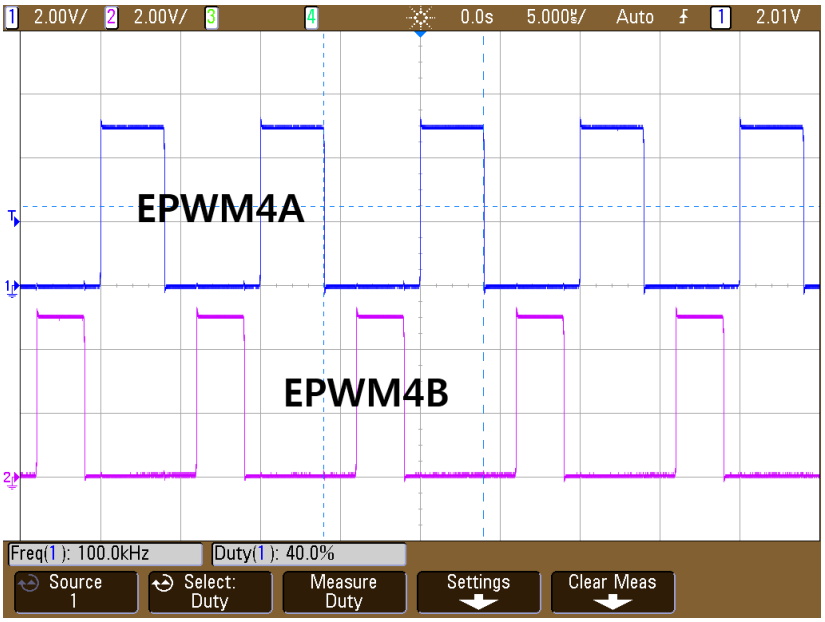
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-07] 2채널, PWM에 Dead-Band 추가하기" 폴더

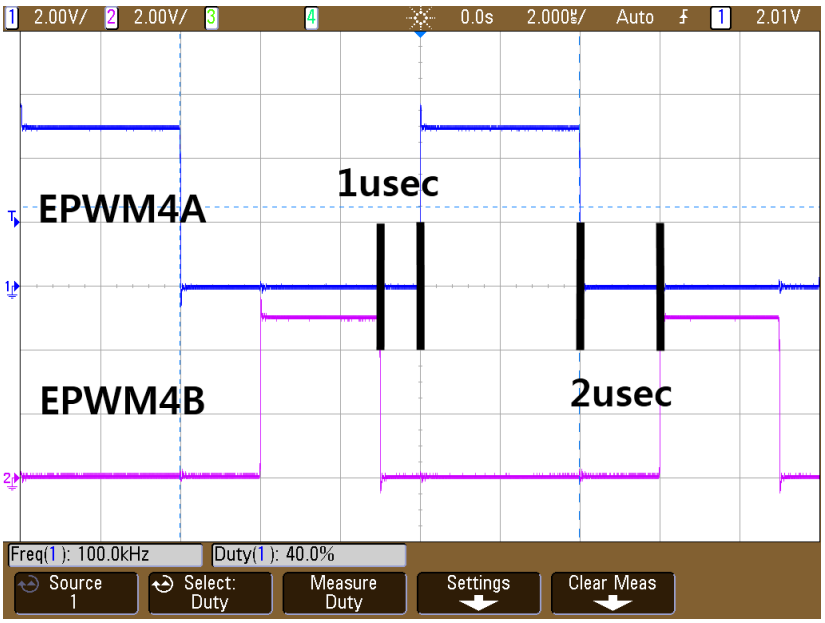
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 2채널 PWM에 Dead-Band 추가하기" 게시물

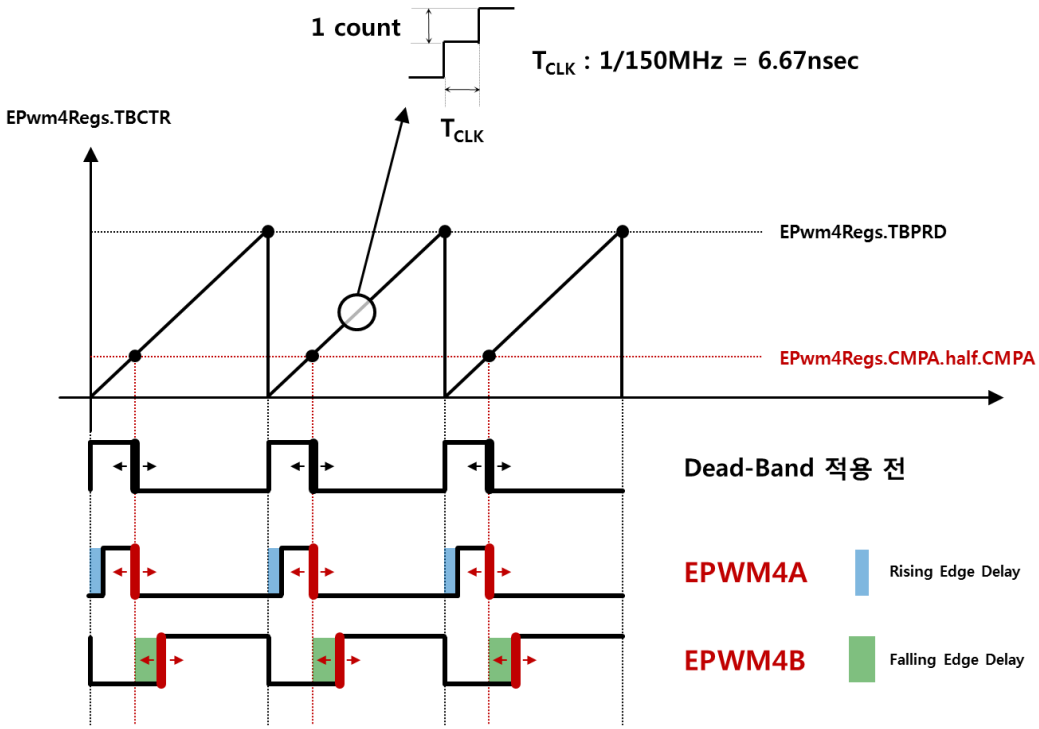
실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B) 핀을 계측장비(Oscilloscope)로 관찰하면, Dead-Band 가 적용된 PWM 출력 파형을 관찰하실 수 있습니다.




1 2.00V/ 2 2.00V/ 3 4 0.0s 5.000µ/ Auto 1 2.01V
EPWM4A
EPWM4B
Freq(1): 100.0kHz
Duty(1): 40.0%
Source 1
Select: Duty
Measure Duty
Settings
Clear Meas


1 2.00V/ 2 2.00V/ 3 4 0.0s 2.000µ/ Auto 1 2.01V
EPWM4A
1usec
EPWM4B
2usec
Freq(1): 100.0kHz
Duty(1): 40.0%
Source 1
Select: Duty
Measure Duty
Settings
Clear Meas


1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPA.half.CMPA
Dead-Band 적용 전
EPWM4A
Rising Edge Delay
EPWM4B
Falling Edge Delay

4.8 PWM 1채널 Chopping

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, Chopping 이 적용된 1 채널 PWM 을 출력해봅니다. (Asymmetric Single Edge Control) Chopping 이 적용된 PWM 은 주로 Pulse Transformer 기반의 Gate Driver 구동용으로 사용됩니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM08_1Ch_PWM_Chopping
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM08_1Ch_PWM_Chopping
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM08_1Ch_PWM_Chopping

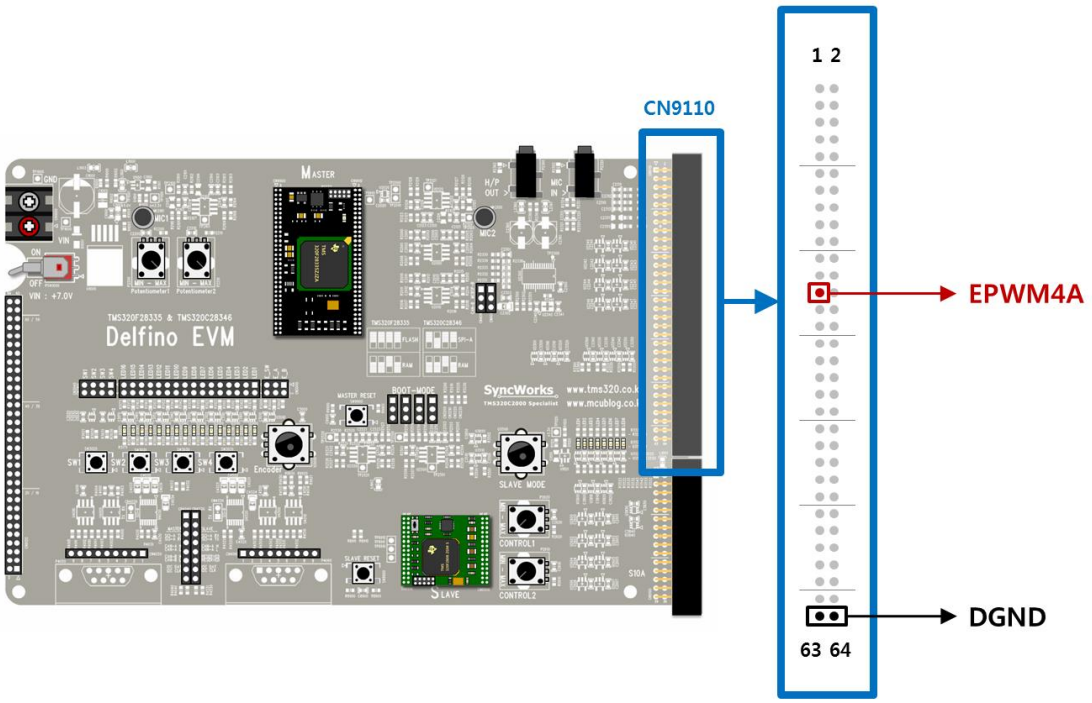
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

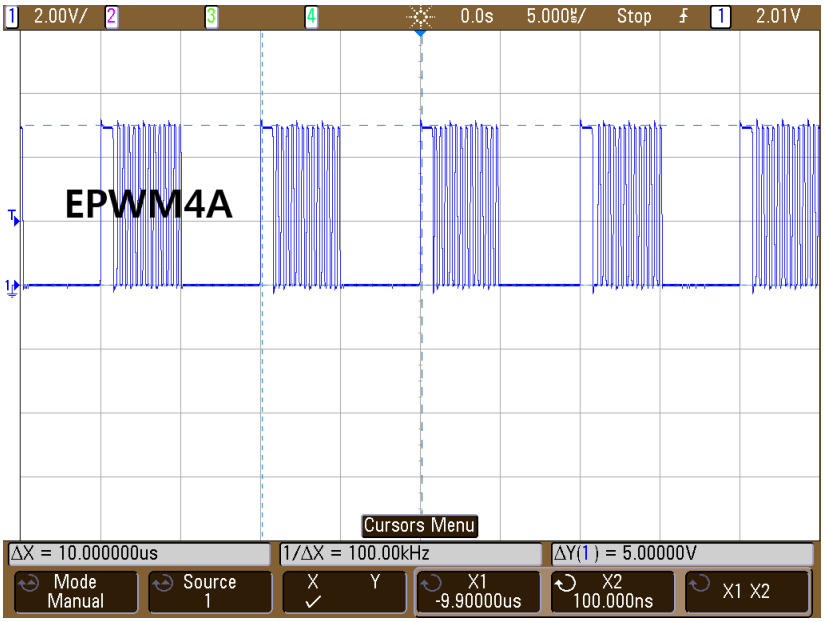
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-08] 1채널, PWM Chopping" 폴더

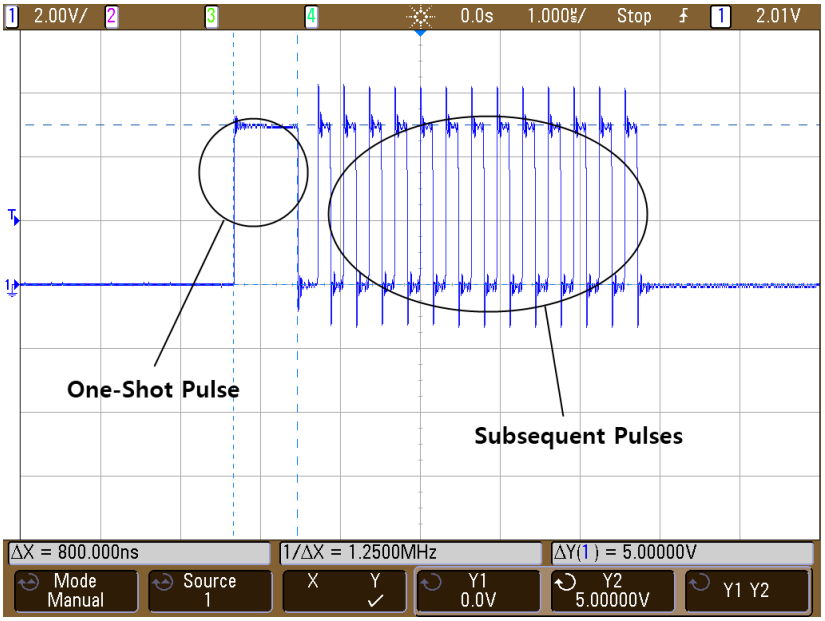
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 1채널 PWM Chopping" 게시물

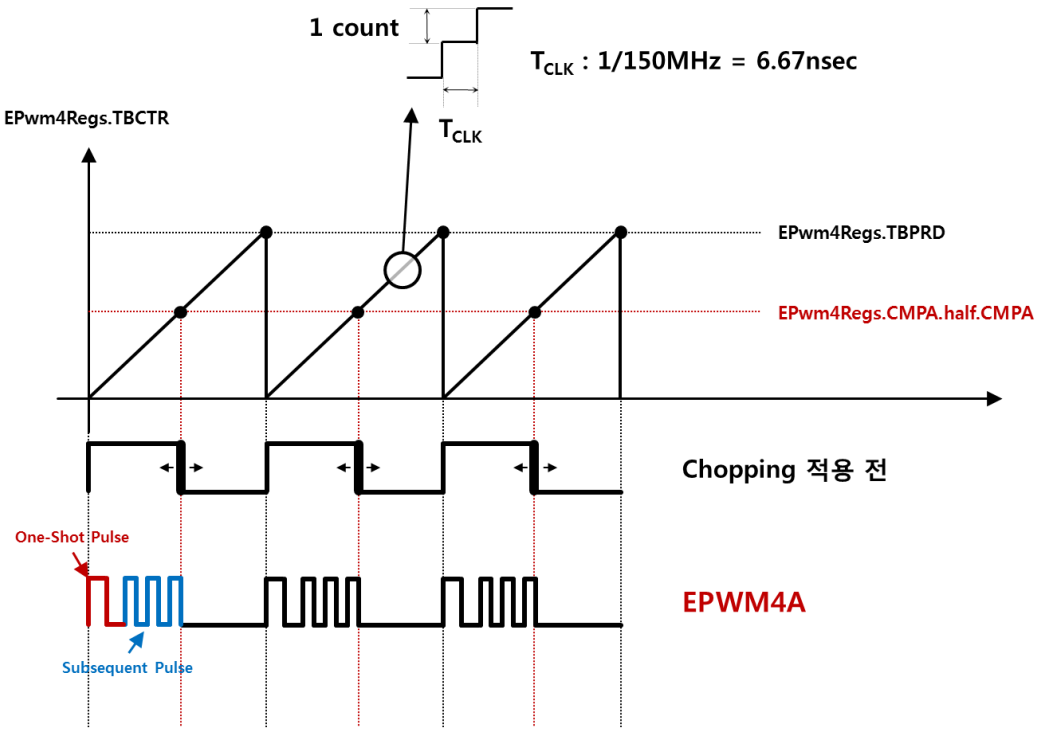
실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A) 핀을 계측장비(Oscilloscope)로 관찰하면, Chopping 이 적용된 PWM 출력 파형을 관찰하실 수 있습니다.




2.00V/
0.0s
5.000µ/
Stop
2.01V
EPWM4A
Cursors Menu
ΔX = 10.000000us
1/ΔX = 100.00kHz
ΔY(1) = 5.00000V
Mode
Manual
Source
1
X
Y
X1
-9.90000us
X2
100.000ns
X1 X2


2.00V/
0.0s
1.000µ/
Stop
2.01V
One-Shot Pulse
Subsequent Pulses
ΔX = 800.000ns
1/ΔX = 1.2500MHz
ΔY(1) = 5.00000V
Mode
Manual
Source
1
X
Y
Y1
0.0V
Y2
5.00000V
Y1 Y2


1 count
T_{CLK} : 1/150MHz = 6.67nsec
EPwm4Regs.TBCTR
T_{CLK}
EPwm4Regs.TBPRD
EPwm4Regs.CMPA.half.CMPA
Chopping 적용 전
One-Shot Pulse
Subsequent Pulse
EPWM4A

4.9 PWM Trip 기능 확인하기

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 서로 상보적인(Complementary) 2 채널의 PWM 을 출력해보고, 여기에 EPWM 모듈이 제공하는 Trip 기능을 적용하여 결과를 확인합니다. Trip 기능은 오류/경고/비상 상황에 대한 PWM 로직 상태를 결정하는 용도로 주로 사용됩니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM09_2Ch_PWM_Trip
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM09_2Ch_PWM_Trip
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM09_2Ch_PWM_Trip

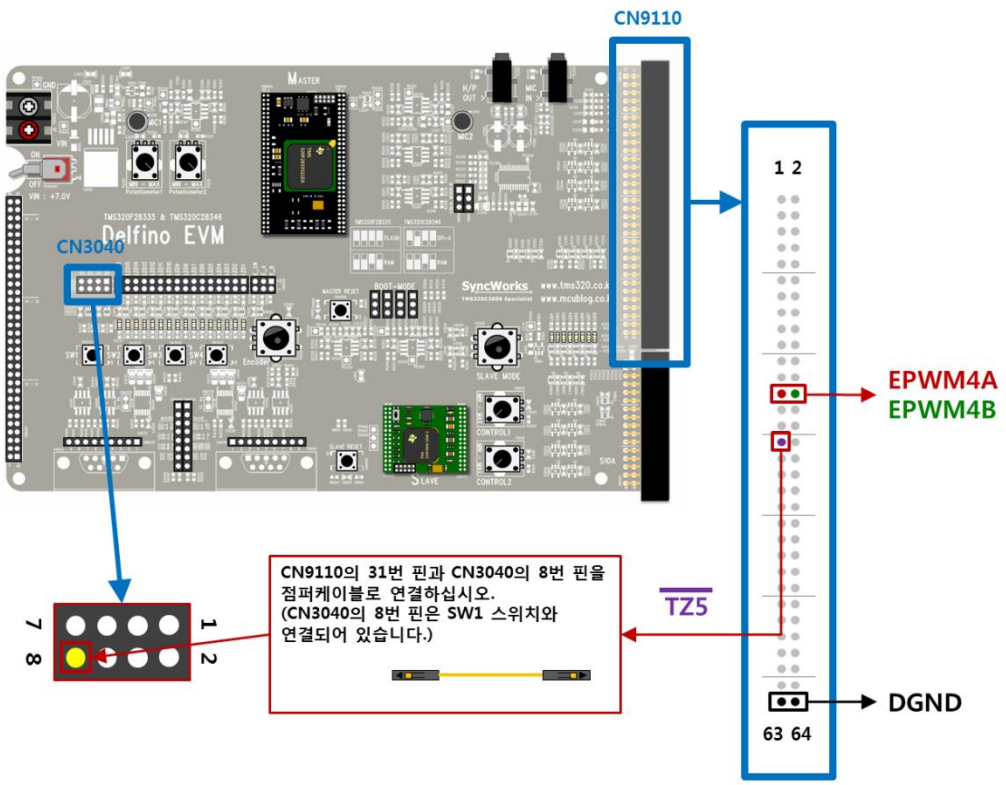
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-09] EPWM 모듈의 Trip 기능 확인하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - EPWM 모듈의 Trip 기능 확인하기" 게시물

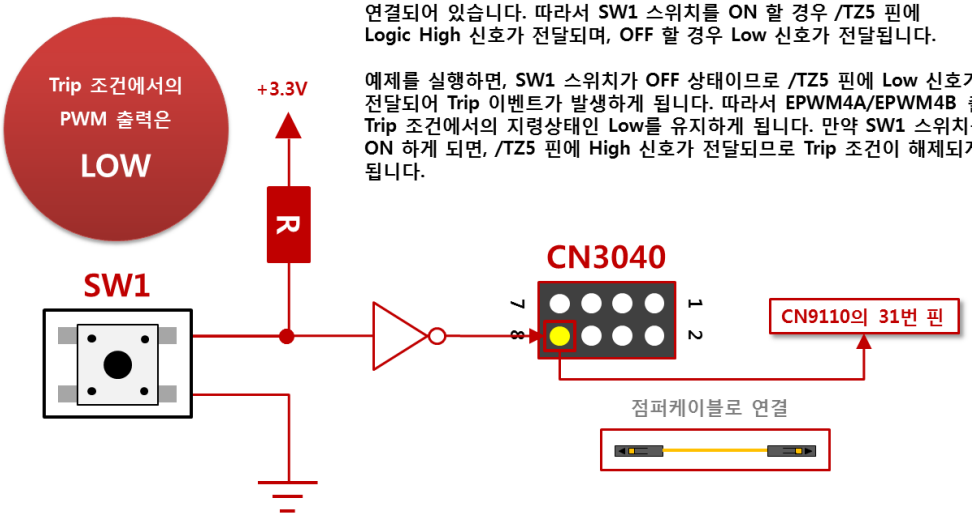
실습해 보기

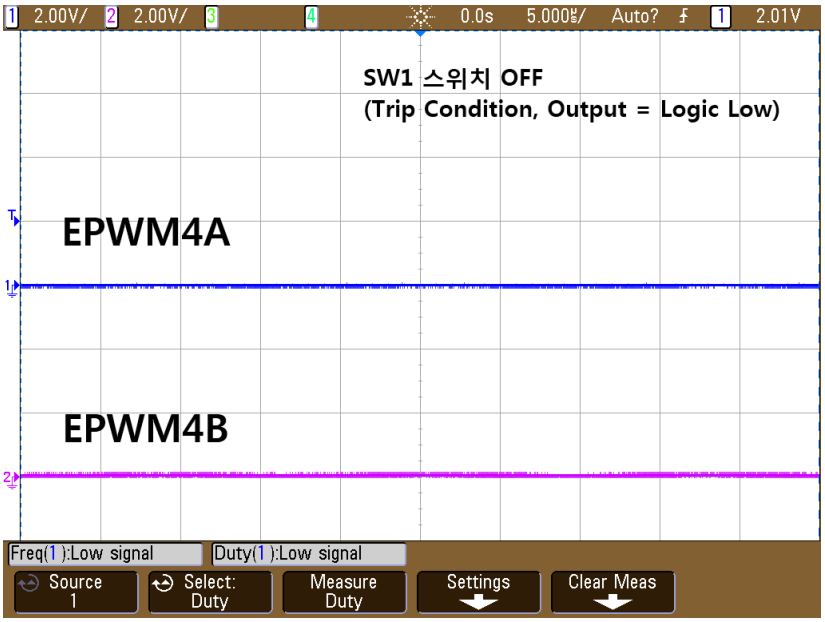
Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B) 핀을 계측장비(Oscilloscope)로 관찰하면서, SW1 스위치(SW3020)를 통해 Trip 조건을 발생시켜 봅니다. 예제 실습을 위한 Delfino EVM 상의 점퍼 케이블 연결은 아래의 이미지를 참고해주십시오.

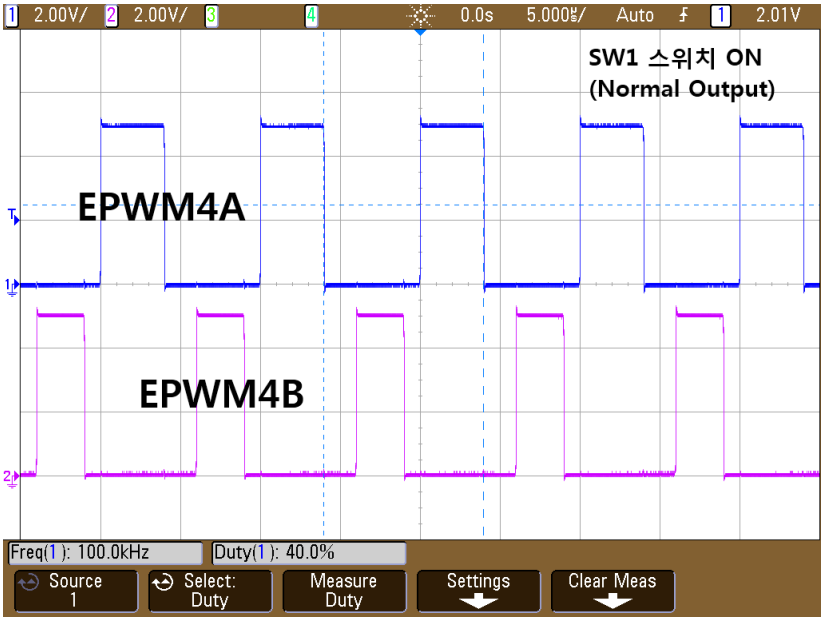


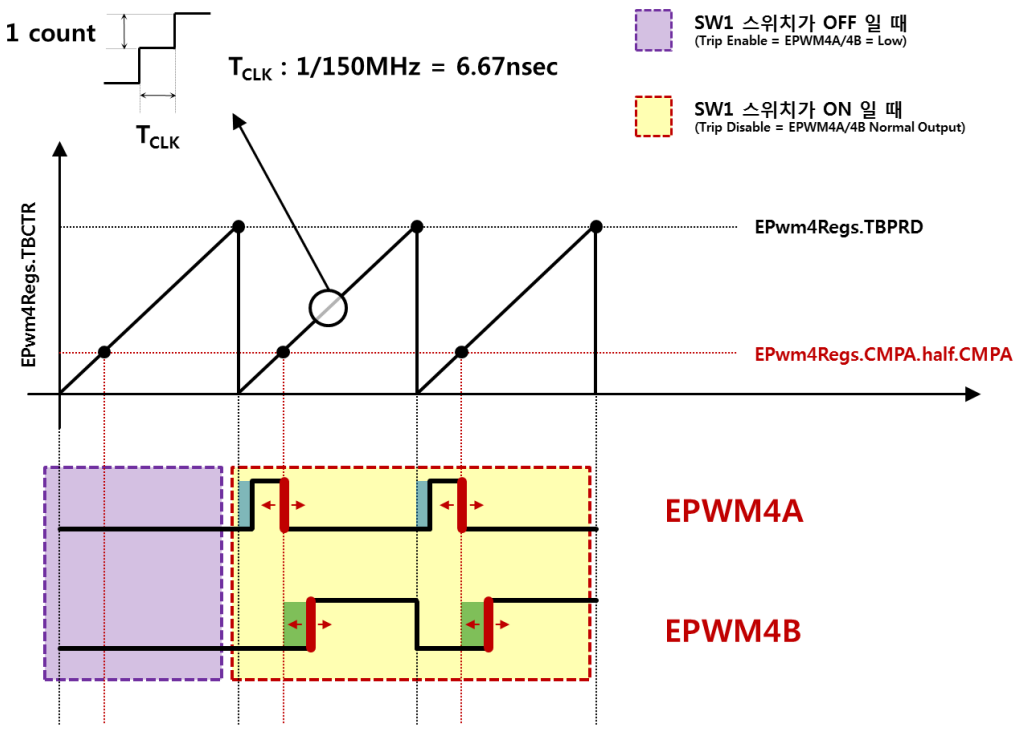
CN3040의 8번 핀은 아래와 같은 회로로 SW1 스위치와 연결되어 있으며, 점퍼 케이블을 통해 CN9110의 31번 핀(/TZ5)과도 연결되어 있습니다. 따라서 SW1 스위치를 ON 할 경우 /TZ5 핀에 Logic High 신호가 전달되며, OFF 할 경우 Low 신호가 전달됩니다.

예제를 실행하면, SW1 스위치가 OFF 상태이므로 /TZ5 핀에 Low 신호가 전달되어 Trip 이벤트가 발생하게 됩니다. 따라서 EPWM4A/EPWM4B 출력은 Trip 조건에서의 지령상태인 Low를 유지하게 됩니다. 만약 SW1 스위치를 ON 하게 되면, /TZ5 핀에 High 신호가 전달되므로 Trip 조건이 해제되게 됩니다.




SW1 스위치 OFF
(Trip Condition, Output = Logic Low)
EPWM4A
EPWM4B
Freq(1):Low signal
Duty(1):Low signal
Source 1
Select: Duty
Measure Duty
Settings
Clear Meas


SW1 스위치 ON
(Normal Output)
EPWM4A
EPWM4B
Freq(1): 100.0kHz
Duty(1): 40.0%
Source 1
Select: Duty
Measure Duty
Settings
Clear Meas


1 count
T_{CLK} : 1/150MHz = 6.67nsec
T_{CLK}
SW1 스위치가 OFF 일 때
(Trip Enable = EPWM4A/4B = Low)
SW1 스위치가 ON 일 때
(Trip Disable = EPWM4A/4B Normal Output)
EPwm4Regs.TBCTR
EPwm4Regs.TBPRD
EPwm4Regs.CMPA.half.CMPA
EPWM4A
EPWM4B

4.10 3상 Sinusoidal PWM 출력하기

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 3 상 Sinusoidal PWM 을 출력해보고, 그 출력에 Low-pass Filter 를 연결해서 PWM 에 실린 정보를 복원해봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM10_3Ch_Sinusoidal_PWM
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM10_3Ch_Sinusoidal_PWM
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM10_3Ch_Sinusoidal_PWM

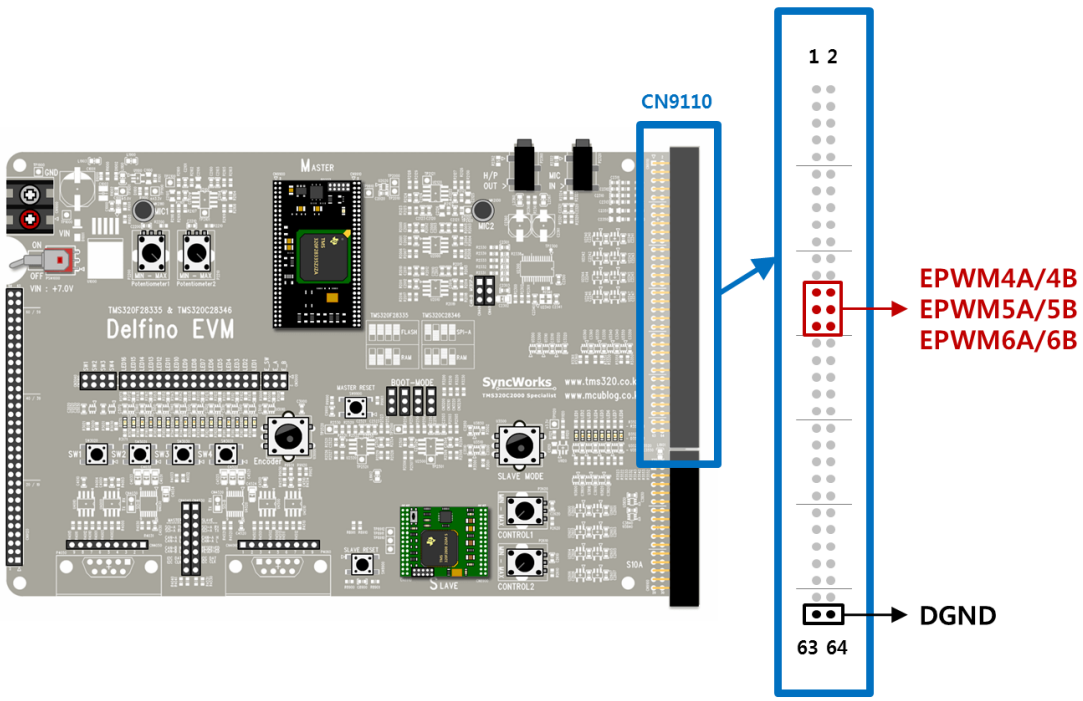
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-10] 3상 Sinusoidal PWM 출력하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 3상 Sinusoidal PWM 출력하기" 게시물

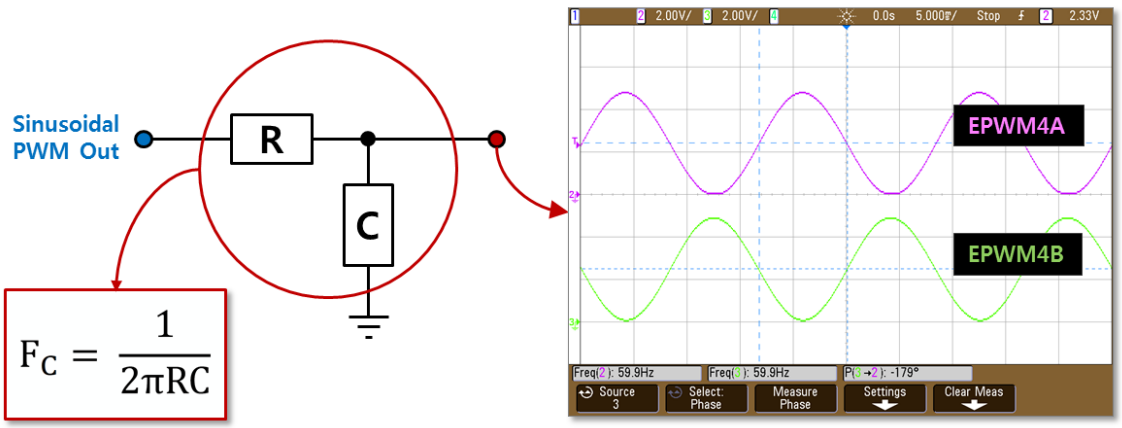
실습해 보기

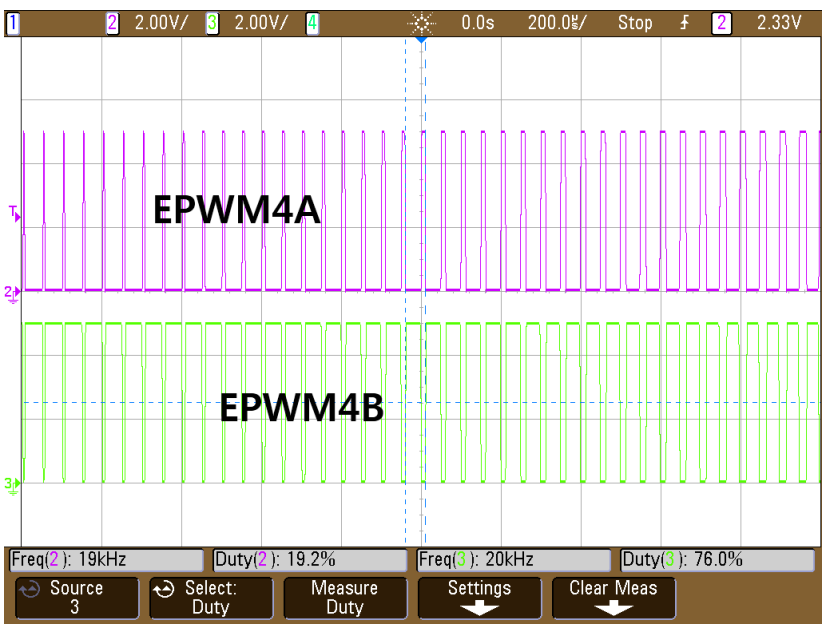
Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B), 27 번(EPWM5A), 28 번(EPWM5B), 29 번(EPWM6A), 30 번(EPWM6B) 핀을 계측장비(Oscilloscope)로 관찰해보고, 출력에 RC Low-pass Filter 를 연결해서 PWM 의 Carrier Signal 을 제거하여, PWM 에 실린 정보를 복원해봅니다.

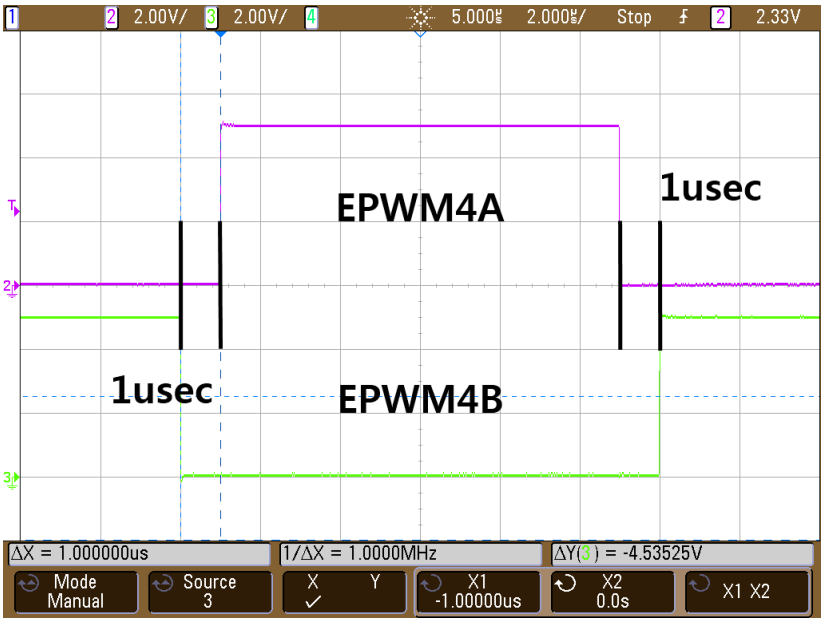


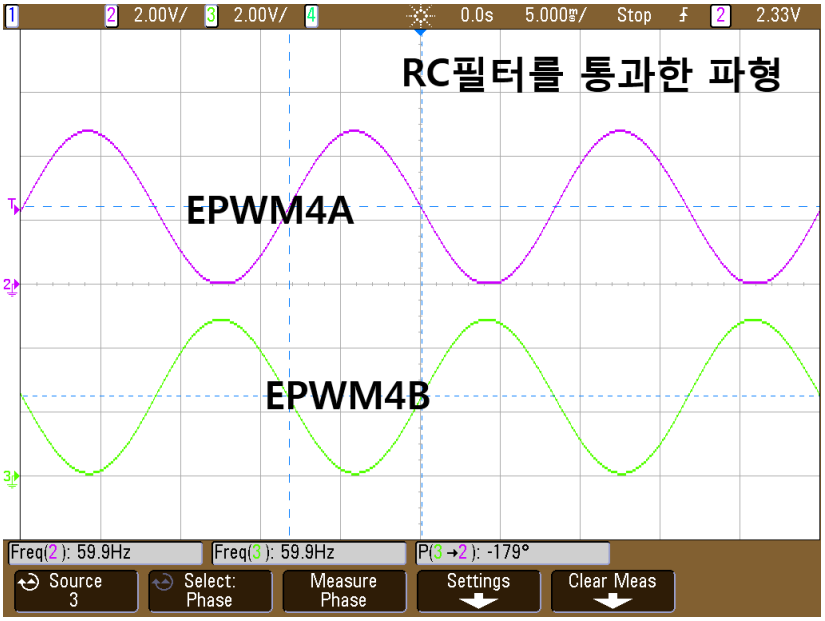
출력되고 있는 Sinusoidal PWM 출력에 R, C로 구성된 간단한 1차 저역통과필터를 연결해봅니다. 필터를 통해, 높은 대역의 PWM Carrier Frequency가 제거되면, 비교 레지스터에 적용했던 원래의 60Hz Sine 파형을 복원해볼 수 있습니다.

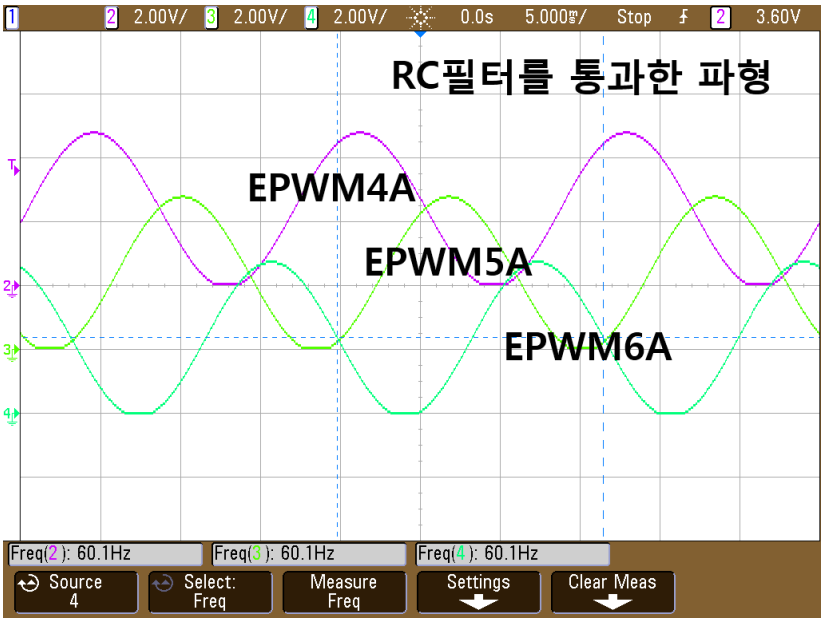
아래의 파형은 R = 1.8kΩ / C = 100nF을 이용한 FC = 884.2Hz의 저역통과필터를 통해 Sinusoidal PWM 출력을 확인한 결과입니다.





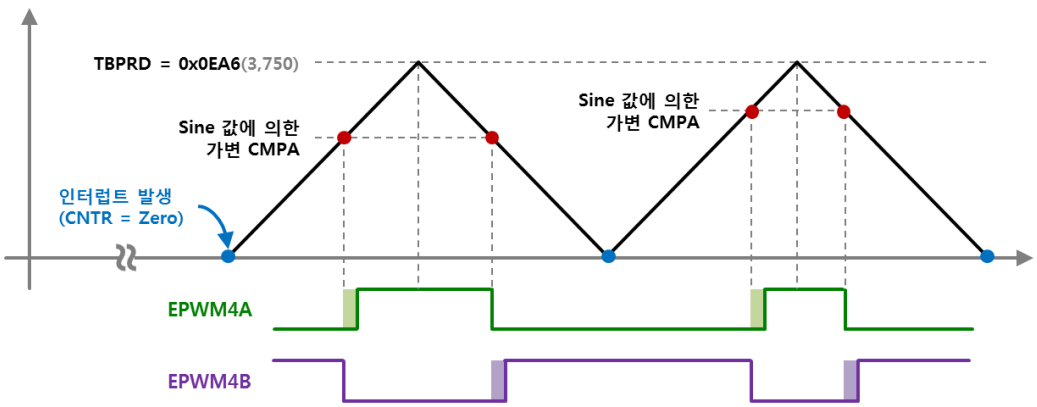

2.00V/
2.00V/
5.000µs
2.000µs/
Stop
2.33V
EPWM4A
1usec
1usec
EPWM4B
ΔX = 1.000000us
1/ΔX = 1.0000MHz
ΔY(3) = -4.53525V
Mode
Manual
Source
3
X
Y
X1
-1.00000us
X2
0.0s
X1 X2


2.00V/
2.00V/
0.0s
5.000ms/
Stop
2.33V
RC필터를 통과한 파형
EPWM4A
EPWM4B
Freq(2): 59.9Hz
Freq(3): 59.9Hz
P(3→2): -179°
Source
3
Select:
Phase
Measure
Phase
Settings
Clear Meas



- ✓ PWM Carrier Frequency = 20kHz (TBPRD = 3,750)
- ✓ PWM Duty-ratio = 20kHz로 샘플링 된 60Hz의 Sine 값으로 가변
- ✓ Timer Count Mode = Up/Down-Count (Symmetric)
- ✓ Timer Clock = 150MHz (TBCLK) = 6.67nsec
- ✓ Set EPWM4A on CNTR = CMPA (When the timer is incrementing)
- ✓ Clear EPWM4A on CNTR = CMPA (When the timer is decrementing)

예제에서 EPWM4B 출력은 타이머 카운터 값과 CMPB 레지스터 값을 통해 Action-Qualifier 서브모듈에서 정의하지 않고 있으며, Dead-Band 서브모듈에 의해 Falling Edge에 지연이 추가된 EPWM4A 출력의 반전형태로 생성되고 있습니다.



EPWM5A/5B, EPWM6A/6B도 동일한 설정

4.11 3상 Phase-Interleaved PWM 출력하기

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 3 상 Phase-Interleaved PWM 을 출력해봅니다. (EPWM 모듈 간 위상차 제어방법 학습).

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM11_3Ch_Phase_Interleaved_PWM
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM11_3Ch_Phase_Interleaved_PWM
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM11_3Ch_Phase_Interleaved_PWM

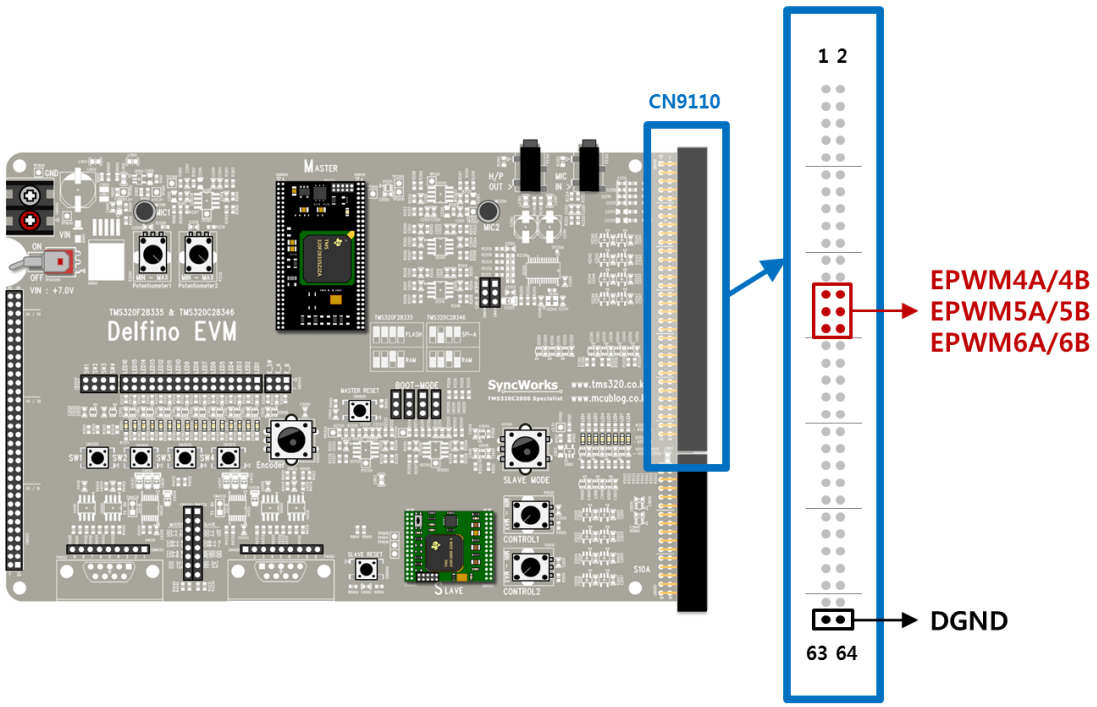
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

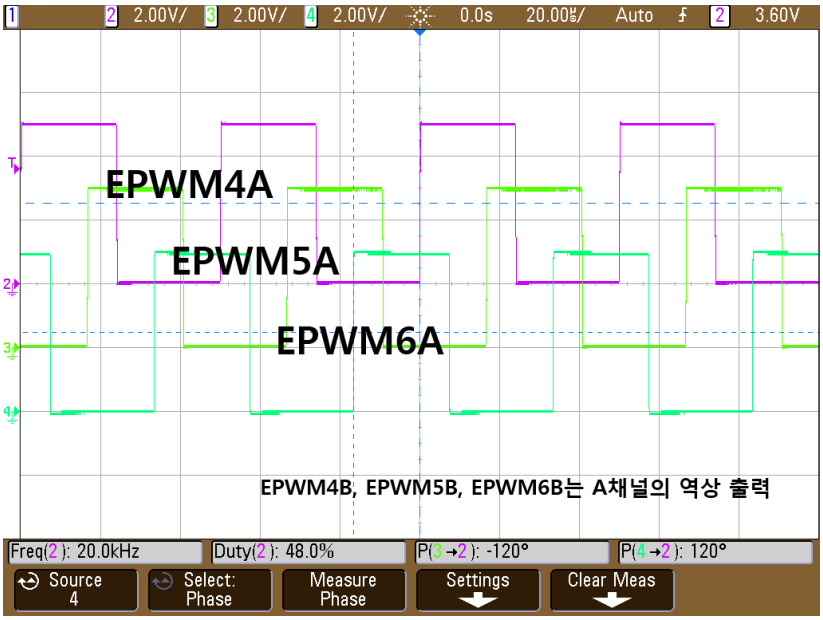
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-11] 3상 Phase-Interleaved PWM 출력하기" 폴더

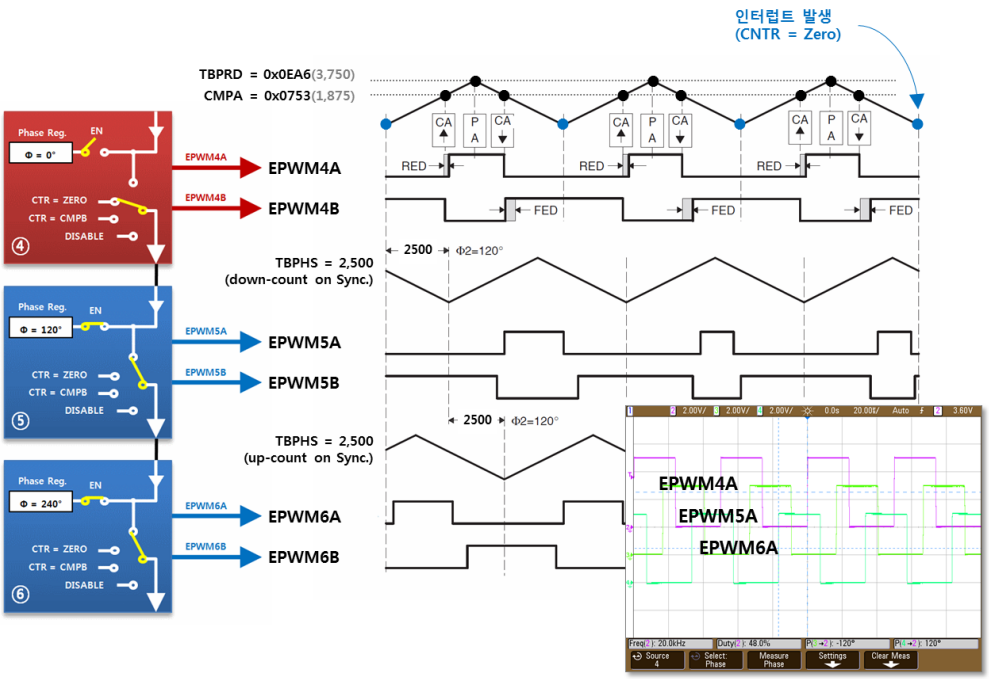
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 3상 Phase-Interleaved PWM 출력하기" 게시물

실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 25 번(EPWM4A), 26 번(EPWM4B), 27 번(EPWM5A), 28 번(EPWM5B), 29 번(EPWM6A), 30 번(EPWM6B) 핀을 계측장비(Oscilloscope)로 관찰해봅니다.




EPWM4A
EPWM5A
EPWM6A
EPWM4B, EPWM5B, EPWM6B는 A채널의 역상 출력
Freq(2): 20.0kHz
Duty(2): 48.0%
P(3→2): -120°
P(4→2): 120°
Source 4
Select: Phase
Measure Phase
Settings
Clear Meas


인터럽트 발생
(CNTR = Zero)
TBPRD = 0x0EA6(3,750)
CMPA = 0x0753(1,875)
Phase Reg.
EN
Φ = 0°
CTR = ZERO
CTR = CMPB
DISABLE
EPWM4A
EPWM4B
RED
FED
TBPHS = 2,500
(down-count on Sync.)
2500
Φ2=120°
Φ = 120°
EPWM5A
EPWM5B
TBPHS = 2,500
(up-count on Sync.)
Φ = 240°
EPWM6A
EPWM6B

4.12 2채널 PWM-to-Analog 출력 구현해보기

실습 목적

TMS320F2833x 의 Enhanced PWM Module 을 통해, 2 채널 PWM-to-Analog(DAC) 출력을 구현해봅니다. (본 예제에서는 Delfino EVM 에 미리 구현된 4 차 Butterworth LPF 를 이용하여, PWM-to-Analog 출력을 구현하고 있습니다.)

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EPWM12_2Ch_PWM_to_Analog
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EPWM12_2Ch_PWM_to_Analog
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EPWM12_2Ch_PWM_to_Analog

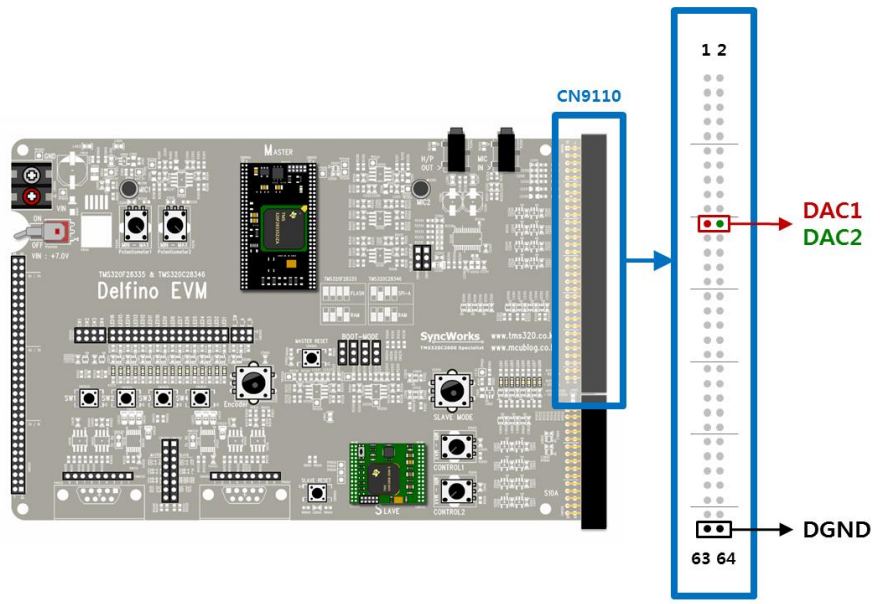
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

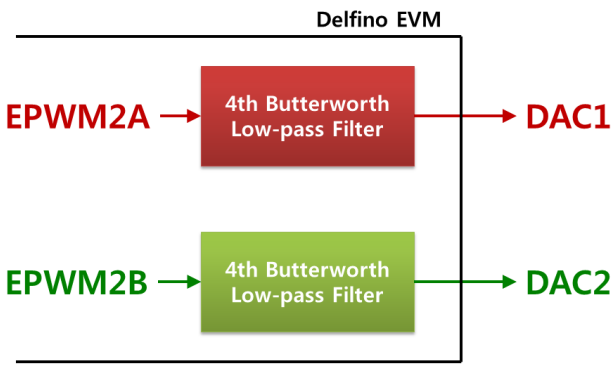
Delfino 개발지원 자료집(CD) 사용자의 경우
"[PWM-12] 2채널 PWM-to-Analog 출력 구현해보기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - 2채널 PWM-to-Analog 출력 구현해보기" 게시물

실습해 보기

Code Composer Studio 에서 예제를 실행하고 델피노 EVM CN9110 핀-헤더의 21 번(PWM-DAC_1), 22 번(PWM-DAC_2) 핀을 계측장비(Oscilloscope, Multimeter)로 관찰하면, PWM 을 4 차 Butterworth 저역 통과 필터를 통해 가공한 아날로그 전압 출력을 확인하실 수 있습니다.

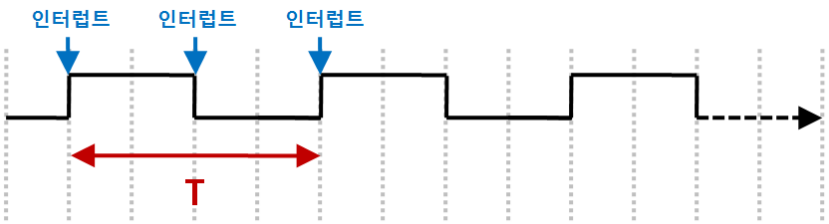




* DAC1/2 출력 전압 = DC+3.3V x PWM Duty-ratio

5 ECAP

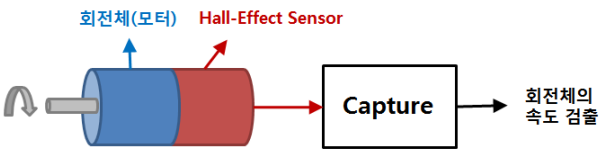
CAP은 Capture의 약자입니다. Delfino 프로세서에서는 CAP은 펄스 신호의 시간정보를 검출하는 기능을 합니다. 아래 그림과 같은 펄스 신호를 입력 받아서 펄스의 edge와 edge 사이의 시간 정보를 검출하여 주기, 듀티 등을 측정할 수 있습니다. 이외에도 아래 그림처럼 펄스의 edge 에서 인터럽트를 발생시킬 수 있어서, 외부 신호의 edge에 동기화하여 일정한 작업을 수행할 수도 있습니다.



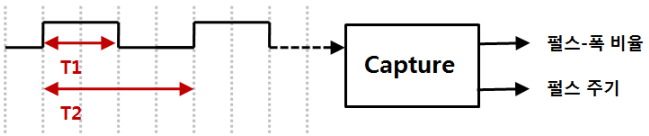
CAP의 응용 예

CAP은 다음과 같은 예에 사용할 수 있습니다.

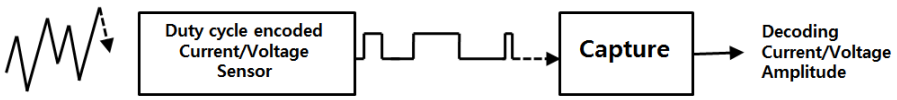
- **회전체의 속도를 검출할 수 있습니다.**
  - 예) Hall-Effect Sensor를 포함한 모터의 속도 검출



- **외부 신호의 펄스 주기, 펄스 폭 등을 측정할 수 있습니다.**
  (Pulse's Period & Duty-ratio)

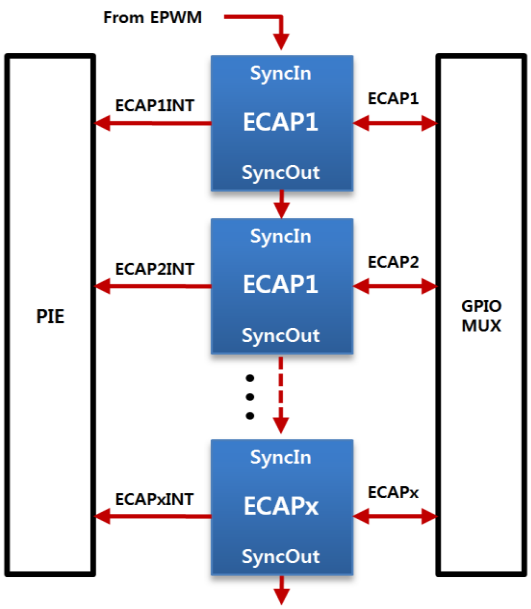


- **펄스-폭 변조 타입의 전류/전압 센서로부터의 출력 정보를 해석할 수 있습니다.**



Delfino 프로세서의 ECAP 특징

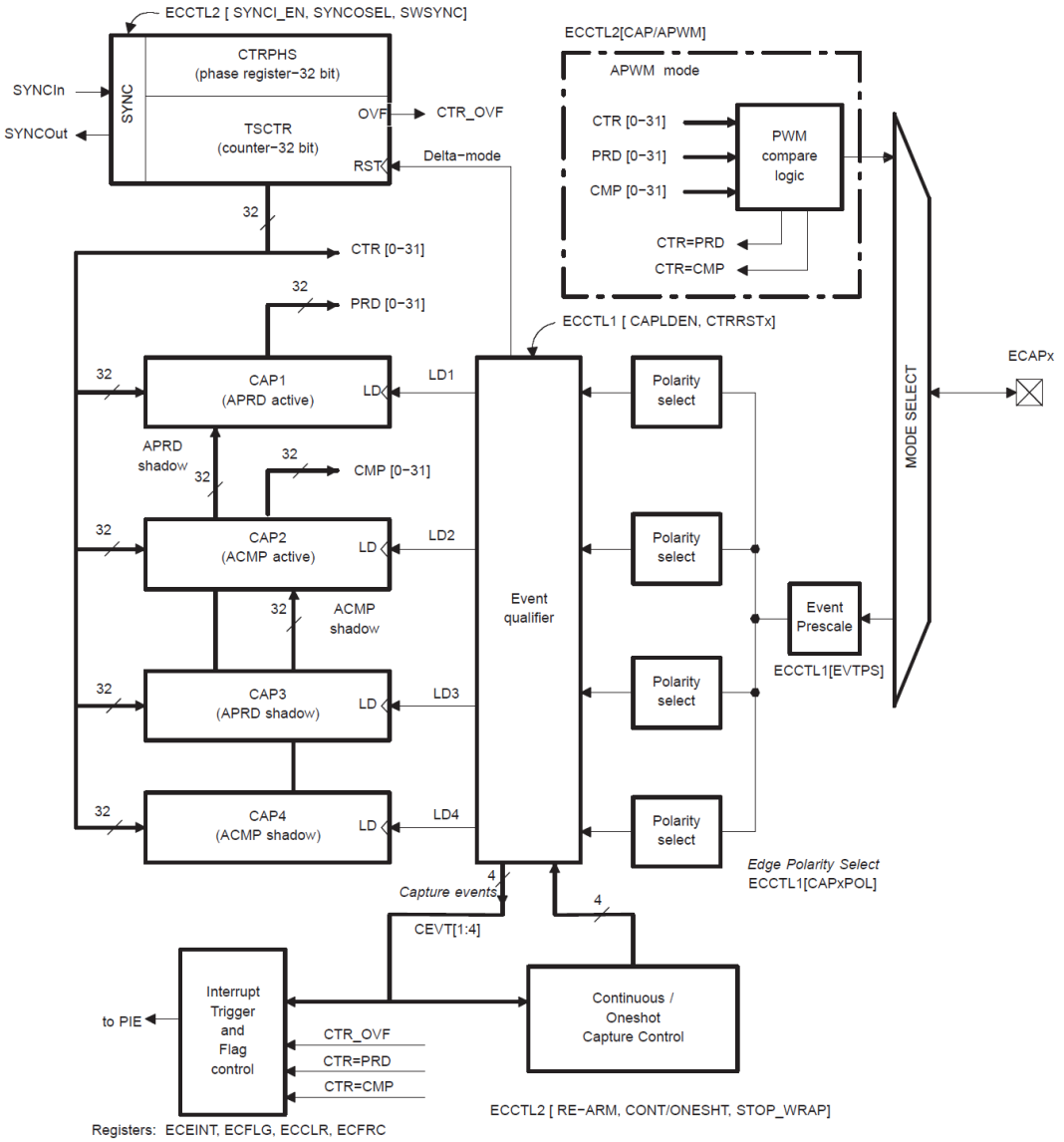
Delfino 프로세서는 CAP는 ECAP 라고 표시되어 있는데, 이는 Enhanced CAP라는 의미로 기존 CAP 보다 진보된, 향상된 CAP 라고 보시면 됩니다.



- 각 Capture 핀 별로 독립된 모듈 제공
- 32Bit 타이머 기반
  - Overflow 없이 최장 42.9sec 신호 타이밍 검출 가능
  - (100MHz 시스템 클럭 기준 / 150MHz의 경우 28.6sec)
- 4개의 32Bit Time-stamp 레지스터 (4 Event)
  - 각 이벤트 별 Edge 설정 및 인터럽트 가능
- 입력 Capture 신호의 Prescaling 가능
- One-Shot/Continuous 운용
- 2가지 동작 모드 존재
  - Absolute Time-Stamp Mode
  - Time Difference(Delta) Mode
- APWM Mode
  - Capture가 필요치 않을 경우 ECAP 모듈을 통해 비-대칭 단상 PWM 출력 가능

Delfino 프로세서의 ECAP 구조

Delfino 프로세서의 CAP 모듈 구조는 아래 그림과 같습니다. 복잡해 보이지만, 핵심적인 것은 32 비트 카운터(TSCTR)과 4개의 버퍼(Time Stamp 레지스터: CAP1~CAP4) 입니다. CAP 모듈의 카운터(TSCRT)는 시스템 클럭(SYSCLKOUT)에 맞춰 카운팅 동작을 하고, 이때 외부에서 입력되는 펄스의 edge가 검출될 때 마다 그 순간의 카운터 값을 4개의 버퍼에 순차적으로 저장하는 방식으로 동작합니다.



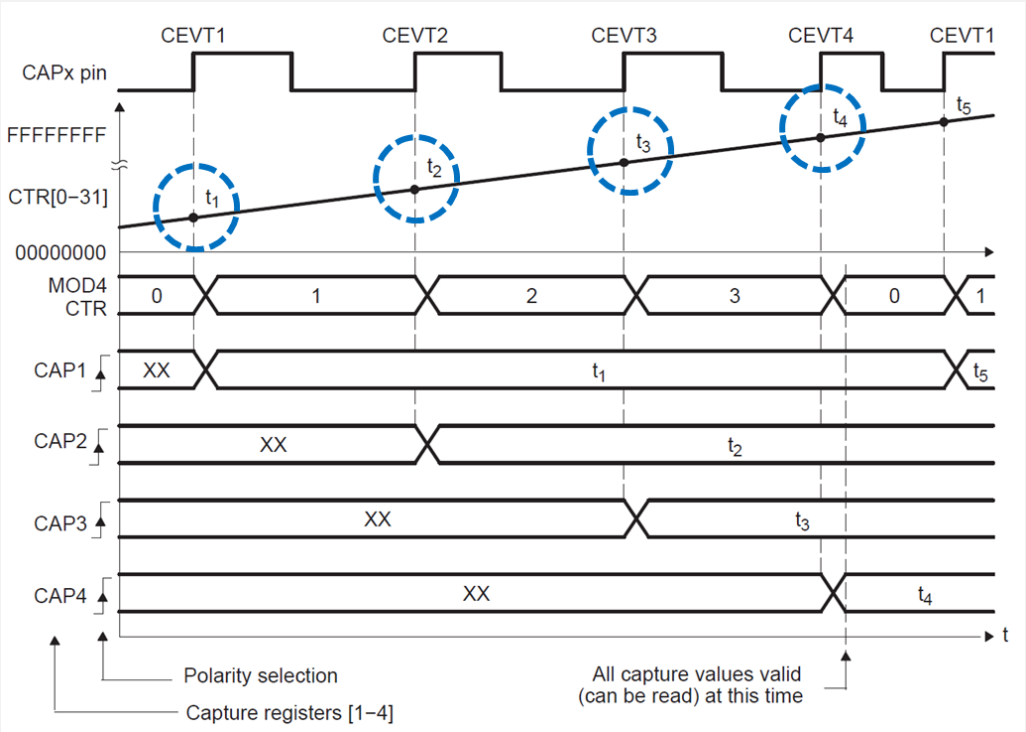
Delfino 프로세서의 ECAP 동작 모드

Delfino 프로세서의 CAP 모듈은 기본적으로 외부 입력 펄스에서 edge가 검출되는 시점의 카운터 값을 저장하는 방식으로 동작합니다. 하지만 여기에는 다음과 같은 2가지 모드가 있어서 사용자는 필요에 따라서 2가지 방식을 하나를 선택해서 사용할 수 있습니다.

◈ **Absolute Time-stamp Mode**

Absolute Time-stamp Mode로 입력 신호의 Rising Edge를 검출하는 예

- Capture Timer를 입력되는 신호의 Edge 검출과 관계없이 32-Bit 최대 값까지 상승 계수를 반복하도록 운용하며(Free-run), Edge 검출 이벤트(CEVTx) 발생 시 해당 Time-stamp 레지스터에 이벤트 발생 시점의 Capture Timer Counter 값을 기록하는 방식

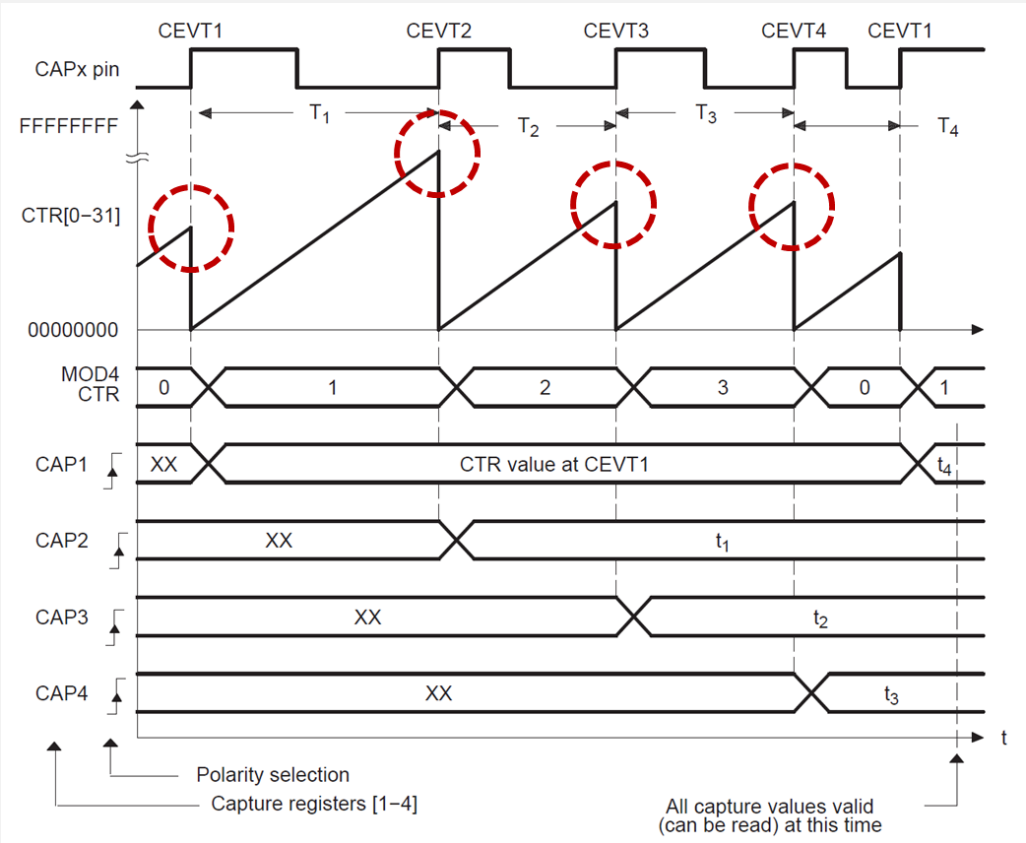


CEVTx에서 타이머 카운터 Reset 안 함 !!

◈ **Time Difference(Delta) Mode**

Time Difference(Delta) Mode로 입력 신호의 Rising Edge를 검출하는 예

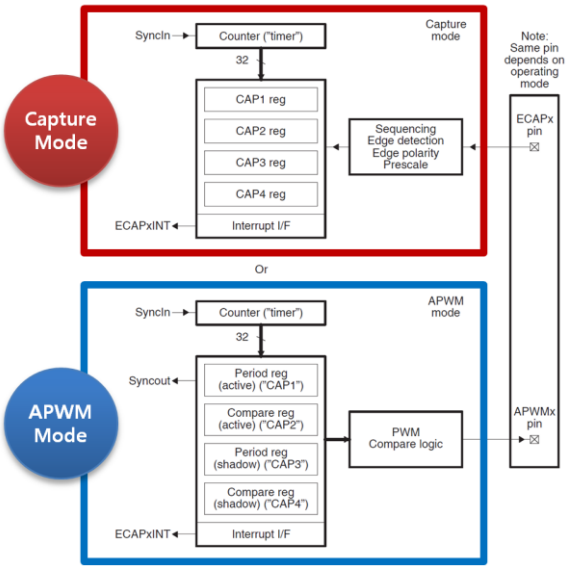
- Capture Timer를 입력되는 신호의 Edge 검출 이벤트 발생 시 "0"(Zero)으로 초기화하여, 이벤트 간 순수한 시간 차 정보를 Time-stamp 레지스터에 기록하는 방식



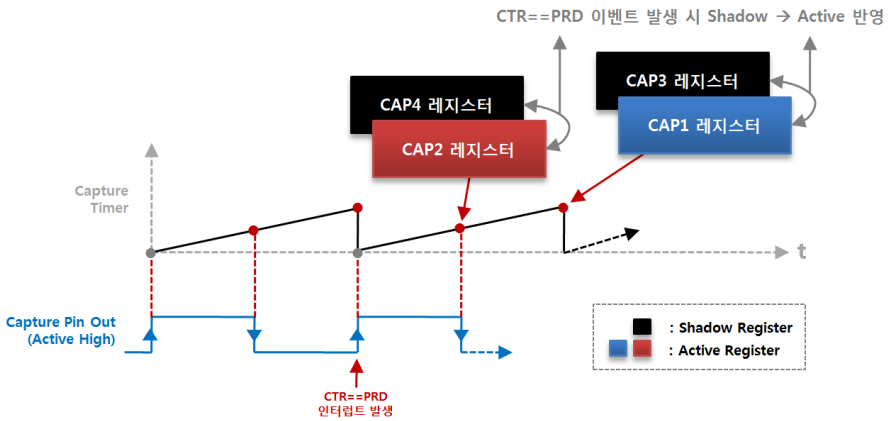
CEVTx에서 타이머 카운터 Reset !!

CAP 모드와 APWM 모드

Delfino 프로세서의 ECAP 모듈은 CAP 기능을 사용하지 않을 때, APWM 모드로 운용하여 단상 PWM을 생성할 수 있습니다. 이때 ECAPx 핀은 출력으로 설정되며, 4개의 Time-Stamp 레지스터가 PWM 출력을 위한 주기 및 비교 레지스터로 활용됩니다.



ECAP을 APWM 모드로 운용할 때 Capture Timer는 Up-Count 모드 만을 지원하며, 32비트 분해능의 비-대칭형 PWM 출력을 만들어 낼 수 있습니다. APWM 모드에서 Active 레지스터인 CAP1, CAP2 레지스터의 값을 수정할 경우 수정한 값이 바로 PWM 출력에 반영되고, Shadow 레지스터인 CAP3, CAP4 레지스터에 값을 넣을 경우 타이머 카운터가 주기 값과 일치할 때 Active 레지스터에 값이 반영되어 PWM 출력을 변화 시킵니다.



5.1 ECAP 모듈의 Edge 검출 인터럽트

실습 목적

TMS320F2833x 의 Enhanced Capture Module 을 통해, 펄스신호의 Edge 를 검출하고, Edge 검출시점에 인터럽트를 발생시켜봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ECAP01_Edge_Detect_Interrupt
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ECAP01_Edge_Detect_Interrupt
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ECAP01_Edge_Detect_Interrupt

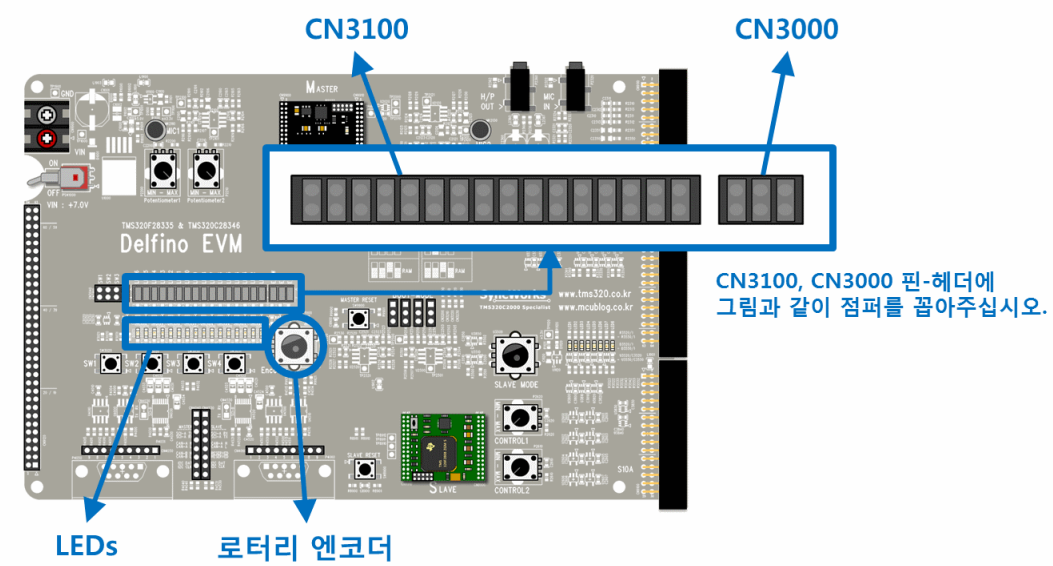
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[CAP-01] ECAP 모듈의 Edge 검출 인터럽트" 폴더

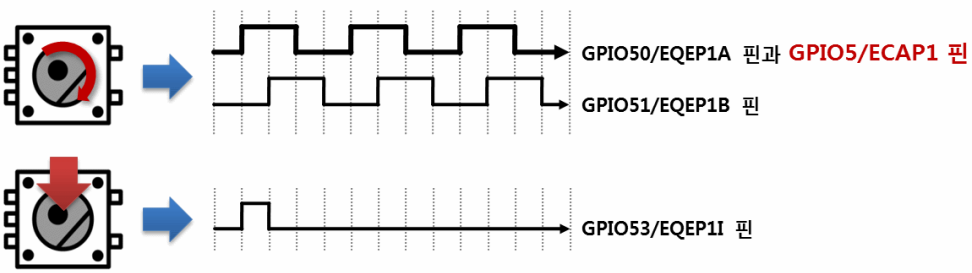
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - ECAP 모듈의 Edge 검출 인터럽트" 게시물

실습해 보기

Code Composer Studio에서 Real-time Mode로 예제를 실행하고, 델피노 EVM 중앙의 로터리 엔코더(U3000)를 회전시키면, 엔코더에 의해 발생되는 펄스의 Edge를 Enhanced Capture가 검출하고 인터럽트를 발생시키게 됩니다. 인터럽트 서비스 루틴에는 델피노 EVM의 LED 16개를 반전(Toggle)시키는 코드가 삽입되어 있습니다. (예제에 기본적으로 포함되어 있는 소스코드에서는 엔코더 펄스 1개 당 LED들이 1회 반전되도록 하고 있습니다. 예제 경로에 함께 포함되어 있는 ECAP01_Edge_Detect_Interrupt_Option.c 파일로 Main 소스코드를 변경하고 예제를 실행하면, 엔코더 펄스 4개 당 LED들이 1회 반전되게 됩니다. 두 소스코드에서 해당 동작을 구현하기 위해 어떻게 Enhanced Capture Module을 설정했는지 살펴보시기 바랍니다.)



Delfino EVM의 로터리 엔코더는 Incremental 방식의 엔코더로 1회전 당 24개의 Quadrature Pulse를 출력하도록 제작되어 있으며, 출력신호 A, B는 점퍼 연결을 통해 TMS320F28335 모듈의 ECAP, EQEP 기능 핀들에 전달되도록 설계되어 있습니다.
(노브 클릭을 통해 스위치로도 활용 가능하며, 이 신호는 EQEP1I 핀에 전달되게 됩니다.)



5.2 ECAP 모듈로 펄스의 주파수 및 듀티비율 알아내기

실습 목적

TMS320F2833x 의 Enhanced Capture Module 을 통해, 펄스신호의 Edge 들을 검출하고, 이를 시간정보로 환산하여, 펄스의 주파수 및 듀티비율을 계산해봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ECAP02_Calc_EdgeToEdge_Time
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ECAP02_Calc_EdgeToEdge_Time
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ECAP02_Calc_EdgeToEdge_Time

통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[CAP-02] ECAP 모듈로 펄스의 주파수 및 듀티비율 알아내기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - ECAP 모듈로 펄스의 주파수 및 듀티비율 알아내기" 게시물

실습해 보기

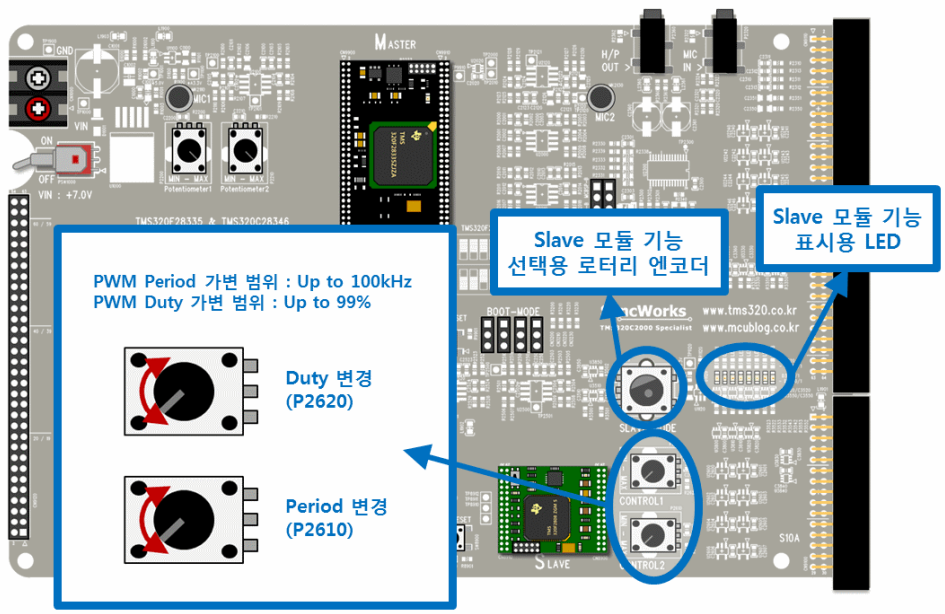
Code Composer Studio에서 Real-time Mode로 예제를 실행하고, 아래의 그림을 참조하여, Slave 모듈의 학습기능 2번을 선택합니다. 기능선택에 따라 Slave 모듈이 TMS320F28335에 전달하는 PWM 펄스를 Enhanced Capture가 검출하고 인터럽트를 발생시키게 됩니다. 인터럽트 서비스 루틴에는 델피노 EVM의 LED 16개를 반전(Toggle)시키는 코드와 함께, ECAP 모듈이 획득한 Edge와 Edge 사이의 차이를 시간정보로 환산하는 코드가 삽입되어 있습니다.

Delfino EVM의 학습용 Slave 모듈이 가변 Period, Duty의 PWM을 출력하도록 설정합니다.

① Slave 모듈 기능 선택용 로터리 엔코더를 회전시켜, LED 2번이 점등되도록 합니다.

② 로터리 엔코더의 노브를 눌러, 기능 2번이 선택되도록 합니다. (LED 수회 점멸)

③ Slave 모듈의 지령 변경용 가변 저항 2개를 통해 출력되는 PWM의 Period 및 Duty를 변경할 수 있습니다.



프로그램 실행한 다음 CCS에서 Capture counter1, 2 값을 확인해 보세요.

5.3 ECAP 모듈을 단상 PWM 발생기로 활용하기

실습 목적

TMS320F2833x 의 Enhanced Capture Module 을 Auxiliary PWM Mode 로 운용하여, 단상 PWM 을 출력해봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩ECAP03_Auxiliary_PWM_Mode
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩ECAP03_Auxiliary_PWM_Mode
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩ECAP03_Auxiliary_PWM_Mode

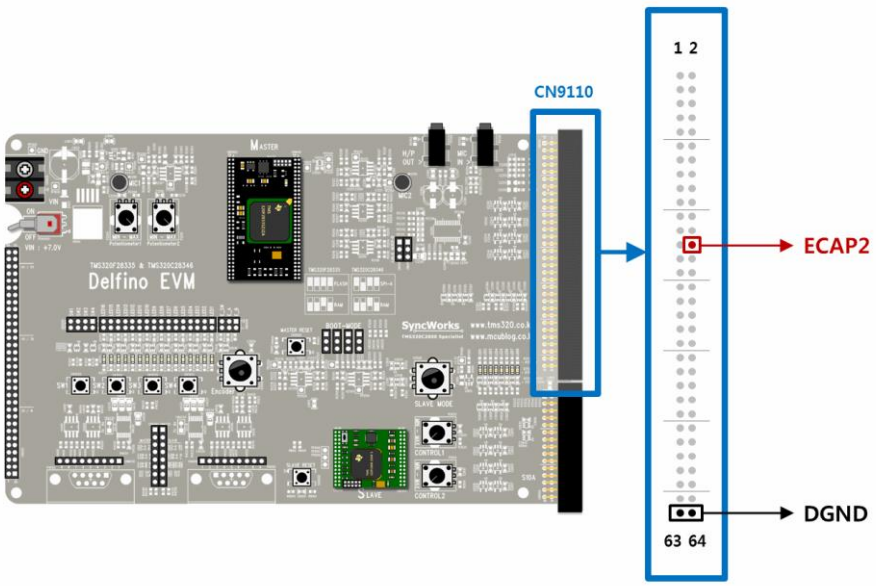
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

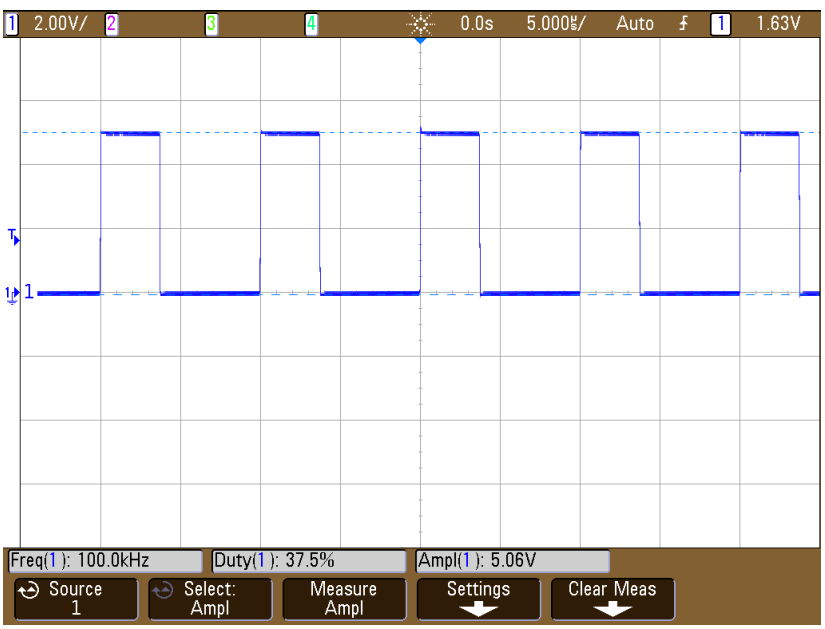
Delfino 개발지원 자료집(CD) 사용자의 경우
"[CAP-03] ECAP 모듈을 단상 PWM 발생기로 활용하기" 폴더

싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - ECAP 모듈을 단상 PWM 발생기로 활용하기" 게시물

실습해 보기

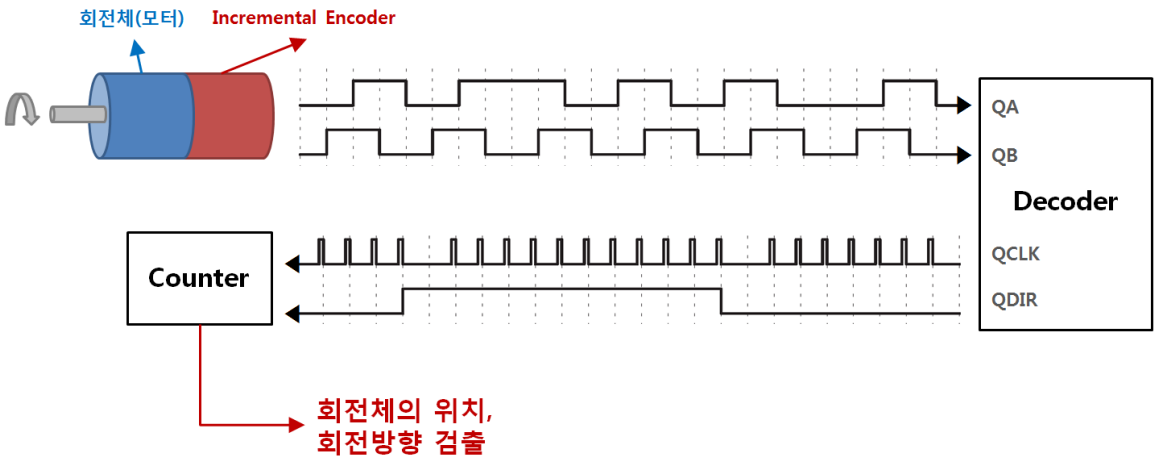
Code Composer Studio에서 Real-time Mode로 예제를 실행하고 델피노 EVM 우측 CN9110 핀-헤더의 26번(ECAP2)을 통해 출력되는 PWM 파형을 계측장비(Oscilloscope)로 관찰해봅니다.





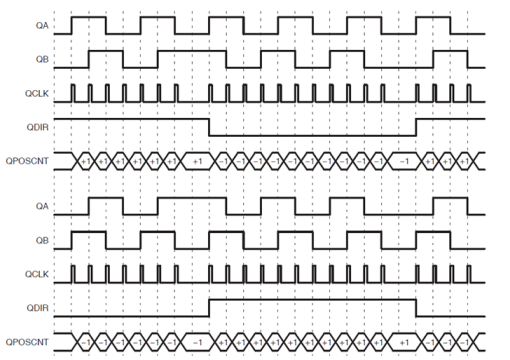
6 EQEP

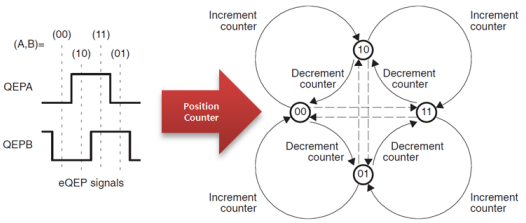
QEP는 Quadrature Encoder Pulse 의 약자입니다. 여기서 Quadrature Encoder Pulse 라는 것은 모터 등의 회전 정보를 나타내는 일종의 펄스 스트림 입니다. 보통 모터 제어 분야에서 사용되는 엔코더에서 출력되는 신호라고 보면 됩니다. 이러한 QEP 신호를 처리하는 장치도 그냥 QEP 라고 지칭하고 있습니다. QEP 모듈은 모터 엔코더에 출력되는 신호를 분석해서 모터의 위치, 회전방향 등의 정보를 획득하는 것이 가능합니다.



- **Quadrature Decoder 진리치표(Truth Table)**
  (Quadrature Count Mode)

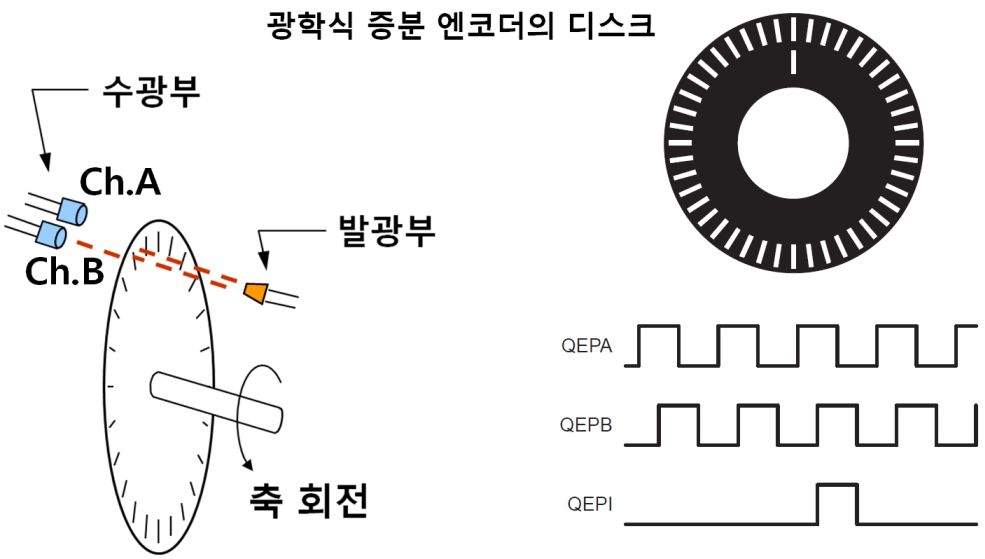
Previous Edge	Present Edge	QDIR	QPOSCNT
QA↑	QB↑	UP	Increment
	QB↓	DOWN	Decrement
	QA↓	TOGGLE	Increment or Decrement
QA↓	QB↓	UP	Increment
	QB↑	DOWN	Decrement
	QA↑	TOGGLE	Increment or Decrement
QB↑	QA↑	DOWN	Increment
	QA↓	UP	Decrement
	QB↓	TOGGLE	Increment or Decrement
QB↓	QA↓	DOWN	Increment
	QA↑	UP	Decrement
	QB↑	TOGGLE	Increment or Decrement





QEP의 원리

아래 그림은 광학식 증분형 엔코더의 동작 원리는 보여 주는 그림입니다. 회전축에 원판을 끼우고, 원판에는 원주를 따라서 구멍(Slit)을 냅니다. 구멍을 기준으로 한쪽에는 발광 다이오드를 장착하고, 반대쪽에서는 수광 다이오드를 그림에서 보듯이 Ch.A와 Ch.B에 설치합니다.



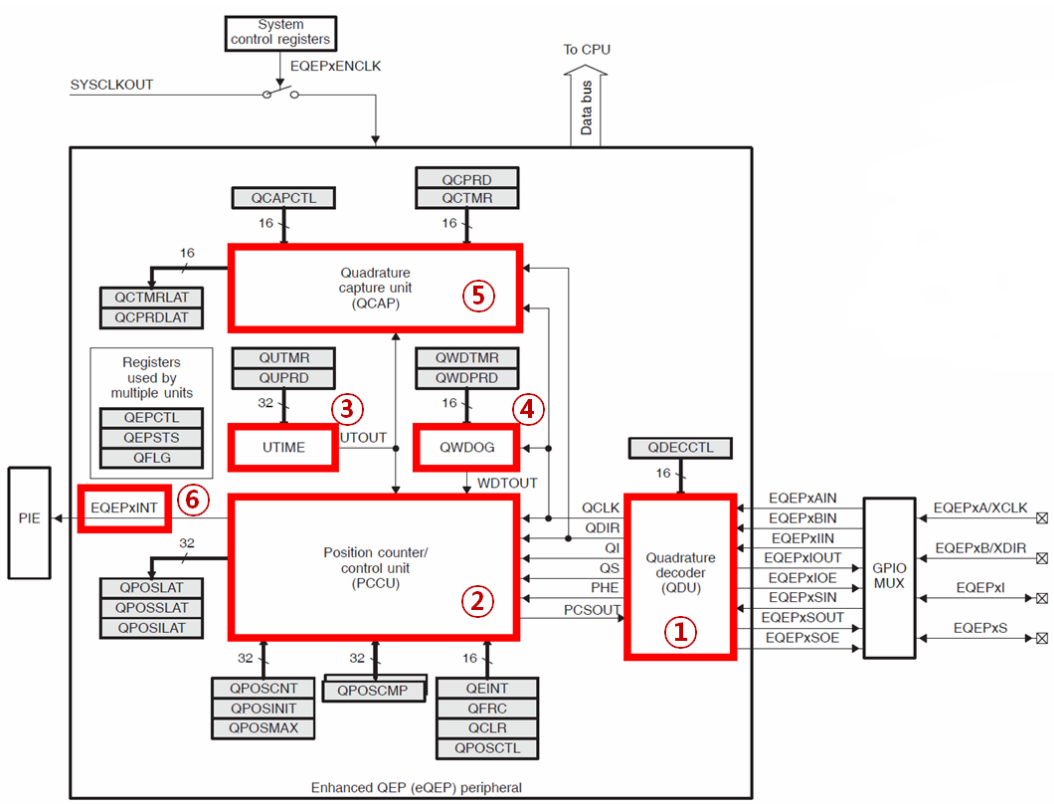
광학식 증분 엔코더의 출력 신호

수광 다이오드가 A,B가 물리적으로 떨어져 있다 보니, 수광 시점에 차이가 생깁니다. 이 차이는 펄스의 위상차이로 나타나게 됩니다. 이를 이용한 것이 바로 QEP 입니다. 우측 하단의 펄스 그림은 Ch.A와 Ch.B에서 반응하는 신호를 각각 나타낸 것입니다. 보시는 것처럼 각 채널에 위상차가 보입니다. 어떤 채널에 신호의 위상이 앞서느냐는 것을 가지고 회전 방향을 구분할 수 있고, 단위 시간당 펄스 수 혹은 주기를 계산해서 각속도 즉 회전 위치 또한 계산할 수 있습니다.

멘 밑에 있는 펄스는 인덱스 펄스(QEPI) 인데, 모터의 초기 위치를 명확하게 하기 위해서 사용됩니다. 인덱스 슬릿을 기준으로 몇 도 회전했는지 계산할 수 있는 것 입니다.

Delfino 프로세서의 EQEP 구조

Delfino 프로세서는 QEP은 EQEP 라고 표시되어 있는데, 이는 Enhanced QEP라는 의미로 기존 QEP 보다 진보된, 향상된 QEP 라고 보시면 됩니다 EQEP 모듈은 아래 그림과 같이 총 6개의 서브 모듈로 구성되어 있습니다. EQEP 모듈은 이러한 서브 모듈 들을 통해서 모터의 위치, 회전방향, 속도 등외 여러가지 이벤트들을 처리할 수 있습니다.

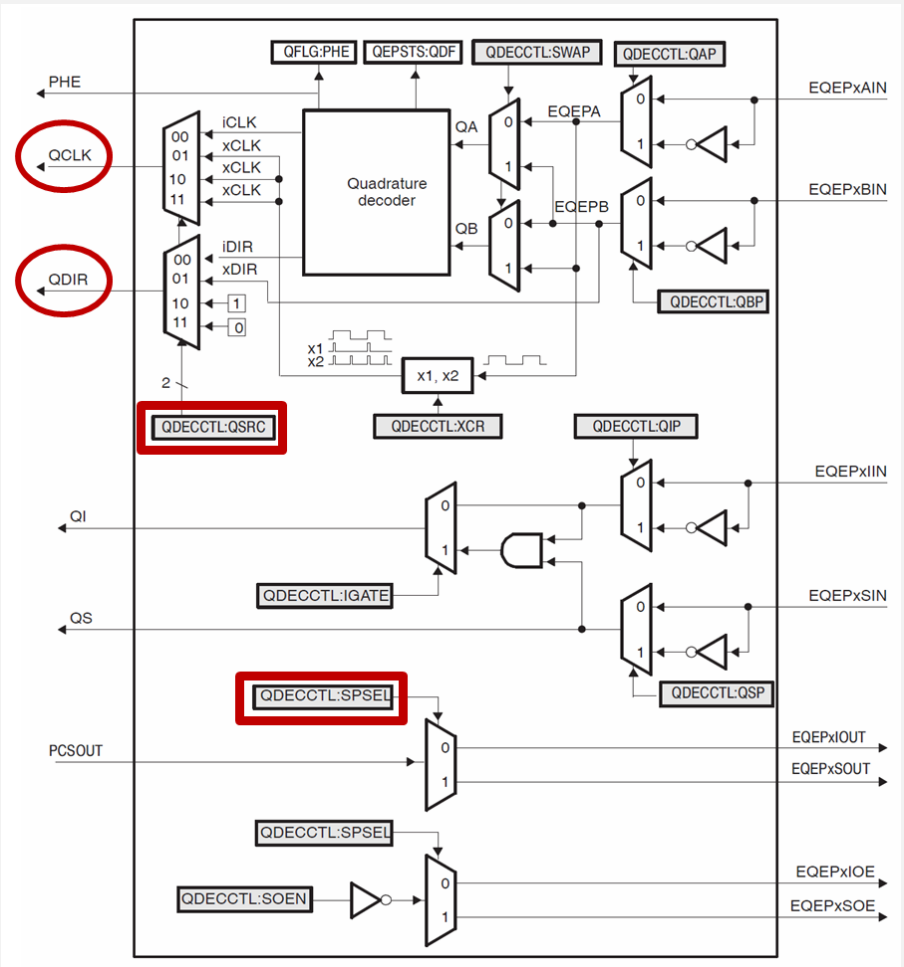


① **Quadrature Decoder**
② **Position Counter**
③ **Unit Timer**
④ **EQEP Watchdog Timer**
⑤ **Quadrature Edge-Capture**
⑥ **EQEP Interrupts**

◈ ① **Quadrature Decoder**

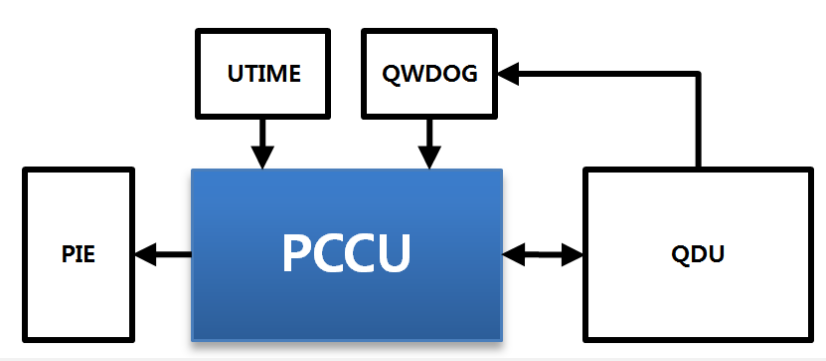
센서로부터의 Quadrature 신호 입력(QEPA, B)을 Pulse Stream(QCLK)으로 가공하고, 신호의 위상 전후 관계에 따라 계수방향(QDIR)을 해석

- **Decode 모드 선택 (QDECCTL:QSRC)**
  - ✓ **Quadrature-count 모드**: 입력되는 두 Quadrature 신호를 Pulse Stream(CLK)과 방향으로 해석
  - ✓ **Direction-count 모드**: EQEPA, B 핀을 통해 외부에서 입력되는 CLK과 방향 정보를 그대로 이용 (일부 센서들은 Quadrature 신호 대신에 미리 해석된 CLK, DIR을 출력함)
  - ✓ **Up-count 모드 (속도 검출용)**: Position Counter 계수 방향을 상승 계수 방향으로 고정하고 외부에서 입력되는 CLK만 전달
  - ✓ **Down-count 모드 (속도 검출용)**: Position Counter 계수 방향을 하강 계수 방향으로 고정하고 외부에서 입력되는 CLK만 전달
- **Position-compare Sync Pulse 출력 (QDECCTL:SPSEL)**
  - ✓ Position Counter로부터 전달받은 Position-compare Sync Output Pulse를설정에 따라 EQEPI 혹은 EQEPS 핀을 통해 출력 가능



◈ ② Position Counter Control Unit (PCCU) - 1

- **4가지 동작모드 제공 (QEPCTL.PCRM)**
  - ✓ **Position Counter Reset on Index Event**
    (Index 이벤트 발생 시 다음 EQEP CLK에서 Position Counter를 Reset)
    - QEPSTS.QDF 비트가 1일 경우: "0"(Zero)으로 Reset
    - QEPSTS.QDF 비트가 0일 경우: "QPOSMAX" 값으로 Reset
  - ✓ **Position Counter Reset on Maximum Position**
    (Position Counter가 "0" 혹은 "QPOSMAX" 값과 일치 할 경우 다음 EQEP CLK에서 Reset)
    - QEPSTS.QDF 비트가 1일 경우: QPOSCNT==QPOSMAX 시 "0"(Zero)으로 Reset (QFLG.PCO → Set)
    - QEPSTS.QDF 비트가 0일 경우: QPOSCNT==0 시 "QPOSMAX" 값으로 Reset (QFLG.PCU → Set)
  - ✓ **Position Counter Reset on First Index Event**
    (P.C. Reset on Index Event 시와 동일)
    - 단, 첫 번째 Index 이벤트에서만 유효
  - ✓ **Position Counter Reset on Unit Time Out Event**
    (속도 측정용, Up/Down-count Decode 모드에서 유용)
    - Unit Time Out 이벤트 발생 시 QPOSCNT 값이 QPOSLAT 레지스터에 기록되고, 계수 방향에 따라"0(Zero)" 혹은"QPOSMAX" 값으로 QPOSCNT를 Reset



◈ ② **Position Counter Control Unit (PCCU) - 2**

Index, Strobe 이벤트 발생 시 Position Counter Latch 가능

- **Index Event Latch (QEPCTL.IEL)**
  - ✓ Index 이벤트 발생 시 QPOSCNT 값을 QPOSILAT 레지스터에 기록하고, 해당 시점에서의 계수 방향을 QEPSTS 레지스터의 QDLF 비트에 기록함 (QFLG.IEL Set)
  - ✓ QEPCTL 레지스터의 IEL 비트가 1(01[2]) 일 경우: Index 핀 입력 신호의 Rising Edge에서 Latch
  - ✓ QEPCTL 레지스터의 IEL 비트가 2(10[2]) 일 경우: Index 핀 입력 신호의 Falling Edge에서 Latch
  - ✓ QEPCTL 레지스터의 IEL 비트가 3(11[2]) 일 경우: Index Event Marker 에서 Latch

- **Strobe Event Latch (QEPCTL.SEL)**
  - ✓ Strobe 이벤트 발생 시 QPOSCNT 값을 QPOSSLAT 레지스터에 기록 (QFLG.SEL Set)
  - ✓ QEPCTL 레지스터의 SEL 비트가 0 일 경우: Strobe 핀 입력 신호의 Rising Edge에서 Latch
  - ✓ (단, QDECCTRL 레지스터의 QSP 비트가 1일 경우 Falling Edge에서 Latch)
  - ✓ QEPCTL 레지스터의 SEL 비트가 1 일 경우:
    - QEPSTS.QDF 비트가 1일 경우: Strobe 핀 입력 신호의 Rising Edge에서 Latch
    - QEPSTS.QDF 비트가 0일 경우: Strobe 핀 입력 신호의 Falling Edge에서 Latch

Position Counter 초기화 기능 제공 (Initialization)

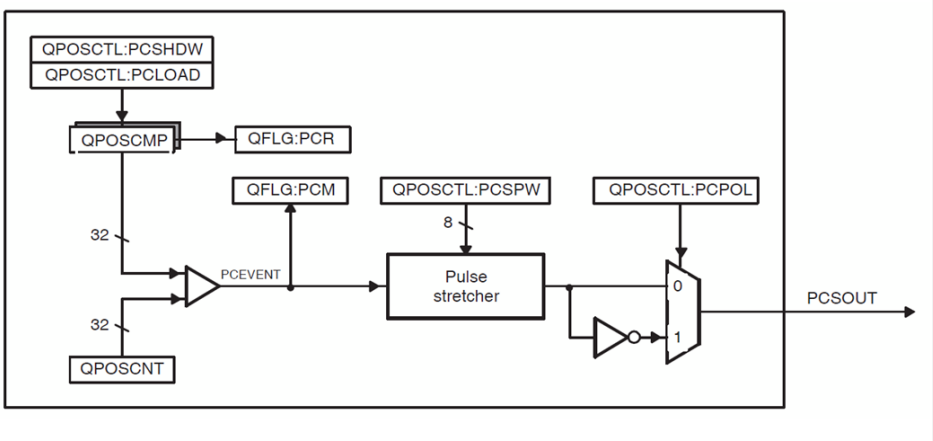
- **3가지 이벤트 발생 시 QPOSCNT 값을 QPOSINIT 레지스터의 값으로 초기화 함**
- **Index 이벤트 초기화 (QEPCTL.IEI)**
  - ✓ QEPCTL 레지스터의 IEI 비트가 2(10[2]) 일 경우: Index 신호의 Rising Edge에서 초기화 (QPOSCNT=QPOSINIT)
  - ✓ QEPCTL 레지스터의 IEI 비트가 3(11[2]) 일 경우: Index 신호의 Falling Edge에서 초기화 (QPOSCNT=QPOSINIT)
- **Strobe 이벤트 초기화 (QEPCTL.SEI)**
  - ✓ QEPCTL 레지스터의 SEI 비트가 2(10[2]) 일 경우 : Strobe 신호의 Rising Edge에서 초기화 (QPOSCNT=QPOSINIT)
  - ✓ QEPCTL 레지스터의 SEI 비트가 3(11[2]) 일 경우 :
    - QEPSTS.QDF 비트가 1일 경우 : Strobe 핀 입력 신호의 Rising Edge에서 초기화
    - QEPSTS.QDF 비트가 0일 경우 : Strobe 핀 입력 신호의 Falling Edge에서 초기화
- **S/W 초기화 (QEPCTL.SWI)**
  - ✓ QEPCTL 레지스터의 SWI 비트를 1로 Set 할 경우 QPOSCNT=QPOSINIT

◈ ② Position Counter Control Unit (PCCU) - 4

- **Position-compare 기능 제공**
  - ✓ 외부 회전체의 특정 위치에서 인터럽트를 발생시키거나, EQEPI 혹은 EQEPS 핀으로 동기(Sync) 신호 출력 가능
  - ✓ 미리 설정한 비교 값(QPOSCMP)과 Position Counter(QPOSCNT) 값이 일치할 경우, QFLG.PCM 비트를 Set (인터럽트로 활용 가능)하고, Quadrature Decoder에 Position-compare Sync Output (PCSOUT) Pulse를 전달

- **PCSOUT 펄스-폭 및 Polarity 설정 가능**
  - ✓ QPOSCTL 레지스터의 PSCPW 비트(펄스-폭)
    - (PSCPW+1) x 4 x SYSCLKOUT cycle
  - ✓ QPOSCTL 레지스터의 PCPOL 비트(Polarity)
    - 0: Active High
    - 1: Active Low

- **QPOSCMP는 Shadow 레지스터 보유**
  - ✓ QPOSCTL 레지스터의 PSSHDW 비트
    - 0: 바로 적용
    - 1: Shadow 모드
  - ✓ QPOSCTL 레지스터의 PCLOAD 비트 (In Shadow Mode)
    - 0: QPOSCNT=0 일 때 Active 레지스터에 적용 (QFLG.PCR → Set)
    - 1: QPOSCNT=QPOSCMP 일 때 Active 레지스터에 적용 (QFLG.PCR → Set)

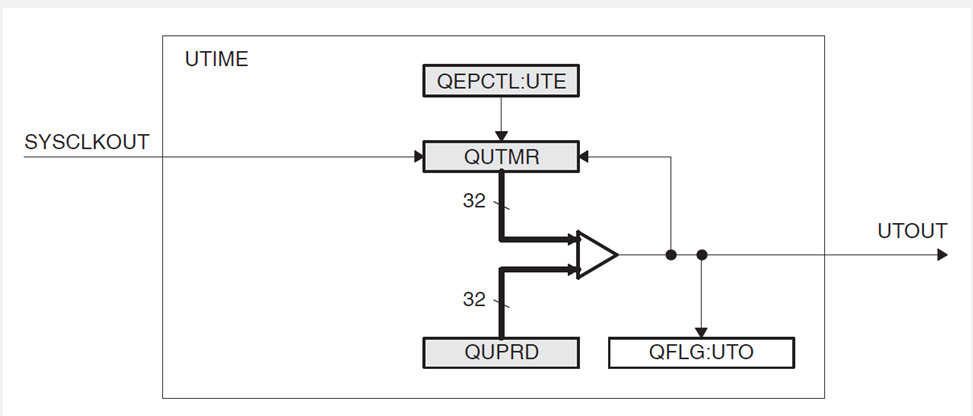


◈ ③ **Unit Timer**

- **시스템 클럭(SYSCLKOUT)을 기반으로 동작하는 32Bit 타이머**
  - ✓ 속도 검출을 위한 기준 시간 제공

- **설정된 주기 값(QUPRD)과 타이머 카운터(QUTMR)가 일치하면**
  Unit Time Out(UTOUT) 이벤트 발생 [QFLG.UTO → Set]
  - ✓ 인터럽트 원으로 활용 가능

- **UTOUT 이벤트 발생 시 3가지 레지스터 값 Latch 가능 (속도검출에 활용)**
  - ✓ Position Counter Latch
    - · QPOSLAT=QPOSCNT
  - ✓ Edge-Capture Timer Latch
    - · QCTMRLAT=QCTMR
  - ✓ Edge-Capture Period Latch
    - · QCPRDLAT=QCPRD

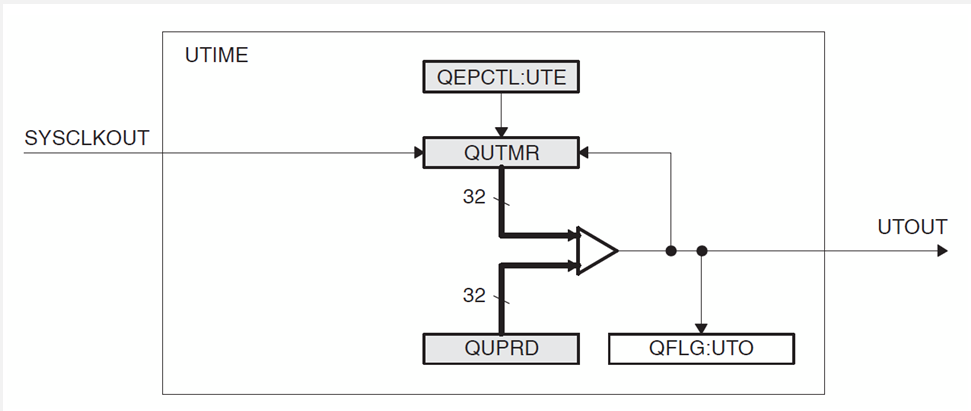


◈ ④ **EQEP Watchdog Timer**

- **고정 64분주된 시스템 클럭(SYSCLKOUT/64) 기반의 16Bit 타이머**
  - ✓ 외부 회전체의 실속(Stall) 검출에 활용
  - ✓ Watchdog Timer Period를 최대 값(0xFFFF)으로 설정 시
    - Period Matching 이벤트까지 약 41.9msec

- **Quadrature Decoder로부터의 QCLK 입력이 Watchdog Timer를 Reset 시킴**

- **QCLK 입력이 없을 경우(Stall 발생)**
  - ✓ Watchdog Time Out 이벤트 발생(QFLG.WTO → Set)
  - ✓ 인터럽트 원으로 활용 가능



◈ ⑤ Quadrature Edge-Capture - 1

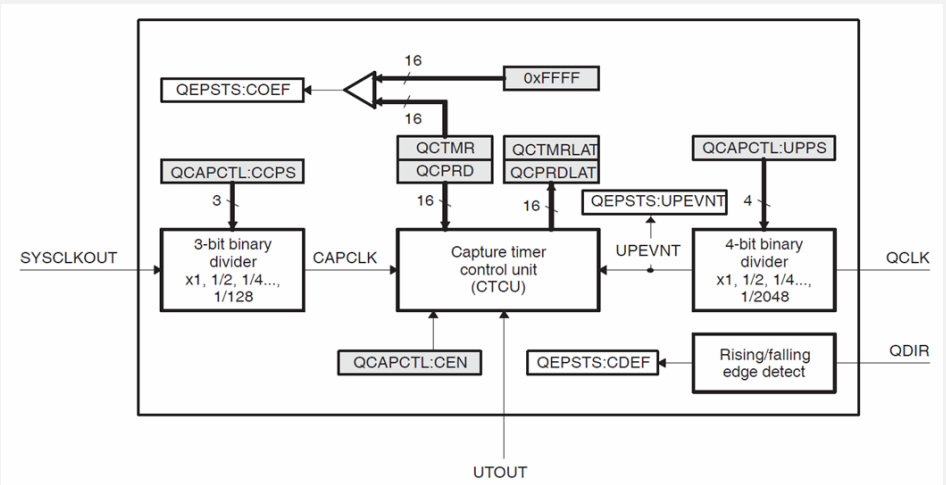
- **EQEP 모듈에 내장된 16Bit Capture 회로**
  - ✓ 저속의 외부 회전체 속도 검출에 활용

- **3Bit 클럭 Prescaler 내장**
  - ✓ 시스템 클럭(SYSCLKOUT)을 최대 128분주하여 Capture Timer 클럭으로 사용

- **Quadrature Decoder의 QCLK이 분주된 UPEVNT 사이 시간 간격을 검출**
  - ✓ 4Bit Prescaler 내장(최대 2048분주) → QCLK을 분주하여 Unit Position Event(UPEVNT) 생성
  - ✓ Prescaler를 거친 UPEVNT 사이의 시간 간격을 16Bit 타이머로 검출

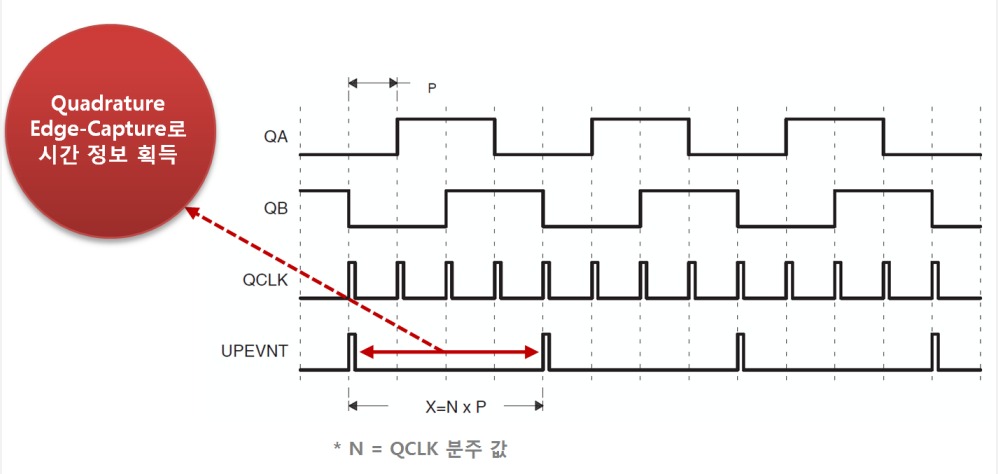
- **두 가지 이벤트 Latch 모드 제공**
  - ✓ CPU가 QPOSCNT 값을 읽을 때
  - ✓ Unit Time Out 이벤트가 발생했을 때
  - ✓ Latch 조건 충족 시
    - QCTMRLAT=QCTMR
    - QCPRDLAT=QCPRD

- **획득한 시간정보의 유효성 판단 필요**
  - ✓ UPEVNT 사이에서 Capture Timer Overflow가 발생 했는가?
  - ✓ UPEVNT 사이에서 Direction이 변경되었는가?

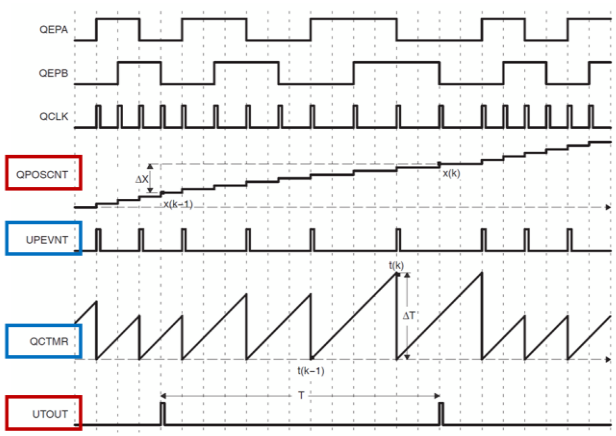


◈ ⑤ Quadrature Edge-Capture - 2

- **QCAPCTL.UPPS = 2(0010[2]) 일 때, UPEVNT 발생 예**
  - ✓ Quadrature Decoder의 QCLK을 4분주



- **EQEP를 이용한 2가지 속도측정 방법**



- **고속에서의 속도 측정**
  - 단위 시간당 위치 변화 이용
  - 저속에서는 단위 시간당 위치 변화 량이 적어 정밀도 저하

$$v(k) = x(k) - x(k-1)/T$$

- **저속에서의 속도 측정**
  - 단위 위치 변화에 소요된 시간정보 이용
  - 저속에서도 높은 정밀도의 속도 산출 가능

$$v(k) = X/t(k) - t(k-1)$$

◈ ⑥ **EQEP Interrupts**

총 11가지 인터럽트 소스가 존재 함.

- **Unit Time Out Interrupt (QFLG.UTO → Set)**
  - ✓ 32Bit Unit Timer(QUTMR)가 설정된 주기 값(QUPRD)과 일치할 경우 발생
- **Index Event Latch Interrupt (QFLG.IEL → Set)**
  - ✓ EQEPI 핀의 신호 Edge가 검출되어 QPOSCNT 값이 QPOSILAT에 기록된 후 발생
- **Strobe Event Latch Interrupt (QFLG.SEL → Set)**
  - ✓ EQEPS 핀의 신호 Edge가 검출되어 QPOSCNT 값이 QPOSSLAT에 기록된 후 발생
- **Position-Compare Match Interrupt (QFLG.PCM → Set)**
  - ✓ QPOSCNT==QPOSCMP 일 때 발생
- **Position-Compare Ready Interrupt (QFLG.PCR → Set)**
  - ✓ QPOSCMP의 Shadow 레지스터 값이 Active 레지스터에 적용된 후 발생
- **Position Counter Overflow Interrupt (QFLG.PCO → Set)**
  - ✓ QPOSCNT==QPOSMAX 일 때 발생 (일치 후 다음 상승 계수 시)
- **Position Counter Underflow Interrupt (QFLG.PCU → Set)**
  - ✓ QPOSCNT==0(Zero) 일 때 발생 (일치 후 다음 하강 계수 시)
- **EQEP Watchdog Time Out Interrupt (QFLG.WTO → Set)**
  - ✓ WDTMR==WDPRD 일 때 발생
- **Quadrature Direction Change Interrupt (QFLG.QDC → Set)**
  - ✓ Quadrature Decoder가 해석한 QDIR이 변경될 때 발생 (방향이 바뀔 때)
- **Quadrature Phase Error Interrupt (QFLG.PHE → Set)**
  - ✓ Quadrature Decoder에 입력되는 두 신호(QEPA, QEPB)의 Edge가 동시에 검출될 경우 발생
- **Position Counter Error Interrupt (QFLG.PCE → Set)**
  - ✓ Position Counter Reset on Index Event 모드에서 QPOSILAT 값이 "0"(Zero) 혹은 QPOSMAX 값이 아닐 경우 발생

6.1 EQEP 모듈로 Incremental 엔코더 신호 해석하기

실습 목적

TMS320F2833x 의 Enhanced QEP Module 을 통해, Incremental 엔코더의 출력신호를 해석해봅니다.

실습 준비

통합 예제 파일을 설치 하셨다면, CCS 버전 별로 다음과 같은 경로에서 예제의 프로젝트 파일을 찾을 수 있습니다.

CCSv3.3용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V330₩EQEP01_Encoder_Pulse_Decoding
CCSv4.2용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V400₩EQEP01_Encoder_Pulse_Decoding
CCSv5.1용 :
C:₩SyncWorks₩DELFINO_EVM_28335₩Example_V500₩EQEP01_Encoder_Pulse_Decoding

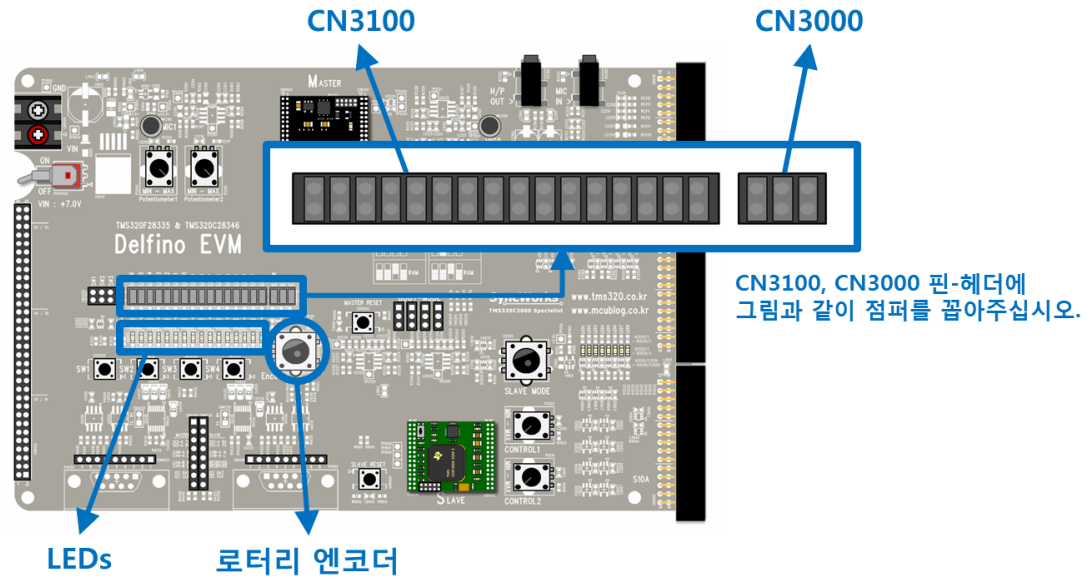
통합 예제 파일이 아닌 개별적인 예제 파일은 다음과 같은 위치에서 다운로드 받을 수 있습니다.

Delfino 개발지원 자료집(CD) 사용자의 경우
"[QEP-01] EQEP 모듈로 Incremental 엔코더 신호 해석하기" 폴더

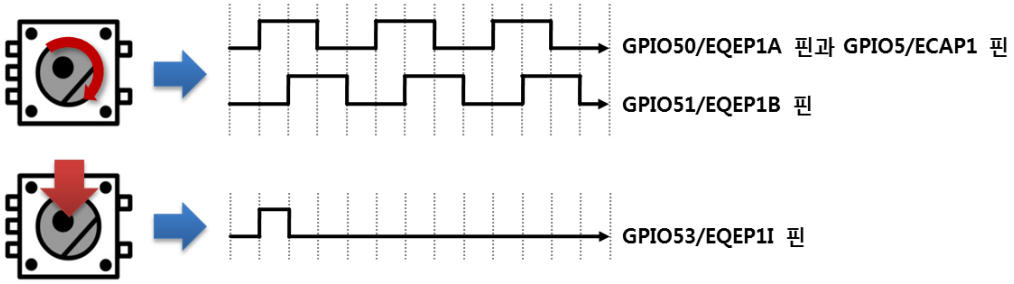
싱크웍스 홈페이지에서 다운받는 경우
"[실습예제] Delfino EVM - EQEP 모듈로 Incremental 엔코더 신호 해석하기" 게시물

실습해 보기

Code Composer Studio에서 Real-time Mode로 예제를 실행하고 델피노 EVM 중앙의 로터리 엔코더(U3000)를 회전시켜봅니다.



Delfino EVM의 로터리 엔코더는 Incremental 방식의 엔코더로 1회전 당 24개의 Quadrature Pulse를 출력하도록 제작되어 있으며, 출력신호 A, B는 점퍼 연결을 통해 TMS320F28335 모듈의 ECAP, EQEP 기능 핀들에 전달되도록 설계되어 있습니다.
(노브 클릭을 통해 스위치로도 활용 가능하며, 이 신호는 EQEP1I 핀에 전달되게 됩니다.)



Delfino 개발보드 예제100선 PART1

2012년 9월 28일 초판 1쇄 발행

저 자	싱크웍스
발 행 인	싱크웍스
발 행 처	도서출판 싱크웍스
	463-760 경기도 성남시 분당구 야탑동 148
	분당테크노파크 B동 501호
전 화	031-781-2812
팩 스	031-706-2834
홈페이지	www.tms320.co.kr
블 로 그	www.mcublog.co.kr
이 메 일	dsptools@syncworks.co.kr
등 록	2002년 10월 29일 제 22-2248호
I S B N	978-89-93343-22-9
정 가	18,000원

* 파본은 바꿔 드립니다.

* 본서의 글, 그림, 내용등은 저작권 보호를 받고 있습니다